

Research Paper

Creating Non-Obvious Innovations

How to Escape the Faster-Horse Trap and Build an Innovation Machine for Products, Systems and Processes

Gonçalo Melo de Magalhães

Independent Researcher, Porto, Portugal

ORCID 0009-0008-6255-7724 · <https://orcid.org/0009-0008-6255-7724>

2 September 2026

Abstract

Innovation requests arrive framed by the current embodiment of a solution: a faster horse rather than the automobile, a better keypad phone rather than the iPhone, a better search box rather than a system that writes the answer, a better oven rather than the air fryer, a faster delivery van rather than the drone. Design fixation has now been measured inside generative systems as well as in human designers. Existing methods escape the embodiment in principle (TRIZ, C-K theory, design by analogy) yet leave four operations unspecified: testing causal necessity, removing the function itself, distinguishing world-admitted from agent-perceived paths, and deciding where abstraction stops. This paper proposes the Freedom Path Innovation Method (FPIM), a causal-path procedure built on the Architecture of Freedom Intelligence (AFI) definition of Freedom as path availability. FPIM represents a product as a use path bound to a desired state by dependencies, applies eight counterfactual probes to a ladder of desired-state descriptions, fixes the actionable causal resolution by two decidable tests, generates alternatives by intervening on binding dependencies, and evaluates candidates ordinally by what they free and block against a competitive frontier normalized to the same desired state. Specified as seven executable steps with stop rules, the method is run on six verified Versuni product families, yielding method-generated hypotheses in four and returning "holds" in two. A pre-registered validation protocol against TRIZ, jobs-to-be-done, C-K/functional analysis and LLM ideation with blind raters is specified; every superiority claim waits for it. The contribution is a methodological synthesis plus a computational representation of innovation as causal path creation, offered with its falsifiers.

Keywords: design fixation; innovation methods; counterfactual intervention; path availability; embodied navigation; product architecture

The paper in one page

The problem. People ask for improvements to the solution they already have. "A faster horse" names a horse; it means transportation, wants speed, tolerates feeding, stabling, waste, fatigue and a 40-km range, and desires arrival. Methods and ideation engines fed with the request produce faster horses.

The evidence. Design fixation is measured in people [1, 2] and, since 2025, inside generative systems [3, 6, 7]: outputs are less diverse than human designs on every metric tested, and a classifier separates the two perfectly.

The gap. TRIZ, C-K theory and analogy escape the embodiment in principle. Six operations stay unspecified everywhere: testing causal necessity, removing the function itself, separating world-admitted from team-perceived paths, creating paths rather than choosing among them, evaluating the option structure left after a design, and deciding where abstraction stops.

The object. Freedom = path availability (AFI). Innovation is a change in which paths are passible, made by changing a dependency.

The method (FPIM), seven steps.

- map: the use path, its agents, its dependencies.
- prune: eight counterfactual probes on a ladder of desired-state descriptions.
- fix the level: two decidable tests — too shallow (the embodiment is still in the description) and too broad (unrelated mechanisms share one capability ledger).
- generate: six dependency changes — substitute, collapse, prevent, delegate, transfer, limit.
- write the ledger: relation changed · frees · blocks · requires.
- compete: twelve ordinal dimensions against the company's best, the best rival, a substitute — verdict moves / holds / loses.
- falsify: the one assumption that would kill the survivor, and the cheapest test.

The results. Six verified Versuni product families: four method-generated hypotheses, two honest losses (the candidate already exists among rivals). Five everyday products and the horse itself dissected the same way.

The test. A pre-registered experiment against TRIZ, jobs-to-be-done, C-K/functional analysis and two LLM baselines with blind raters; the method fails if causal distance rises by less than $d = 0.3$ or usefulness falls by more than $d = 0.3$.

The claim. A methodological synthesis with three procedural additions (stopping rule, path-creation ledger, survival test) and a computational representation of innovation as causal path creation — offered with its falsifiers and with every AFI negative result kept in view.

1. Introduction

A request for "a faster horse" carries horse, transport, present technology and present path inside the request. The desired outcome admits structurally different paths (bicycle, automobile, rail, aircraft), and at other causal levels the outcome itself dissolves: telepresence replaces travel, asynchronous coordination replaces presence, delegated execution replaces coordination (Figure 1). Innovation methods have long recognized this embodiment trap; the question examined here is whether a procedure can separate a desired state from its current embodiment at a level that still constrains product architecture, and do so repeatably. Two failure modes bound the search: too shallow, where the current mechanism still defines the solution space, and too broad, where materially unrelated solutions become indistinguishable and the description stops guiding architecture. A method must decide where between these bounds to stop.

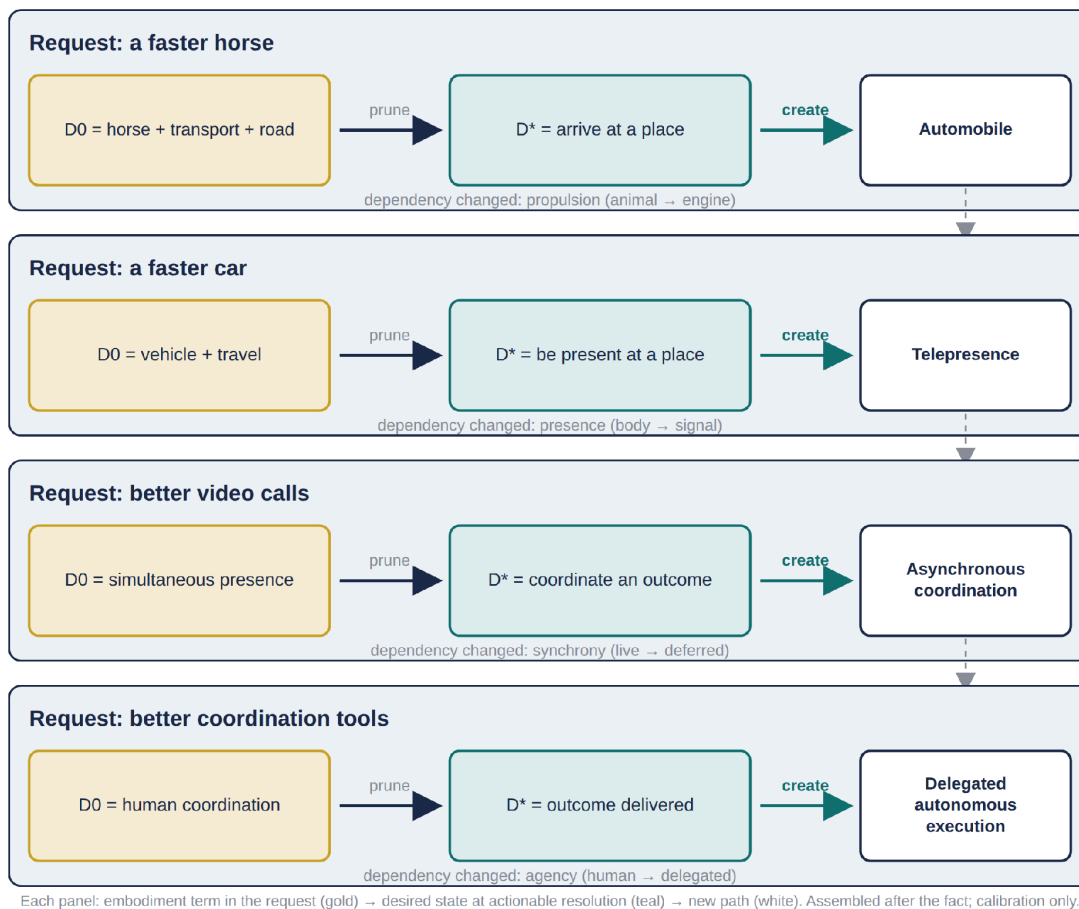
Figure 1 · The embodiment trap: four path creations on one ladder

Figure 1. The embodiment trap. One abstraction ladder, four historical panels (horse → automobile → telepresence → coordination). Each panel shows the embodiment term D_0 in the request and the desired state D^* that opened the next path; arrows carry the dependency changed (propulsion, presence, synchrony, agency). Conceptual; assembled after the fact as calibration.

1.1 What the faster-horse problem is

Definition. The faster-horse problem is the systematic gap between what people ask for and what they need, produced by the fact that people describe improvements to the solution they already have. The request names an embodiment and an attribute; the need is a function, an outcome, and the removal of the obligations the embodiment imposes.

History (the case from 150 years ago).

- The phrase is attached to Henry Ford — "if I had asked people what they wanted, they would have said faster horses". The quotation is undocumented; the situation it names is dated and real.
- 1880: city transport ran on horses — tens of thousands of animals per large city, each needing daily feed and water, a stable, a farrier, veterinary care, rest, and street cleaning of its waste; working range a few tens of kilometres a day.
- 1886: Karl Benz's motor carriage answers the same function with a different embodiment. 1908: Ford's Model T; millions built over two decades; the working horse leaves the cities within a generation.

- Customers of 1880 described the world in the only terms they had — a faster, stronger, cheaper horse. The innovation that served them changed the terms rather than the horse.

Model (Figure 2). A request is the five-tuple $R = \langle e, a, f, O(e), o \rangle$:

- e — the embodiment named: horse.
- a — the attribute wanted: faster.
- f — the function meant: move a person or a load.
- $O(e)$ — the obligations e imposes, left unspoken: feeding, stabling, waste, fatigue, range.
- o — the outcome desired: arrive at a distant place within an acceptable time.

Two searches follow from the tuple:

- Improvement within the request keeps e and raises a . Output: faster horses. Every obligation in $O(e)$ stays.
- Innovation on the need holds f and o fixed, treats $O(e)$ as the list of dependencies a new embodiment may remove, and searches for an e' with a different $O(e')$. Output: engine, pedal, rail, wire.

The faster-horse problem is the failure to move from the first search to the second; its cost is every path only the second search reaches. The rest of the paper builds the second search as a procedure: the five layers of a request (Section 2.2), the ladder and probes that find where f and o separate from e (Sections 2.4 and 5.3), the dependency changes that generate e' (Sections 2.5 and 5.4), and the ledger of what each e' frees and blocks (Section 5.5).

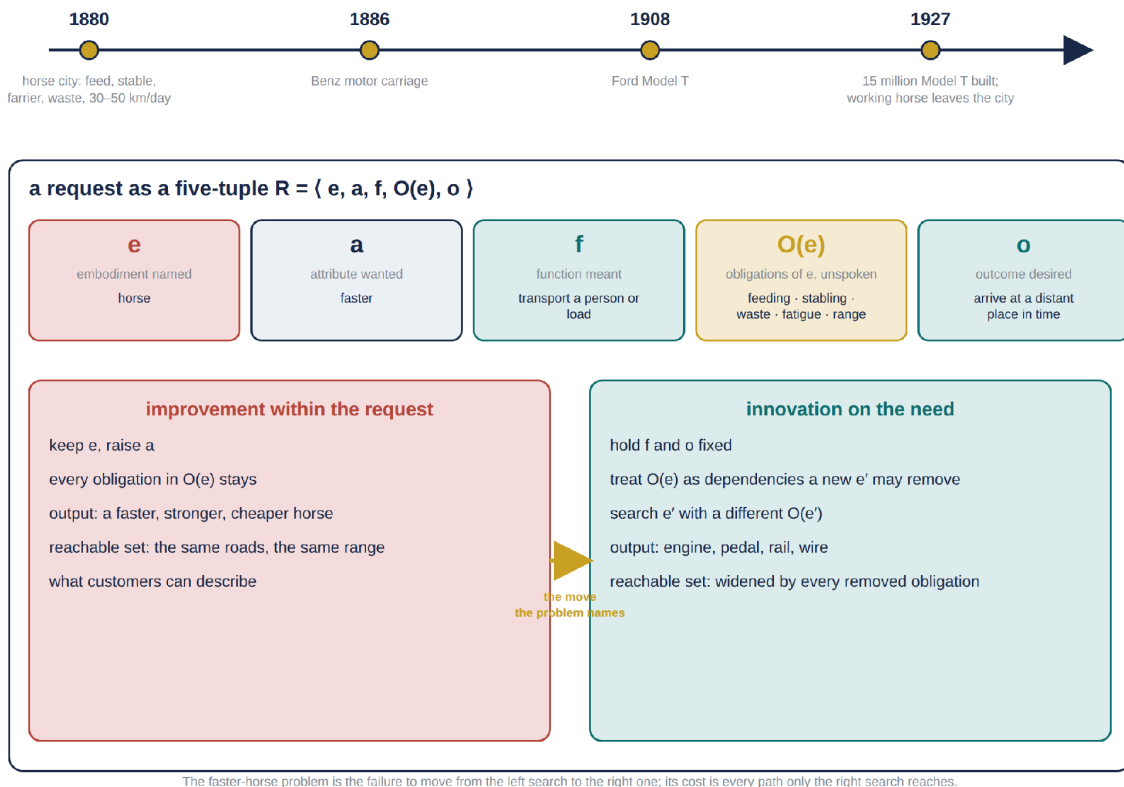
Figure 2 · The faster-horse problem, defined

Figure 2. Which obligations each alternative removed. Rows: the obligations of the horse system; columns: bicycle, railway, automobile, telegraph; a filled cell marks an obligation removed. Different subsets explain why the alternatives coexisted. Historical, illustrative.

1.2 Design fixation: what it is, and how it poisons innovation

Definition. Design fixation is the tendency of a designer to reproduce features of a solution they have been exposed to — including its flaws — when generating new solutions, and to search inside the space that solution defines [1, 76]. It is the cognitive mechanism behind the faster-horse problem: the horse is the exposed example, and the request is the fixated search.

Where the effect comes from, in the order it was discovered:

- **Functional fixedness (1945):** people fail to see a familiar object used in an unfamiliar way — the box of tacks that could be a candle shelf — because the object's usual function occupies its representation [74].
- **Einstellung, the mechanization of thought (1942):** a procedure that worked on the last problems is applied to the next one even when a simpler route exists; success trains the fixation [75].
- **Structured imagination (1994):** asked to invent a new animal, people reproduce the category structure they know — symmetry, eyes, legs — because generation follows the path of least resistance through existing categories [77].
- **Design fixation proper (1991):** engineers shown an example design reproduce its features, flaws included, in their own concepts, and the effect survives explicit instructions to avoid it [1]; the effect has a thirty-year replication record and an agenda for tooling and training [2, 76].
- **Fixation inside the machines (2024–2026):** generative systems reproduce the distribution they were trained on — chairs designed with GPT-4o and Midjourney showed the human pattern [3];

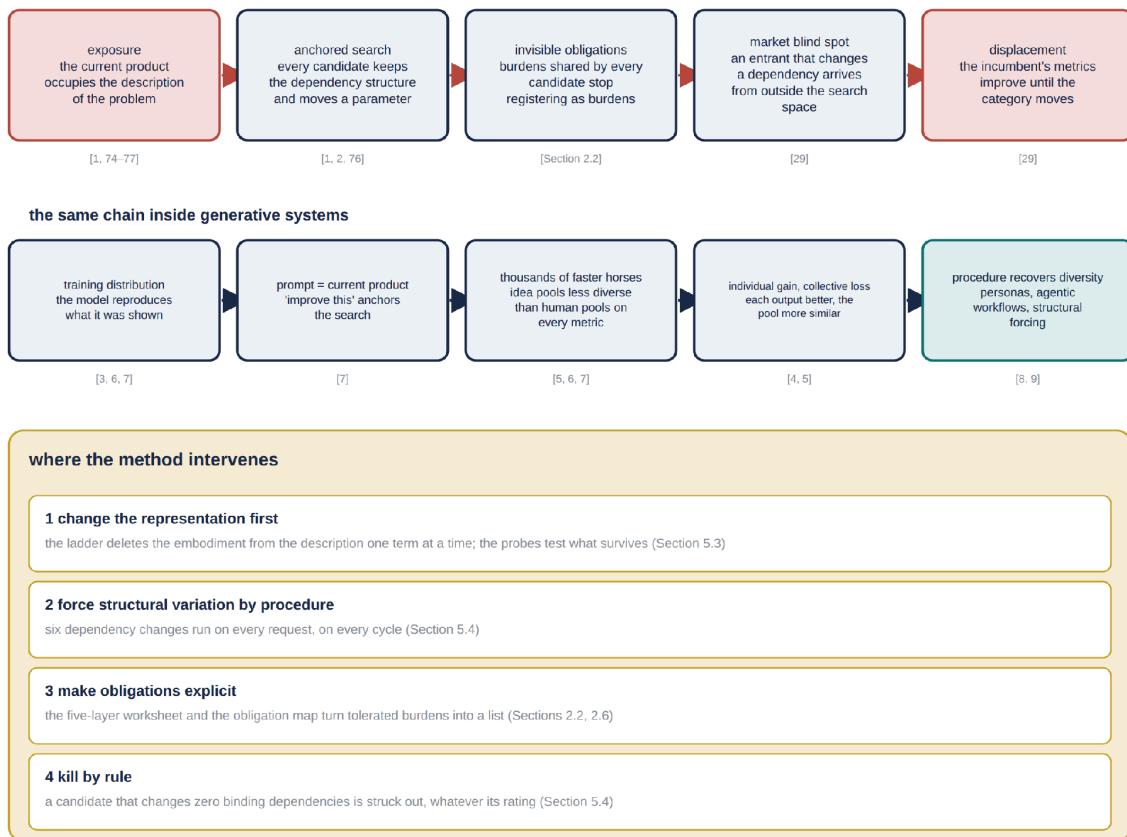
language-model idea pools are less diverse than human pools on every metric and a classifier separates the two perfectly [6, 7]; assisted writers gained individually and lost collectively [4, 5].

How fixation poisons innovation (Figure 3):

- Exposure anchors the representation: the current product occupies the description of the problem, so the description already contains the answer.
- The search stays inside the embodiment: every candidate keeps the dependency structure and moves a parameter — the ten-percent roadmap.
- The obligations stay invisible: burdens shared by every candidate stop registering as burdens, so the paths that remove them stay ungenerated.
- The market blind spot follows: an entrant that changes a dependency arrives from outside the search space, and the incumbent's metrics keep improving until the category moves [29].
- Scale multiplies the effect: an ideation engine fed the current product produces faster horses in thousands, with the diversity loss measured [6, 7].

Why a method, rather than advice, is the remedy:

- Fixation acts on the representation before any idea is generated, so "think outside the box" arrives too late; the representation itself must be changed first. The ladder of Section 5.3 deletes the embodiment from the description one term at a time, and the probes test what survives.
- Fixation survives instruction [1]; it yields to procedure: multi-persona prompting and agentic workflows raise diversity where a bare prompt lowers it [8, 9]. A method is a procedure that forces structural variation on every run.
- Fixation hides in obligations, so a method must make obligations explicit — the five-layer worksheet of Section 2.2 and the obligation map of Section 2.6 do exactly this.
- Fixation is invisible to the fixated, so a method needs a rule that kills its products automatically: any candidate that changes zero binding dependencies is struck out (Section 5.4).

Figure 3 · How fixation poisons innovation, and where the method intervenes

Fixation acts before any idea is generated; advice arrives too late, procedure arrives in time.

Figure 3. Where existing methods stop. Abstraction ladder $D_0 \dots D_n$ on the horizontal axis; one bar per method family ending where its own procedure stops (functional analysis, TRIZ, C-K, jtbd/laddering, LLM ideation); the band marked "actionable causal resolution" is the level the stopping rule targets. Derived from the Phase-1 gap matrix [21, 10, 11, 13, 7].

The problem has fresh empirical weight. Design fixation — reproduction of features of an exposed example, flaws included — is an established effect [1, 2]. In 2025 it was reproduced inside generative systems: chairs designed with GPT-4o and Midjourney showed fixation analogous to human designers, and image-model outputs were significantly less diverse than award-winning designs [3]. Across writing, brainstorming and engineering concept generation, generative assistance raises individual output quality while lowering collective diversity: ai-assisted stories were about 10.7% more similar to each other [4]; ChatGPT-assisted brainstorming produced a lower share of unique ideas across all tasks [5]; across 102 humans and 22 language models, model response variability was far lower than human at matched originality [6]; 4,000 GPT-4 design concepts were less diverse than 100 human solutions on every metric regardless of temperature or prompting, and a classifier separated the two perfectly [7]. Procedure recovers part of the loss — multi-persona prompting [8] and agentic workflows (design-space coverage +11.4%, diversity +18.5%) [9] — which indicates that the surrounding method, rather than the model, decides whether generation escapes the training distribution. As ideation shifts toward such engines, a procedure that separates desired state from embodiment before generation gains relevance in proportion to that shift.

Existing methods address parts of the problem. TRIZ trims components and states an ideal final result [10], C-K theory expands concepts beyond current knowledge [11], design by analogy re-represents the problem to reach distant source domains [12], and jobs-to-be-done fixes the job rather than the product [13]. A twenty-column comparison of thirteen method families (Phase 1 of this programme, summarized in Section 3) shows that escape from embodiment is solved in principle

and that a residual gap remains at one intersection: (i) testing whether a requirement is causally necessary rather than assuming it from interviews or function trees; (ii) removing the function itself as a legitimate move; (iii) separating paths the world admits from paths the design team perceives; (iv) representing innovation as creation of paths rather than choice among perceived ones; (v) evaluating the option structure left after a design is realized; and (vi) an explicit stopping rule for abstraction depth. Interventionist causation [14, 15], empowerment and option preservation [16–18], viability theory [19] and learned world models for robot navigation [20] each supply one of these elements; a design procedure that joins them was absent from the literature searched (Table 1, Figure 16).

Research question. Can a repeatable procedure (a) fix the causal resolution at which a product's desired state separates from its embodiment while still constraining architecture, (b) generate structurally different paths from that resolution, and (c) evaluate them by their effect on path availability rather than by feature comparison?

Contribution. The paper offers a methodological synthesis rather than a new theory of design: the Freedom Path Innovation Method (FPIM), an executable seven-step procedure with a decidable stopping rule, a formal path-creation operation with a frees/blocks ledger, a competitive survival test normalized to the desired state, worked runs on six verified product families of one manufacturer, and a pre-registered validation protocol against strong baselines. AFI supplies the design object — path availability — and the world-versus-perceived distinction; prior art on option preservation is cited as ancestry. All generated products are labelled method-generated hypotheses; every superiority claim waits for the protocol in Section 7.

How the paper connects. Figure 4 draws the chain of the argument: fixation evidence (Section 1) motivates a procedure; the faster-horse chapter (Section 2) says what a non-obvious path is and how it is dissected; the gap in existing methods (Section 3) says what the procedure must add; AFI (Section 4) supplies the object the procedure works on — path availability — and the distinction between paths the world admits and paths a team perceives; the method (Section 5) is the dissection turned into seven executable steps; the worked cases (Section 6) run the steps on real products; the validation protocol (Section 7) states how the method can lose. Every later section uses the terms defined in the earlier ones, and the glossary at the end collects them.

Figure 4 · How the paper connects

Each section uses the terms defined in the earlier ones; the right column names what each section hands to the next; the glossary collects every term.

Figure 4. Four historical path creations as one ledger. Horse → automobile, ice trade → refrigeration, courier → telegraph, film → digital photography: desired state held, dependency changed, obligations removed, obligations installed. Historical, illustrative.

2. The faster-horse problem: why non-obvious paths matter and how to dissect them

Section 1 named the trap and its mechanism. This section takes the horse apart, layer by layer, so that the reader sees where a non-obvious path was hiding in 1880 and where it hides in any request today; Sections 5 and 8 turn the same moves into a procedure and a protocol.

2.1 Why the problem matters

Three facts make the parable of Section 1.1 a research problem rather than an anecdote. First, every request for a product carries an embodiment inside it, because the person asking can only describe what they have seen: a faster horse, a quieter mower, a bigger battery. Designers, interviews and function trees inherit the embodiment through the same channel, so the methods that promise to escape it are fed by the very descriptions that contain it. Second, the tendency is measured, in people and now in machines: exposure to an example reproduces its features, flaws included [1, 2]; generative systems reproduce the distribution they were trained on, with lower diversity than human designers on every metric tested and a classifier that tells the two apart perfectly [3, 6, 7]. An ideation engine fed with the current product and asked for improvements produces faster horses at scale. Third, the difference between an increment and a path creation is where category outcomes are decided: firms that improved the incumbent path were displaced by entrants on a different one, in a pattern documented across disk drives, steel and retailing [29]. The horse breeders of 1900 improved horses; the reachable set of their customers moved to a different dependency structure.

2.2 What people say and what they mean

Key points.

- "Horse" was the word; transportation was the meaning.
- "Faster" was the attribute; "arrive sooner, spend less of the day travelling, reach farther" was the need.
- The largest part of the request was silent: the problems associated with horses, tolerated because every alternative shared them.
- Non-obvious paths live in the silent part.

A request has five layers (Figure 5), and a good dissection reads all five:

- Object named — horse.
- Attribute wanted — faster.
- Function meant — transport of a person or a load.
- Obligations tolerated — feed on idle days, stable, farrier, veterinary care, waste removal, rest, replacement, a range of tens of kilometres.
- Outcome desired — arrive at a distant place, on time, safely, with little of the day spent on it.

Where methods fail:

- Stopping at the first two layers produces faster horses.
- Jumping to the last layer alone produces "mobility" and "wellbeing" — words that leave an engineer empty-handed.
- The dissection works in the middle three: function + obligations + outcome define the desired state; each obligation is the visible trace of a dependency that a different embodiment could remove.

Read this way, "faster horses?" means: better transportation, by any means, with the problems associated with horses removed. The customer has already stated the function, the direction of improvement and — by omission — the list of dependencies a new path may dissolve. Each historical answer took a different subset:

- Automobile — same function, large gain on the attribute, animal obligations removed, fuel and road obligations installed.
- Bicycle — short range, one person, zero feeding.
- Railway — long range, many people, fixed routes and timetables.
- Telegraph — the case where the traveller carried only information.

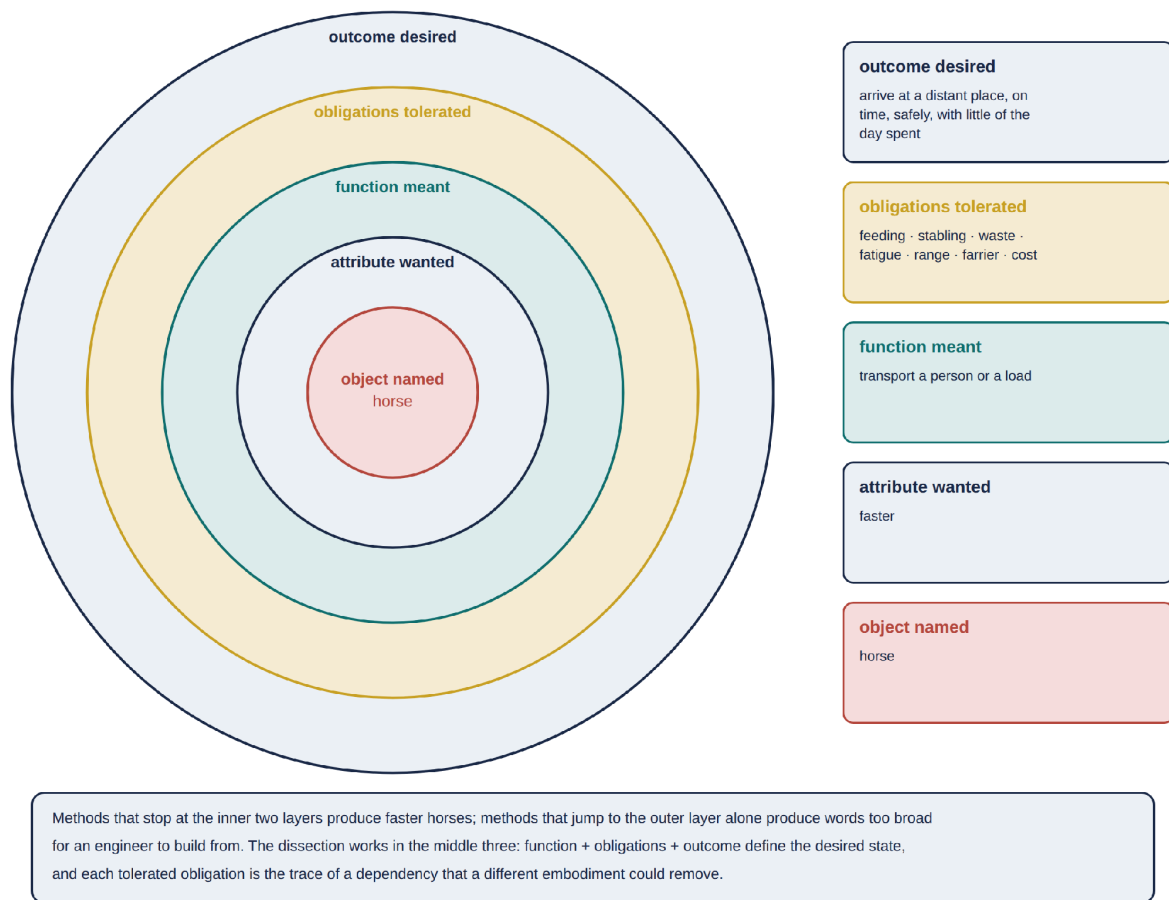
Figure 5 · What people say and what they mean: the five layers of a request

Figure 5. Worked case: Home Safety. Current path strip; seven operator outputs as nodes; kills crossed with their reason; the survivor ledger (frees / blocks / capabilities / new dependencies) for the Interlocked Home hypothesis. Method-generated; product facts from [70].

2.3 What makes a path non-obvious

Key points.

- Obvious: same dependency structure as the current product, one or more parameters improved.
- Non-obvious: the same desired state reached through a different dependency structure.
- Non-obvious is structural, and distinct from surprising: a surprising idea can keep every dependency; a plain-looking path can change the one relation that binds the system.
- The test is mechanical: which binding dependency does the candidate change? Zero changes = feature addition.

The same pattern, four times in history (Figure 8):

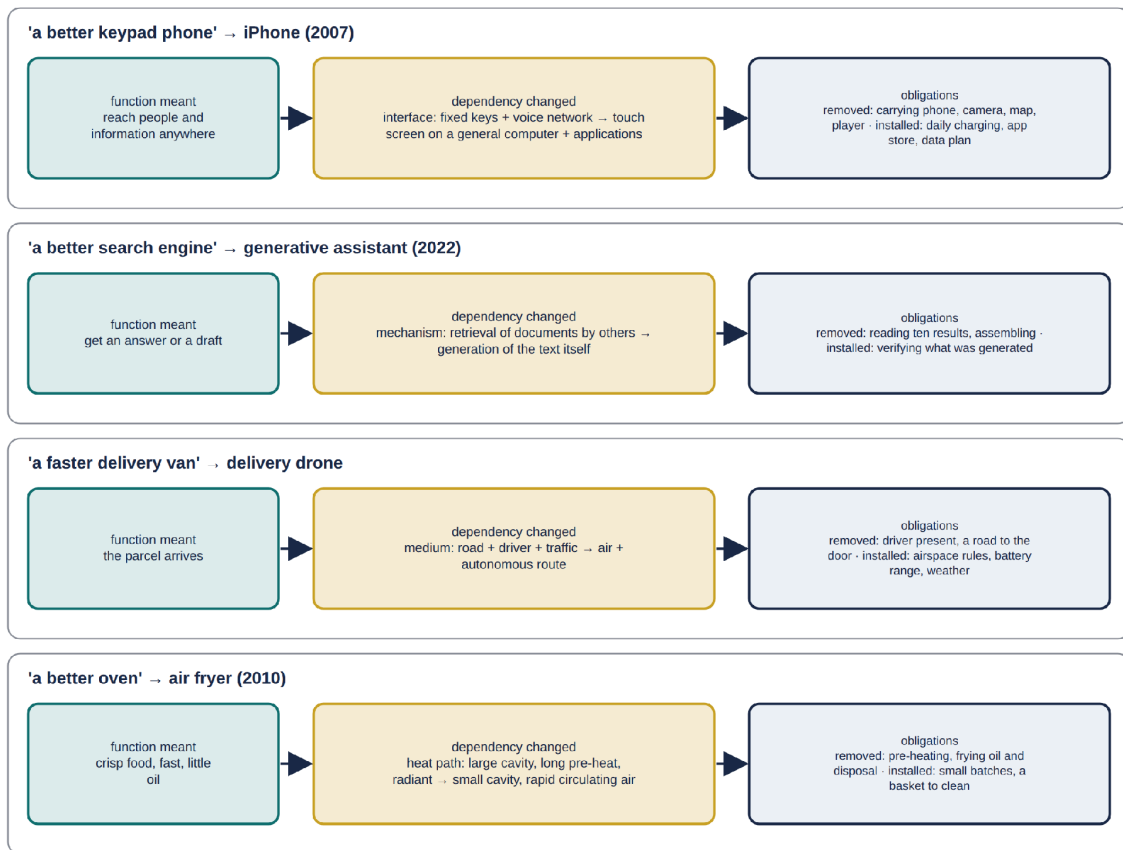
- Horse → automobile: propulsion by animal → propulsion by engine; feeding, stabling, waste, fatigue removed; fuel, roads, mechanics installed.
- Ice trade → refrigeration: cold from a distant frozen lake and a delivery route → cold from electricity at the point of use; harvesting, storage and delivery removed; a power supply installed.
- Letter by courier → telegraph: the message carried by a body → the message carried by a signal; the speed of a horse stops bounding the speed of information; wire, operator and text-only installed.

- Film → digital photography: image held by chemistry and a lab → image held by a sensor and a screen; darkroom obligations removed; batteries, storage and screens installed.

The same pattern in living memory (Figure 6):

- "A better keypad phone" → the iPhone (2007): the request named keys and calls; the function meant was reaching people and information anywhere; the dependency changed was the interface — fixed keys and a voice network → a touch screen on a general computer with a network of applications; obligations removed: carrying a phone, a camera, a map and a music player; installed: charging daily, an app store, a data plan.
- "A better search engine" → generative assistants (2022): the request named ranking; the function meant was getting an answer or a draft; the dependency changed was the mechanism — retrieval of documents written by others → generation of the text itself; obligations removed: reading ten results, assembling the answer; installed: verifying what was generated.
- "A faster delivery van" → the delivery drone: the request named the vehicle; the function meant was the parcel arriving; the dependency changed was the medium — road, driver and traffic → air and an autonomous route; obligations removed: a driver present, a road that reaches the door; installed: airspace rules, battery range, weather limits.
- "A better oven" → the air fryer (2010): the request named the cavity; the function meant was crisp food, fast, with little oil; the dependency changed was the heat path — a large cavity with long pre-heat and radiant heat → a small cavity with rapid circulating air; obligations removed: pre-heating, deep-frying oil and its disposal; installed: small batches, a basket to clean.
- In every case the request named the old embodiment, the answer changed one dependency, and the winners were the companies that dissected the request rather than improved the object.

Figure 6 · Four path creations in living memory



In every case the request named the old embodiment; the answer changed one dependency. Dates from public record; illustrative.

Figure 6. The dissection protocol for any concept. Fourteen steps from the verbatim request to the three audit questions, with the quality gate at each step and the section of the paper that defines it. Derived from Table 6.

The recognition test (Figure 9):

- Does the candidate change a binding dependency? Zero → feature addition, struck out.
- Which of the six changes is it — substitute, collapse, prevent, delegate, transfer, limit?
- Does it survive the probes at the actionable level?
- Does it free more than it blocks against the frontier?
- Only a candidate that passes all four is a non-obvious path; a candidate that passes the first and fails the rest is a surprising decoration.

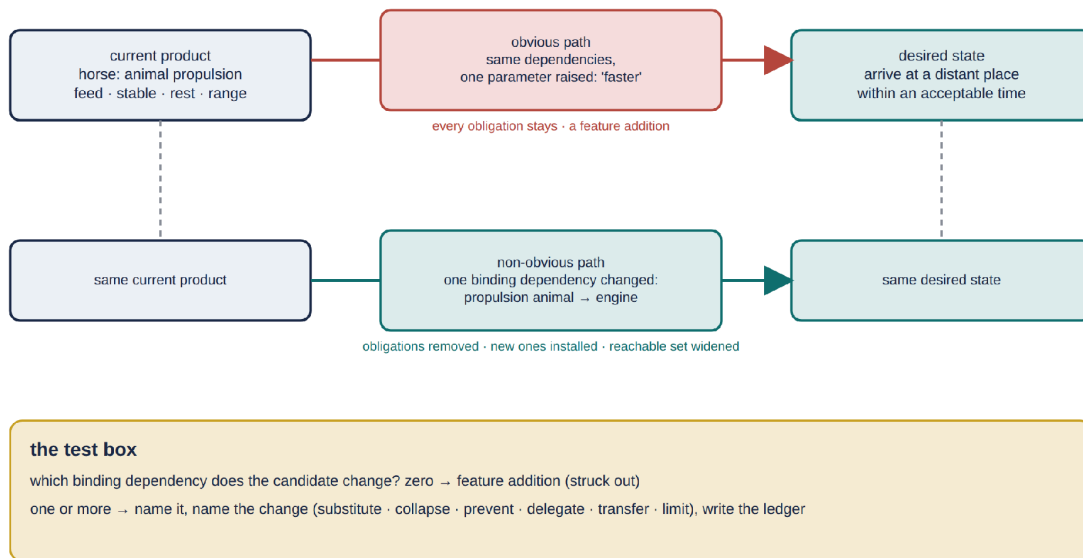
Figure 7 · Obvious versus non-obvious paths to the same desired state

Figure 7. Hot water that precedes the wish. From "a faster kettle" to a routine-driven reservoir at the tap: layers, desired state, dependency changed, the loop, frees and blocks, falsifier. Method-generated hypothesis; the instant tap exists among rivals, the anticipation is the added element.

Figure 8 · Four historical path creations as one ledger

	horse → automobile	ice trade → refrigeration	courier → telegraph	film → digital
desired state held	arrive at a distant place in time	keep food cold	the message arrives	keep the image
dependency changed	propulsion: animal → engine	cold source: distant lake + route → electricity at the point of use	carrier: a body → a signal	medium: chemistry + lab → sensor + screen
obligations removed	feeding · stabling · waste · fatigue · range	harvesting · ice house · delivery · melt loss	rider · horse · days of travel	darkroom · chemicals · waiting · prints
obligations installed	fuel · paved roads · mechanic · licence	power supply · a machine to maintain	wire · operator · text only	battery · storage · screen

Same four lines every time: the desired state holds still, one dependency changes, a set of obligations vanishes, a new set arrives. Historical, illustrative.

Figure 8. now → next → limit and the validation loop. Left: six families placed on now/next/limit, four converging into the shared homeostatic loop and two staying device-local. Right: the validation protocol as a loop (briefs → six methods → blind rating → contrasts → failure threshold). Derived from Sections 6–7; the empirical slot is unfilled.

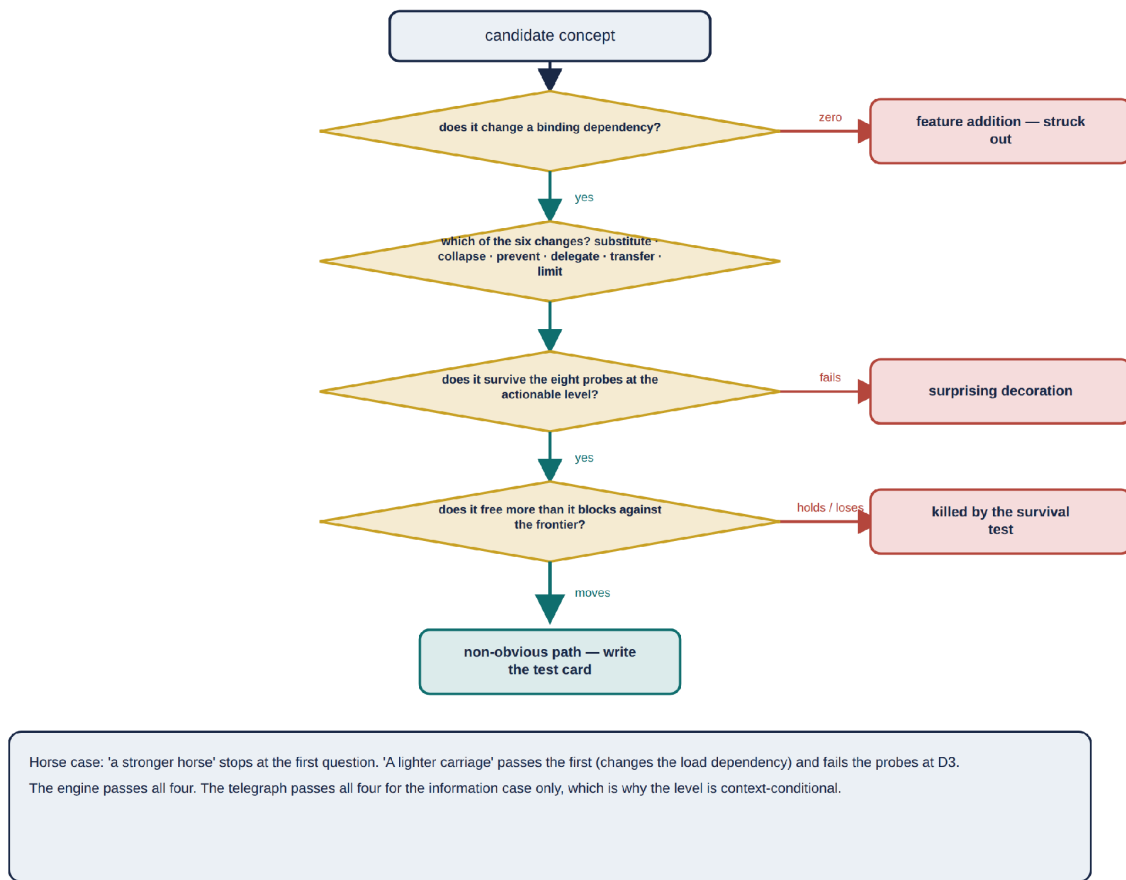
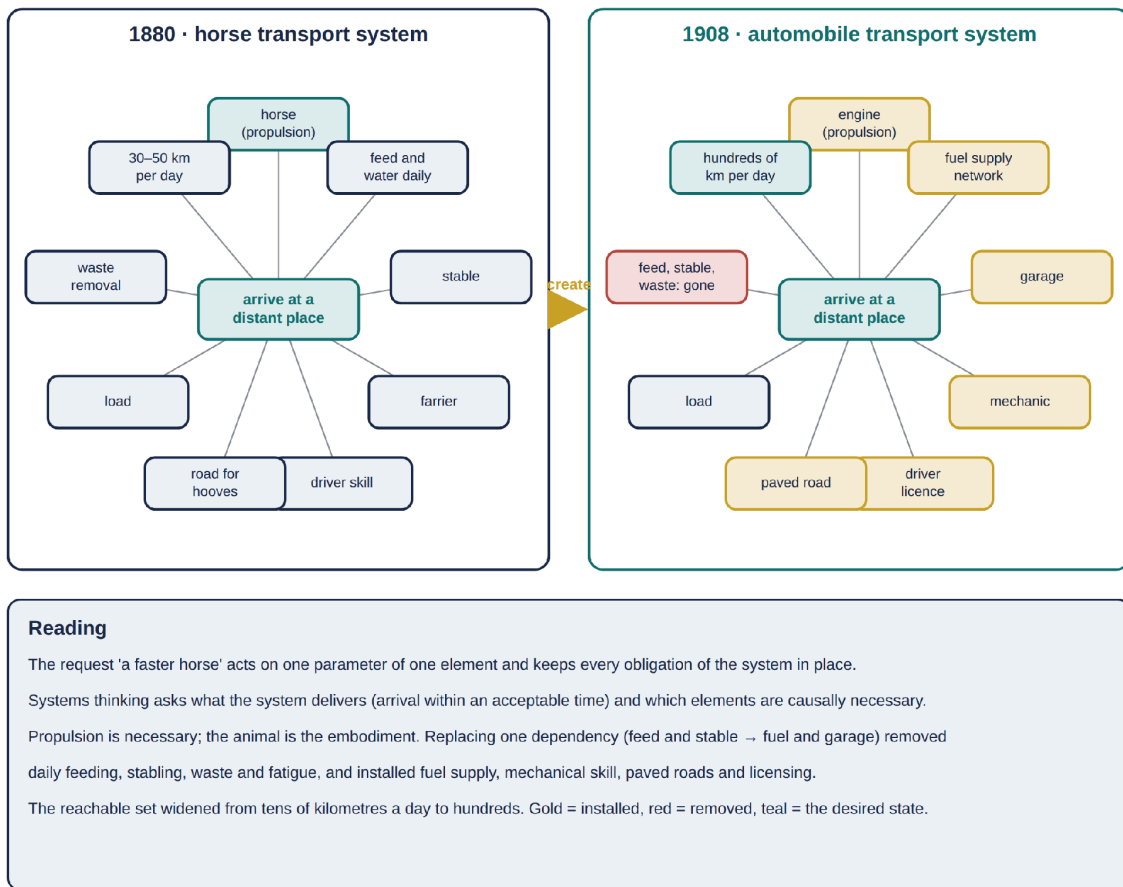
Figure 9 · The recognition test for a non-obvious path

Figure 9. Four path creations in living memory. Keypad phone → iPhone, search engine → generative assistant, delivery van → drone, oven → air fryer: the request, the function meant, the dependency changed, the obligations removed and installed. Illustrative; dates from public record.

2.4 The dissection of the horse, step by step

Figure 10 draws the transport system before and after the substitution of propulsion; Figure 11 runs the eight probes on the horse ladder.

Figure 10 · The faster-horse problem as a system, before and after**Figure 10. The obligation map: where paths hide.** Transitions of a use episode (bean-to-cup coffee) against nine kinds of human burden; filled cells mark obligations; the densest rows and columns are where a dependency change pays most. Illustrative.

The ladder and what each level survives:

- D_0 "a faster horse" — fails every probe; the object is the request.
- D_1 "faster horse-drawn transport" — survives replace (a lighter carriage) and delegate (a coachman); fails the rest.
- D_2 "faster land transport" — survives replace (engine, pedal, rail), bypass (a boat), delegate, agent-shift, context-shift; fails remove and goal-removal. The description is free of the animal and still names something people pay for: the shallow test clears.
- D_3 "arrive at a distant place within an acceptable time" — survives every probe except goal-removal; road vehicle, rail vehicle, boat and aircraft each need a different capability ledger, so the broad test stays false: actionable for bodies.
- D_4 "be present at a distant place" — survives all eight probes; a car and a telephone satisfy it with different ledgers: actionable for information.
- D_5 "achieve the outcome for which presence was wanted" — car, telephone, letter and delegation share one ledger ("some channel to the outcome"): the broad test trips.
- Result: two actionable levels, reported as context-conditional on whether the traveller carries a body or only information.

Figure 11 · The horse ladder under the eight probes

	remove	replace	bypass	prevent	delegate	context shift	agent shift	goal removal	tests
D0 a faster horse	×	×	×	×	×	×	×	×	S=T B=F
D1 faster horse-drawn transport	×	✓	×	×	✓	×	×	×	S=T B=F
D2 faster land transport	×	✓	✓	×	✓	✓	✓	×	S clears S=F B=F
D3 arrive at a distant place within an acceptable time	✓	✓	✓	✓	✓	✓	✓	×	S=F B=F
D4 be present at a distant place	✓	✓	✓	✓	✓	✓	✓	✓	S=F B=F
D5 achieve the outcome for which presence was wanted	✓	✓	✓	✓	✓	✓	✓	✓	B trips S=F B=T

Reading

D0 collapses under every probe: the object is the request. D1 survives replace (lighter carriage) and delegate (coachman).

D2 is free of the animal and still names something people pay for: the shallow test clears.

D3 survives all but goal-removal and still separates road, rail, boat and aircraft by their capability ledgers: actionable for bodies.

D4 survives every probe; a car and a telephone satisfy it with different ledgers: actionable for information.

D5 is where car, telephone, letter and delegation share one ledger ("some channel to the outcome"): the broad test trips.

Result: two actionable levels, reported as context-conditional on whether the traveller carries a body or only information.

Figure 11. How the paper connects. The chain from fixation evidence to gap, to path availability, to the seven steps, to worked cases, to the validation protocol, with the section that carries each link and the term each link hands to the next. Conceptual.

Typing the requirements (rules in Section 5.3):

- propulsion — survives every probe: necessary.
- the animal — fails replace: architectural (one embodiment of propulsion).
- daily feeding — fails remove once the animal is removed: derived.
- the road — fails context-shift (a river changes it): context-conditional.
- the driver — fails delegate: a preference in 1880, a legacy today.
- range per day — fails replace: derived from the animal's stamina.

The typed list is the binding dependency set Δ^* of the horse system. Figure 12 shows which of these obligations each historical alternative removed — bicycle, railway, automobile and telegraph each deleted a different subset, which is why they coexisted.

Figure 12 · Which obligations of the horse system each alternative removed

	bicycle	railway	automobile	telegraph
feed and water daily	removed	removed	removed	removed
stable and muck out	removed	removed	removed	removed
farrier and veterinary	removed	removed	removed	removed
rest every 30–50 km	kept	removed	removed	removed
replace the animal	removed	removed	removed	removed
driver skill and presence	kept	removed	kept	removed
road for hooves	kept	removed	kept	removed
carry the traveller's body	kept	kept	kept	removed
cost per idle day	removed	removed	removed	removed
obligations installed instead	legs; short range	timetable; fixed route	fuel; mechanic; paved road	wire; operator; text only

Each alternative deleted a different subset of the horse's obligations and installed its own — which is why they coexisted rather than replacing one another. Historical, illustrative.

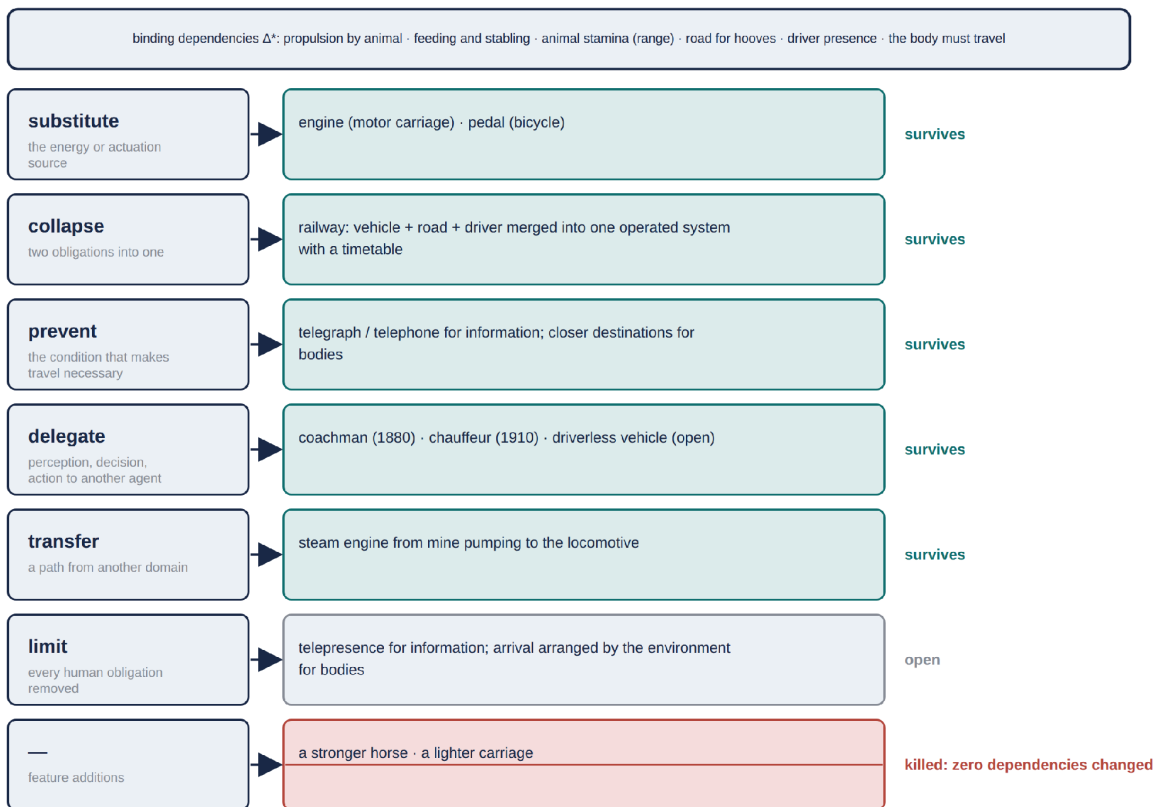
Figure 12. The frees/blocks competitive frontier. Twelve-dimension ordinal grid for one family: rows candidate, same-company analogue, direct rival, cross-category substitute; cells better/equal/worse; verdict moves/holds/loses in the margin. Derived from Section 5.6 applied to Table 3.

2.5 Generating the alternatives from the dissection

Figure 13 shows the population the six dependency changes produce from Δ^* , with kills:

- Substitute the energy or actuation source → engine; pedal.
- Collapse obligations → the railway (vehicle, road and driver merged into one operated system with a timetable); later the bus.
- Prevent the condition that makes travel necessary → telegraph and telephone for information; closer destinations for bodies.
- Delegate perception, decision and action → coachman (1880), chauffeur (1910), driverless vehicle (open).
- Transfer a path from another domain → the steam engine, from mine pumping to the locomotive.
- Limit — every human obligation removed → telepresence for information; for bodies, arrival arranged by the surroundings rather than by the traveller (open).
- Killed: "a stronger horse", "a lighter carriage" — feature additions; every binding dependency stays in place.

This population is what a design team of 1880 would have obtained by running the dissection rather than by asking customers; the same population is obtainable today for any product by the same steps.

Figure 13 · The alternatives population for 'faster horse'

Each candidate names the dependency it changes; a candidate that changes none is a feature addition and is struck out. Derived from Section 2.4.

Figure 13. Five everyday products on one ladder. Umbrella, kettle, toothbrush, lawn mower and alarm clock, each with the too-shallow level, the actionable level chosen by the two tests, the too-broad level, and one candidate path. Derived from Table 5; illustrative.

2.6 Where non-obvious paths hide: the obligation map and the funnel

- Non-obvious paths hide in the obligations a user tolerates. Figure 14 maps them for one product: each transition of the use episode against the nine kinds of burden (perceive, decide, remember, move, operate, verify, clean, maintain, pay). The dense cells are where a dependency change pays most.
- The whole dissection is a funnel (Figure 15): one request → five layers → a ladder of six levels → eight probes per level → a handful of binding dependencies → six changes → a population of candidates → kills → survivors → one test. The horse case gives the counts.

Figure 14 · The obligation map: where paths hide (bean-to-cup coffee)

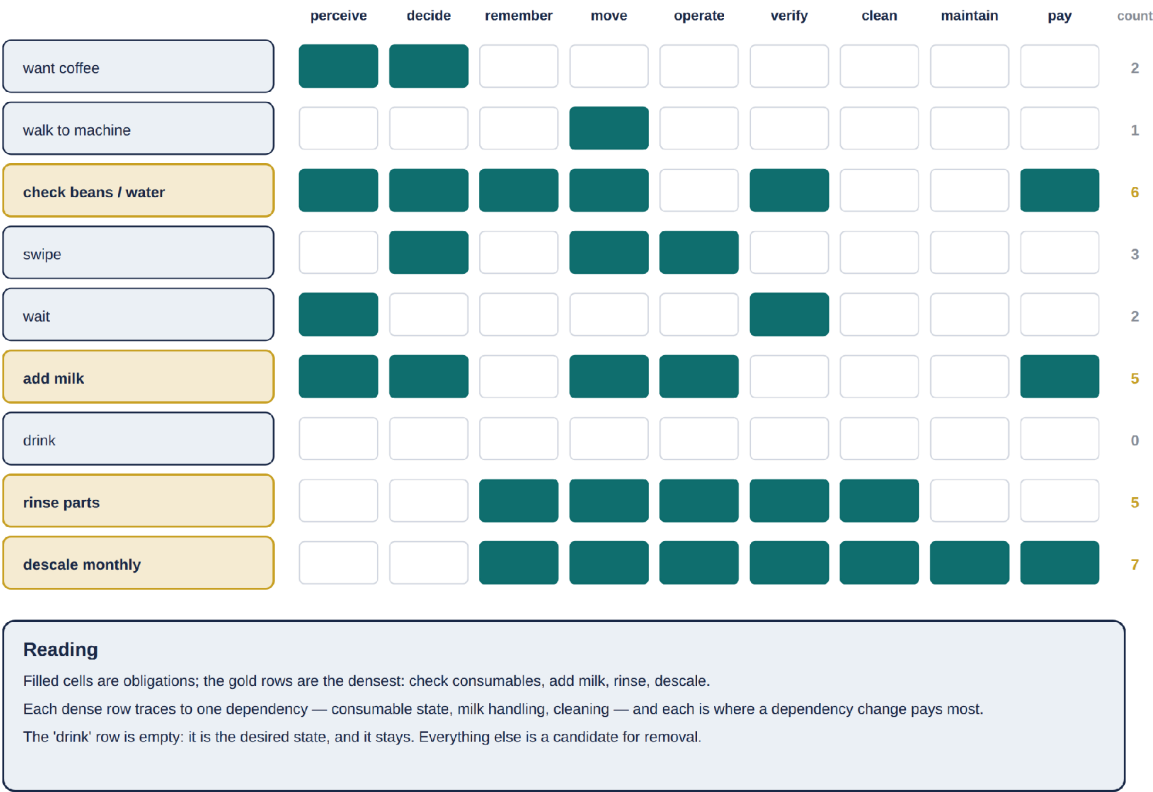
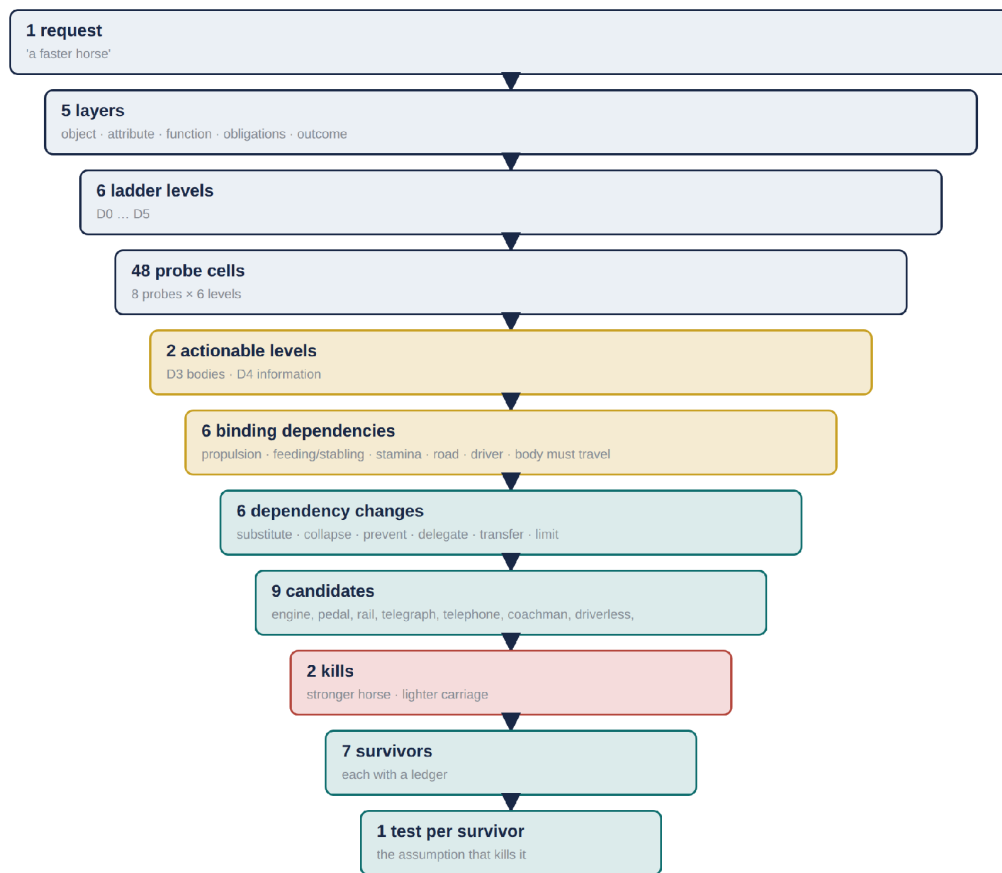


Figure 14. What people say and what they mean: the five layers of a request. Concentric layers around "a faster horse": the object named, the attribute wanted, the function meant, the obligations tolerated, the outcome desired; the dissection works in the middle three. Conceptual.

Figure 15 · The request-to-path funnel, with the horse counts

Counts from the horse case (Section 2). The funnel narrows by tests rather than by taste; every kill carries its reason.

Figure 15. Navigate versus create. The same loop with one added box, create(p, d). Left: Δ fixed, paths varied (navigation). Right: D^* fixed, Δ varied (innovation), with frees and blocks fans drawn from the changed relation. Derived from Section 5.5.

2.7 Why dissection needs a rule

- Left to itself, dissection stops where the analyst tires or where a value word appears — both endpoints lose the architecture.
- Too shallow: the horse stays inside the answer. Too broad: car, telephone and delegation collapse into "achieve the outcome".
- Section 5 supplies a stopping rule with two decidable tests and a measured reliability; Section 8 states the dissection as a protocol for any concept.

3. Related work and the gap

The dissection of Section 2 used six operations. This section asks which of them existing methods already perform, so that the paper claims only what is missing.

Key points.

- Escape from the embodiment is solved in principle by TRIZ, C-K theory, analogy and jobs-to-be-done.
- Six operations stay unspecified across all thirteen method families examined: causal-necessity testing, function removal, world-versus-perceived paths, path creation, future option structure, an explicit stopping rule.

- Causal-reasoning approaches supply intervention semantics and leave the design procedure open; agency and robotics literatures hold the option-structure content with a design-method counterpart still unlocated.
- The gap is narrow and demonstrable; it dies if any one method occupies the intersection, and the search found it empty (Table 1, Figure 16).

3.1 Design methods and the embodiment

Functional analysis remains the main bridge from product to abstraction; a 2026 PRISMA review of forty studies finds four competing approaches (black box, function–means tree, systems architecture, hybrid), only 20% tool-supported, with scalability, repeatability and "systematicity versus creativity" as open problems [21]. FAST and value engineering ascend a function tree by asking "why" and descend by asking "how"; the ascent stops where the analyst stops. The function–behaviour–structure framework reformulates design situations across three variables with reformulation loops and leaves causal necessity unmodelled [22]. TRIZ presupposes a formulated contradiction, resolves it by inventive principles and trims components toward ideality; its 2025 automation [23] and its critics [24] agree that it omits a problem-redefinition step and fails wherever a clear contradiction is absent. C-K theory expands a concept space against a knowledge space [11]; LLM agents executing C-K operators showed concept-to-concept transitions dominating with diversity trending downward, exposing a gap in the theory's account of expansion [25]. Jobs-to-be-done and outcome-driven innovation fix the job by interview and rank outcomes by opportunity score [13]; means-end laddering ascends attribute → consequence → value until a value term appears. Axiomatic design [26] and affordance-based design [27, 28] provide, respectively, an independence criterion within a chosen set of functional requirements and a relational account of action possibilities. Disruptive-innovation analysis is the sole family that routinely compares against substitutes from another category [29]. Across these families the "why" question is answered by interview, by tree ascent or by omission; in each case the analyst, rather than a test, decides where the ascent ends, and the current embodiment survives inside the description whenever the ascent ends early.

3.2 ai-assisted design

LLM-based tools now automate TRIZ reasoning [23], structure analogical mapping [30] and organize multi-agent design workflows [9]; conceptual accounts describe a shift to "backward" generative design in which humans define objectives and constraints [31], and field studies report usefulness alongside limits in coherence and alignment with design objectives [32, 33]. Engineering practice embeds causal reasoning in prescriptive rules while machine learning remains associational, which weakens transfer to novel conditions [34].

3.3 Causality, agency and path structure

Interventionist causation defines X as causally relevant to Y when an intervention on X changes Y [14, 15]; causal abstraction formalizes when a coarse model faithfully abstracts a fine one [35, 36]. Agent-centric objectives that value future options are converging: empowerment as the channel capacity of the actuation–sensing loop [16, 17], process empowerment robust to noise [37], a unification of empowerment, active inference and occupancy objectives as constrained entropy maximization [38], entropy regularization biasing reinforcement learning toward states with many viable future actions [18], and the theorem that optimal policies tend to seek power by moving toward larger sets of reachable terminal states [39]. Viability theory supplies kernels of states from which constraints can be maintained [19]; ecological psychology supplies affordances and, in 2025, intention prediction as affordance perception [28]. Capability sets formalize well-being as achievable functionings [40].

3.4 Embodied path planning

Contemporary robots plan by simulating candidate futures inside learned world models: a 1B-parameter diffusion transformer predicts egocentric observations from past frames and actions and ranks candidate trajectories, imagining paths in unseen environments from one image [20]; action-conditioned models raise interaction-prediction fidelity sharply [41]; one fixed-configuration world-model agent masters over 150 tasks across eight domains [42]. Vision-language-action policies map observation and instruction to action [43–45]; active perception is learned from human demonstrations [46] and coupled with rule-guided reinforcement learning for object navigation [47]; affordances are extracted label-free from egocentric video [48]. Safety enters as barrier-function constraints under sensing uncertainty [49–51]. Benchmarks locate the failures: the best multimodal model averages 28.9% across 1,128 embodied tasks [52]; household whole-body manipulation requires bimanual coordination, stable navigation and reachability [53]; smart-home agents collapse on invalid multi-device instructions (GPT-4o at 0.0%) [54]. Every component of this stack holds the environment's relational structure fixed during planning (Section 4.4).

3.5 The residual gap

Table 1 condenses the Phase-1 gap matrix. Escape from product form and category is covered; causal-necessity testing exists only in causal-reasoning approaches that supply semantics and leave the design procedure open; function removal, the world-versus-perceived distinction, path creation and the future option structure live in the agency and robotics literatures with a design-method counterpart still unlocated; an explicit stopping rule appears only in trivial cases. The gap is narrow, demonstrable, and dies if any single method occupies the intersection; the search found it empty. A peer-reviewed 2025 critique of jobs-to-be-done was searched for and was absent, which is recorded as a negative finding. Section 4 introduces the object that fills the gap — path availability — and Section 5 builds the procedure on it.

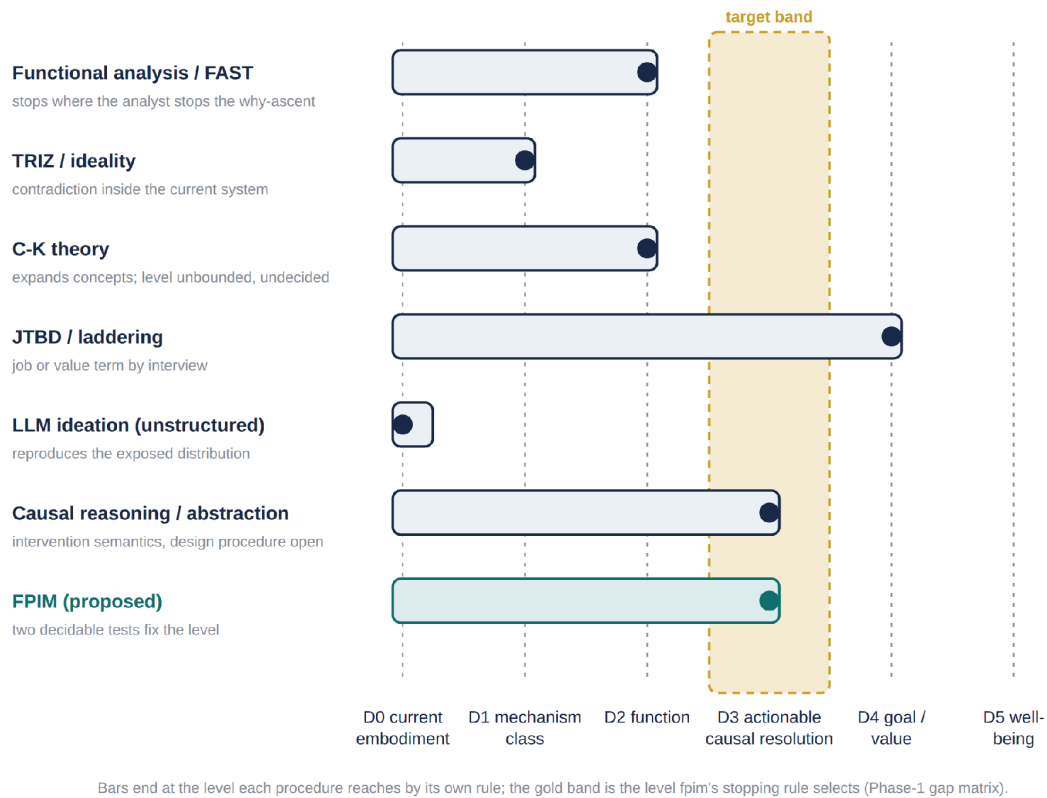
Figure 16 · Where existing methods stop on the abstraction ladder

Figure 16. The faster-horse problem, defined. Top: the timeline 1880–1927 from the horse city to the motor carriage and the Model T. Bottom: the request as a five-tuple $\langle e, a, f, O(e), o \rangle$; improvement within the request keeps the embodiment and raises the attribute; innovation holds function and outcome fixed and searches for an embodiment with a different obligation set. Conceptual, dates from public record.

Table 1. Closest prior art and the residual gap. S = gap supported, P = partial, N = the method already does it (Phase-1 matrix, 20 columns; six shown).

Method family	Causal necessity test	Function removal	World vs perceived paths	Path creation	Future option structure	Stopping rule
Functional analysis / FAST [21]	P	S	S	S	S	S
TRIZ / ideality [10, 23, 24]	P (trimming)	P	S	P	S	P
C-K theory [11, 25]	S	P	S	P	S	S
JTBD / outcome-driven [13]	S	S	S	S	S	P
Affordance-based design [27]	S	S	P	S	S	S
Causal reasoning / abstraction [14, 35, 36]	N	N	S	S	P	P
Empowerment / viability [16–19, 39]	—	—	P	S	N	—
Robot world-model planning [20, 42]	—	—	N (belief vs world)	S (environment fixed)	P	—
LLM ideation [7, 9, 23]	S	S	S	S	S	S

4. AFI and path availability

The gap of Section 3 has a shape: the missing operations all concern paths — which ones exist, which ones a team can see, which ones an intervention opens. This section supplies the object that carries those distinctions.

Key points.

- Freedom = path availability: the set of paths an agent can take from its present state (a definition, and a structure rather than a number).
- Keep five things apart: possibility (what the laws admit), passibility (what is reachable from here by this agent), world paths, perceived paths, distortion (the gap between the last two).
- Path choice selects; path discovery widens the perceived set; path creation changes a relation so that an impassible path becomes passible.
- The agent-relative law $\text{Freedom}(x) = (\text{Perception}(x) / \text{Distortion}(x))^{\alpha\infty}$ is a candidate law; FPIM uses it only as an ordinal diagnostic of a team's map, with every negative result kept in view.
- Navigation searches paths with dependencies fixed; innovation searches dependencies with the desired state fixed.

Only the part of the Architecture of Freedom Intelligence needed by the method is stated; each expression carries its status.

4.1 Definitions

- **Freedom = path availability** [definition]. For an agent a in world W at state s , Freedom is the set of paths available to a from s : a path is a sequence of transitions each admitted by the world's laws and each executable by the agent's body [55, 56]. Freedom is a structure; any scalar reading of it is a readout rather than the structure.
- **Possibility** — what the laws admit from some configuration. **Passibility** — what is reachable from the present state by this agent along an executable path; passibility $\subseteq$ possibility.
- **World paths** — the paths the world admits. **Perceived paths** — the paths the agent's representation carries. Two map errors follow: carrying a path the world refuses, and missing a path the world offers — the second produces zero signal of its own [57].
- **Perception P** — the agent's accessible representation of relevant state and available paths, read operationally as the set of distinctions the agent holds; zero implication of experience.
- **Distortion D** — the typed divergence between perceived and world structure: omission, false relation, latency, aliasing, blockage treated as absolute.
- **Path choice** — selecting among perceived passible paths. **Path discovery** — widening the perceived set toward the world set with the world unchanged. **Path creation** — altering relations or resources so that a previously impassible path becomes passible; under fixed laws it decomposes into enablement and unblocking, both widening access from below [57].
- **Future path availability** — the path set left after a path is realized (Figure 17).

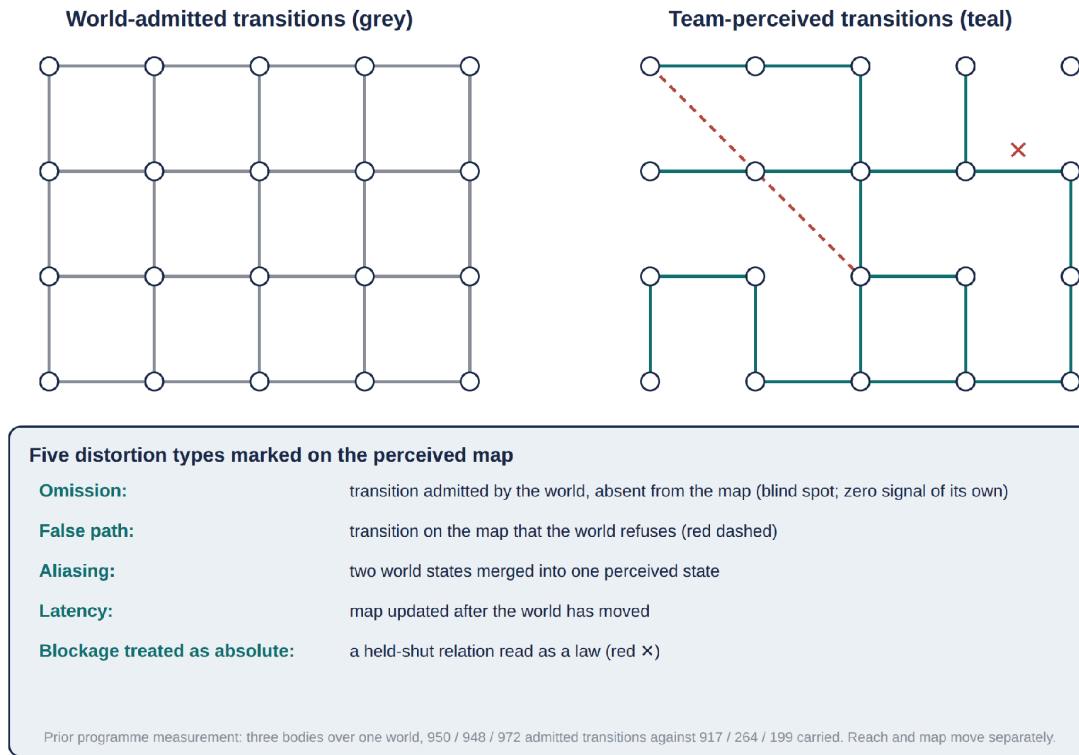
Figure 17 · World paths, perceived paths, distortion

Figure 17. The FPIM algorithm. Seven steps with stop rules as gates; the falsify → generate return arrow; kill bins under generate (feature addition), create-path (empty frees) and compete (holds, loses). Derived from Table 2.

4.2 The agent-relative law and its status

The public AFI record states an agent-relative relation, written in full [55, 58]:

$$\text{Freedom}(x) = (\text{Perception}(x) / \text{Distortion}(x))^{\alpha\infty} \text{ [candidate law]}$$

The ratio form is derived from three axioms — monotonicity, scale-covariance and separability — and is unique up to the exponent [derived relation] [58]; $\alpha\infty$, the limit of a recursive exponent closed through the Lambert W function, is a hypothesis of the programme with its measurement stated [59]. Seven modes (M0 the plain ratio, M1 a fixed exponent, M2 iterative exponent, M3 infinite power tower where justified, M4 $F_{(n+1)} = (\text{Perception}/\text{Distortion})^{(F_n)}$, M5 $F = (\text{Perception}/\text{Distortion})^{(\alpha F)}$, M6 multi-agent recursion) are kept separate in the author's project material; none is declared canonical because it fits better.

Empirical status [empirical model], reported with its negatives [58]: in over 200,000 simulated trials, distortion composed geometrically rather than additively (R^2 0.993 vs 0.856); perception was observer-dependent and, in graph navigation, irreducible to landscape features plus algorithm identity (hidden-variable R^2 0.058); step-wise alignment predicted navigation efficiency at R^2 0.82 against a maximum of 0.03 for graph-theoretic metrics; fitted exponents ($a \approx 1.29$, $b \approx -1.09$) excluded unity. Refuted or weak: perception alone outperformed the ratio at graph level (p 0.343 vs 0.108); a multiplicative layer decomposition of the FLRP architecture failed (R^2 0.0002); a scalar field formulation failed in continuous space (R^2 0.004); every result is simulation-based. These results bear on specific operationalizations — the composition of D, the measurability of P, the redundancy of the readout where rerouting is open — and leave the path-availability definition untouched. Their transfer to product innovation is a hypothesis.

Consequence for the method. FPIM uses the readout in one role only: an ordinal diagnostic (mode Mo) of a design team's map quality before and after pruning. Selection among concepts runs on a path preorder (Section 5.5); FPIM defines zero composite "AFI score".

4.3 Innovation as a change of passibility structure

Navigation searches paths with the dependency structure fixed. Innovation, in this framing, holds the desired state fixed and searches the dependency structure: it asks which relation must change for an impassible path to become passible, at what cost, and what that change frees and blocks afterwards. The distinction is the formal content the method takes from AFI; everything else is inherited from the literatures above.

4.4 The robotics bridge, stated precisely

The 2025 stack — sense → perceive → estimate state → model world → generate or imagine paths → select and plan → safe control → act → observe consequence → replan [20, 42–51] — separates what the world admits (dynamics), what the agent perceives (belief or scene representation), what it can do (action set), rollout of candidate futures, selection under constraints, and verification (Figure 18). The world-admitted versus agent-perceived gap is measured in both fields [20, 58], and the twelve navigation components reappear in the method's steps. The single primitive absent from the stack is environment alteration: a navigation world model learns a fixed environment, whereas the innovation move changes relations (Figure 19). The analogy is productive for the component decomposition and for reachability and viability formalisms; it misleads whenever the fixed-environment assumption is imported. Learned dynamics models transfer only once product innovation has a state space, which is yet to be defined. The next section turns the definitions of this section into steps: the path table of step 1 is the perceived path structure, the dependency list is the guard structure, the probes of step 2 are interventions on it, and the ledger of step 5 records the change in passibility.

Figure 18 · The embodied navigation loop (2025–2026)

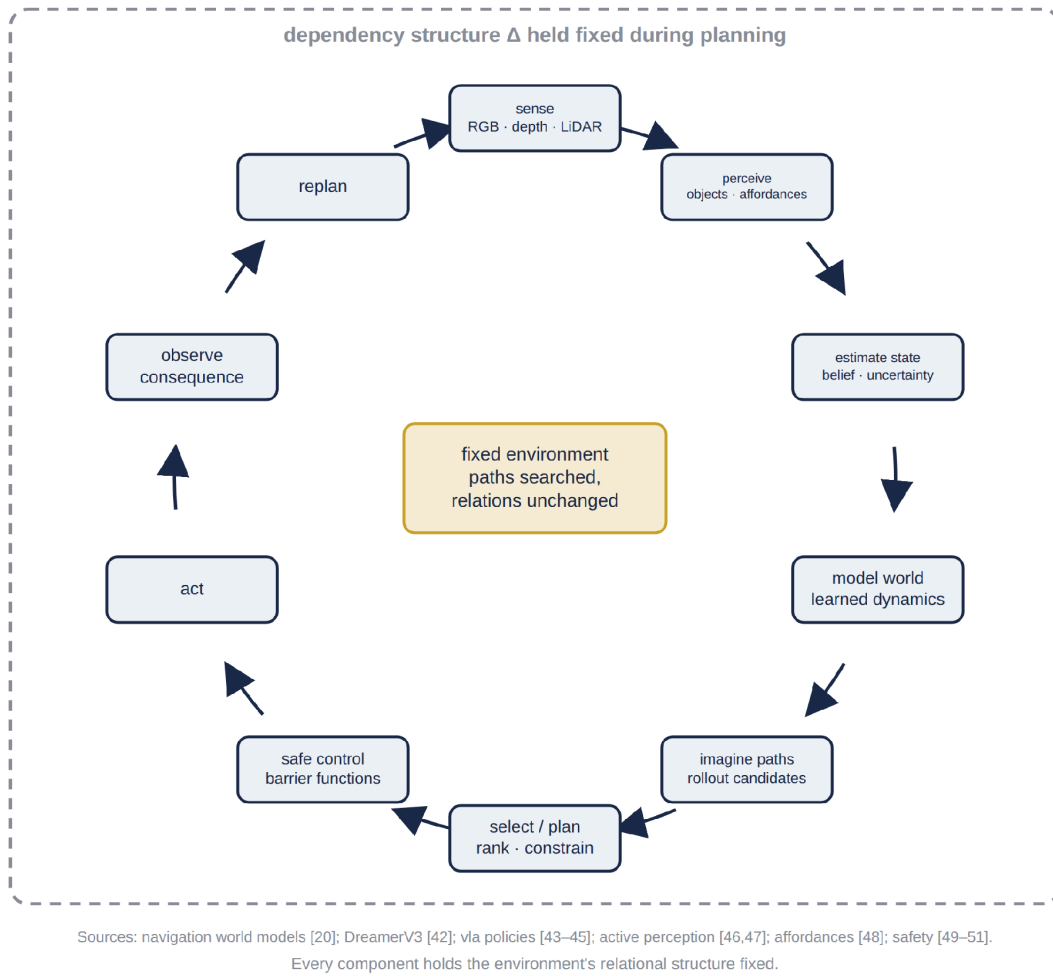
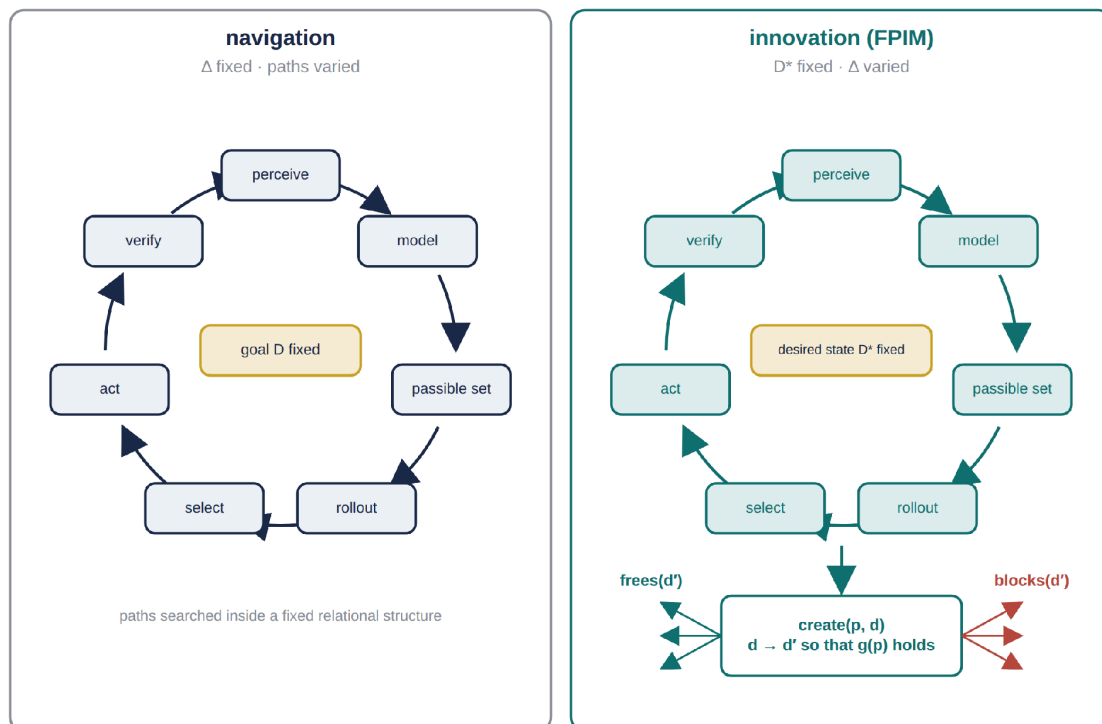


Figure 18. The horse-to-car ledger. The first path creation of Figure 1 written in the form of step 5: the relation changed, what it frees, what it blocks, what it requires, and what became newly reachable. Historical; illustrative of the ledger form.

Figure 19 · Navigate within a structure versus create a path

The twelve navigation components reappear in FPIM steps 1–7; the single added primitive is create, which alters a relation and yields the frees / blocks ledger.

Figure 19. The five-layer worksheet. Figure 31 left blank for any product: the five layers with their guiding questions, the desired-state line built from the middle three layers, and the six dependency changes to fill in. Template; print or copy.

5. The Freedom Path Innovation Method

With the object in hand, the dissection of Section 2 becomes a procedure: seven steps, each with an input, an operation, an output and a stop rule, so that two teams run the same thing.

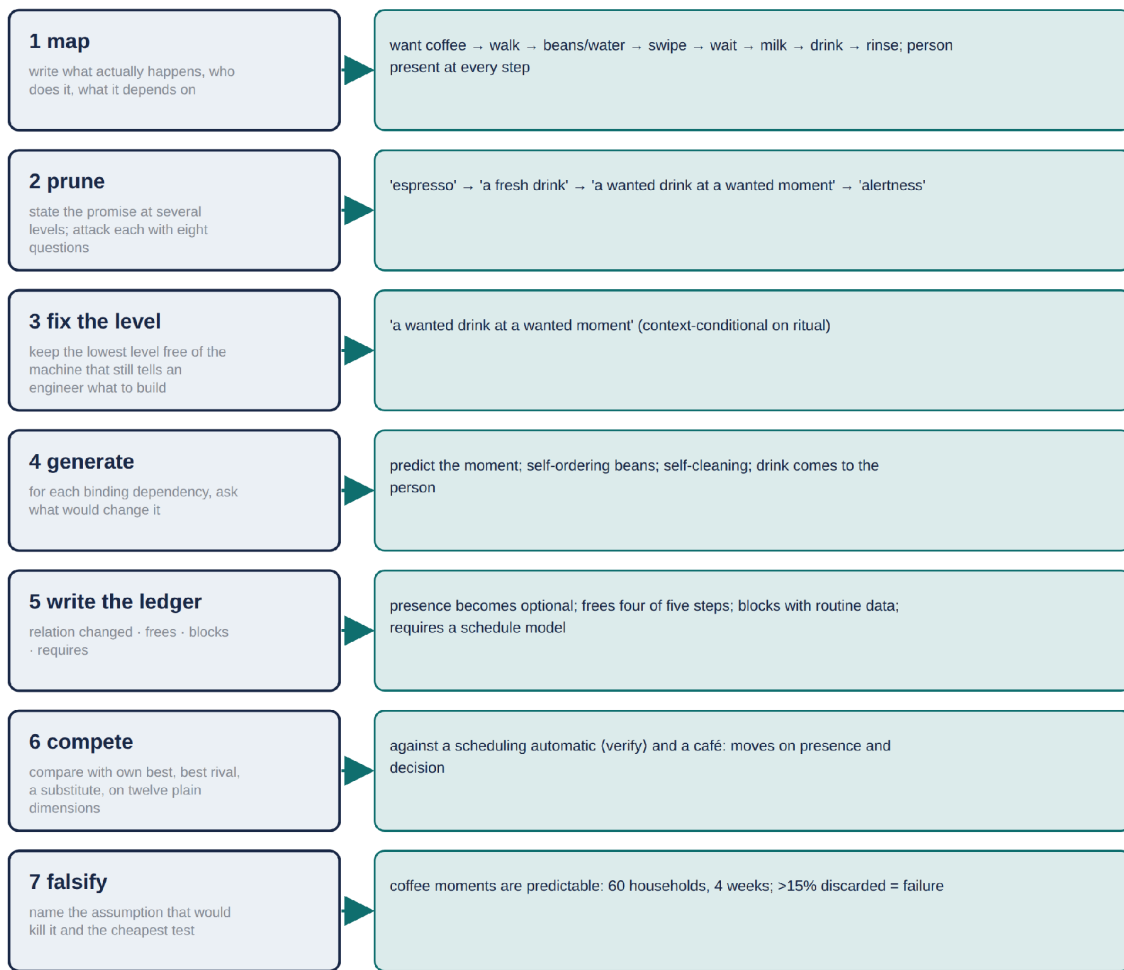
5.0 The method in plain words, with one example

The whole method on a bean-to-cup coffee machine (Figure 20):

- **1 map.** Write what actually happens: want a coffee → walk to the machine → check beans and water → swipe → wait → add milk → drink → rinse the parts. For each transition note who does it and what it depends on (the person present, beans in the hopper, a rinse afterwards). Mark what physics imposes (hot water must contact ground coffee) and what this machine imposes (the person must stand at it).
- **2 prune.** Write the promise at several levels — "espresso by 16-bar extraction" → "a fresh coffee drink" → "a wanted drink at a wanted moment" → "alertness" — and attack each level with eight questions: remove it, replace it, bypass it, prevent the need for it, delegate it, change the setting, change the person, supply the result some other way.
- **3 fix the level.** Keep the lowest level free of the machine that still tells an engineer what to build: "a wanted drink at a wanted moment". "Alertness" is too broad (a nap satisfies it); "espresso" is too shallow (it still contains the machine).
- **4 generate.** For each dependency binding the drink to the moment — presence at the machine, consumables, rinsing — ask what would change it: predict the moment from routine; let the machine order beans and clean itself; let the drink come to the person.

- **5 write the ledger.** For the strongest candidate: relation changed (presence becomes optional), frees (four of five human steps), blocks (dependence on routine data), requires (a schedule model, a self-cleaning path).
- **6 compete.** Compare with the company's best machine, the best rival and a substitute (a café, a capsule system) on twelve plain dimensions — outcome, obligations, attention, decisions, handoffs, time, skill, presence, robustness, agency, future options, new dependencies — using only better, equal or worse.
- **7 falsify.** Name the assumption that would kill it (coffee moments are predictable enough) and the cheapest test (sixty households, four weeks, prediction against a fixed timer; more than 15% of drinks discarded means failure).

The rest of Section 5 states each step precisely so that two teams run the same procedure.

Figure 20 · The method in plain words: the coffee example beside each step

Left: what the team does. Right: what the coffee-machine team writes. Section 5.0.

Figure 20. The alternatives population for "faster horse". The six dependency changes applied to the binding dependencies of the horse system, with the candidate each produces and the two feature additions killed. Derived from Section 2.5.

5.1 Input contract

I1 one shipped or specified product with a primary-source mechanism description; I2 one documented use episode, step by step, trigger to end state, at least five steps; I3 the agent who perceives, decides, acts and pays at each step; I4 a frontier set — best same-company analogue, best direct rival, best cross-category substitute, each with a primary-source mechanism; I5 the scope limits the sponsor accepts (regulatory, price band, horizon). A brief lacking I4 is rejected; a brief lacking I5 runs at the limit case only.

5.2 The seven steps

Three candidate algorithms were compared in Phase 2 on six product families: a linear sequence (observe → map → de-embody → intervene → generate → compare → falsify), a robotics loop with a path-creation operator, and a causal-core-first sequence. The linear sequence produced feature additions in 41% of outputs on two families because de-embodiment preceded the counterfactual battery; the robotics loop reproduced the current product on every family until path creation was added; the causal-core-first sequence, run with the shallow test alone, climbed to "being fed" and "well-being" on four families. An operation was retained only when deleting it changed the output population on at least one family; seven remain (Table 2, Figure 22).

Table 2. The FPIM algorithm.

#	Step	Input	Operation	Output	Stop rule
1	map	I1–I3	List the current path as transitions $t_1...t_n$; per transition record agent, perception, decision, handoff, time, skill, presence; mark each world-imposed (physics, regulation) or embodiment-imposed	Path table; embodiment set E; dependency list Δ (energy, material, information, agent, sequence, cost, rule)	Every transition has one agent and one dependency type
2	prune	Path table; ladder $D_0...D_n$ (D_0 the product's own claim; each level drops one embodiment term)	Apply eight probes to each level — remove, replace, bypass, prevent, delegate, context-shift, agent-shift, goal-removal — recording whether the level still names something the user would pay for	Survivor table (level $\times$ probe)	Eight probes at every level
3	fix-level	Survivor table	Apply the stopping rule (§5.3); type the result invariant, bundle, loop, context-conditional or unresolved	Desired state D^* at actionable causal resolution; binding dependencies Δ^*	One level or unresolved
4	generate	D^*, Δ^*, E	Apply the seven operators (§5.4) to Δ^* , one candidate per operator minimum; each names the dependency it changes	Path population	A candidate with an empty changed-dependency field is a feature addition and is discarded
5	create-path	Each candidate	Write the dependency intervention (§5.5): relation changed, at whose cost, what becomes possible, what it frees, blocks, requires	Ledger: frees / blocks / capabilities / new dependencies	An empty frees line discards the path
6	compete	Ledgers, I4	Twelve-dimension ordinal comparison (§5.6) against the frontier set normalized to D^*	Survivors and kills	Each candidate labelled moves / holds / loses
7	falsify	Survivors	State the critical unknown, the cheapest resolving test, and the result that counts as failure	Test card per survivor	Every survivor carries a stated failure result

A failed survivor returns to step 4 with its dependency marked fixed; two empty loops return unresolved.

5.3 The causal stopping rule

Key points. Two decidable tests fix the level. Shallow: the embodiment is still in the description. Broad: two unrelated mechanisms satisfy the description with the same capability ledger. The actionable level is the highest one that clears the shallow test before the next level trips the broad test.

- **The ladder.** $D_0...D_n$, each level deleting one embodiment term from the previous: "crispy food from a basket with hot air" $\rightarrow$ "crispy food with hot air" $\rightarrow$ "crispy food" $\rightarrow$ "a hot meal" $\rightarrow$ "being fed".
- **Shallow test $S(D_i)$.** True when the description still contains the embodiment: $D_i \cap E \neq \emptyset$. Removing the embodiment leaves the level unstatable.
- **Broad test $B(D_i)$.** True when two paths from materially unrelated mechanism classes (different energy source, material or agent) satisfy D_i with identical required-capability ledgers — the level gives zero architectural constraint.
- **Procedure.** Ascend while the shallow test holds; keep ascending while the broad test of the next level is false; stop. D^* is the highest level with S false and B true at the next level (Figure 21).
- **Edge cases.** S true at every level $\rightarrow$ unresolved-shallow (an embodiment-defined product). B true at the first non-shallow level $\rightarrow$ that level is reported as context-conditional, with the context terms that restore constraint.
- ****Shape of D^* .** **** Invariant** — one description survives all eight probes. **Bundle** — two or more survive jointly and each alone fails a probe. **Loop** — the survivor is a regulation ("keep X within a band"). **Context-conditional** — survival depends on a named context term. **Unresolved** — otherwise. A bundle is reported as a bundle; the rule imposes zero hierarchy.

- **Reliability.** Measured rather than assumed: two analysts run both tests independently; $\kappa \geq 0.6$ on D^* accepts the rule (Section 7).

Typing a requirement from its survivor pattern (the probes are Woodward-interventions on a requirement list [15]):

- intrinsic — survives goal-removal.
- physical — survives every probe except removal of the physical condition.
- functional — fails remove, survives replace.
- architectural — fails replace within the embodiment class.
- derived — fails prevent or bypass.
- legacy — fails agent-shift or context-shift.
- preference — fails delegate.
- optional — survives remove.

The typing is a proposal whose inter-analyst reliability is part of the validation protocol.

Figure 21 · Counterfactual causal pruning: the airfryer ladder

	remove	replace	bypass	prevent	delegate	context shift	agent shift	goal removal	tests
D0 crispy-juicy food, hot air + steam, basket	×	×	×	×	×	×	×	×	S=T B=F
D1 crispy-juicy food, hot air + steam	×	×	×	×	✓	×	×	×	S=T B=F
D2 cooked food with controlled moisture	×	✓	×	×	✓	✓	✓	×	S clears S=F B=F
D3 a wanted meal at the wanted time	✓	✓	✓	✓	✓	✓	✓	×	S=F B=F
D4 being fed	✓	✓	✓	✓	✓	✓	✓	✓	B trips S=F B=T

Rule

Ascend while $S(D_i) = \text{true}$ ($D_i \cap E \neq \emptyset$); keep ascending while $B(D_{i+1}) = \text{false}$; stop.

$D^* = D_3$ — the highest level with S false and B true at the next level.

Shape: bundle (wanted × timed × low-obligation): each term alone fails a probe, jointly they survive seven.

Binding dependencies Δ^* : ingredients present · human prep · human loading · human scheduling.

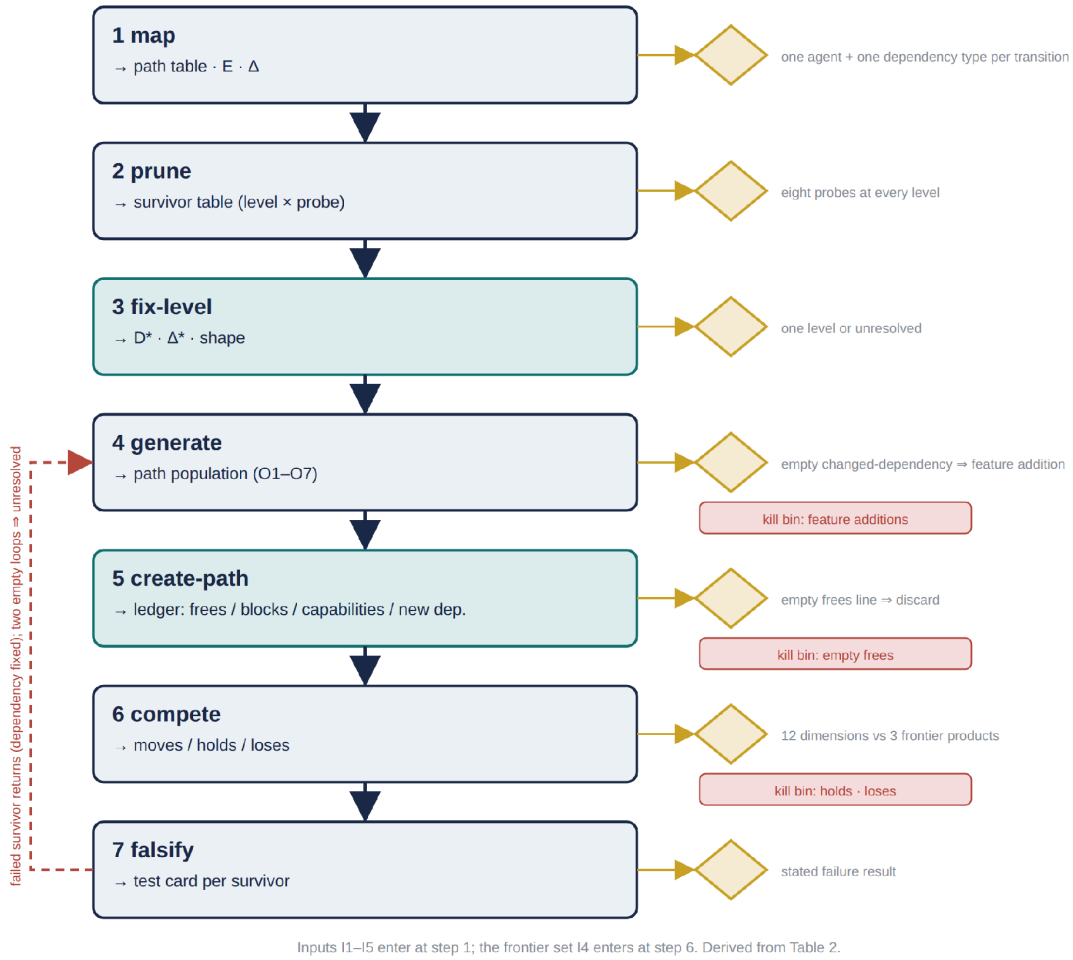
Typing from the survivor pattern: 'crispy' = architectural; 'hot air' = functional; 'timed' = derived from scheduling.

Dashed lines: S clears where the description is free of E; B trips where delivery, restaurant and cooker share one capability ledger.

✓ level still names something the user would pay for after the probe · × level collapses. Phase-2 run, case 1.

Figure 21. The recognition test for a non-obvious path. A decision tree from candidate to verdict: dependency changed? which of the six changes? survives the probes at the actionable level? frees more than it blocks? Outputs: non-obvious path, surprising decoration, feature addition. Derived from Section 2.3.

The probes are Woodward-interventions on a requirement list [15]. Their survivor pattern types a requirement: intrinsic (survives goal-removal), physical (survives every probe except removal of the physical condition), functional (fails remove, survives replace), architectural (fails replace within the embodiment class), derived (fails prevent or bypass), legacy (fails agent- or context-shift), preference (fails delegate), optional (survives remove). The typing is a proposal whose inter-analyst reliability is part of the validation protocol.

Figure 22 · The FPIM algorithm: seven steps, gates and kill bins**Figure 22. The innovation machine.** Radar → dissection engine → population → competition → test cards → learn, with the governance metrics and the kill rule for feature additions. Derived from Section 11.1.

5.4 Generation operators

Each operator takes (D^* , Δ^* , E) and names the dependency it changes; a candidate that changes none is a feature addition and dies.

- **o1 increment** — keep E; improve one transition. Retained as baseline only.
- **o2 substitute** — replace the mechanism behind one embodiment-imposed transition with another that satisfies the same dependency. Killed when the class equals E.
- **o3 collapse** — delete a transition by merging its dependency into a neighbour. Killed when it deletes a world-imposed transition.
- **o4 upstream** — intervene on the cause that makes the desired state needed, so episodes start later or less often. Killed when the cause lies outside the scope limit.
- **o5 autonomize** — move perception, decision and actuation of a transition to a non-human agent; the human keeps veto (auto, assist and manual preserved). Killed when human presence stays required at that step.
- **o6 transfer** — import a verified path that satisfies D^* in another domain. Killed when the example is unverified.
- **o7 limit** — remove every human obligation; state what remains and what the environment must then do. Killed when the remaining dependencies are unsatisfiable in any horizon.

5.5 Path creation, formally

A path $p = \langle s_0 \rightarrow \dots \rightarrow s \rangle$ carries a guard $g(p) = \bigwedge \square d \square$ over dependencies $d \square \in \Delta$; p is passible for agent a from s_0 iff $g(p)$ holds and every transition lies in a 's actuation set $A(a)$. The method adds to the navigation loop's select the operation

$\text{create}(p, d) := \text{replace relation } d \text{ by } d' \text{ such that } g(p)[d := d'] \text{ holds, at cost } c(d \rightarrow d'), \text{ with } \text{frees}(d') = \{q : g(q) \text{ newly holds}\} \text{ and } \text{blocks}(d') = \{q : g(q) \text{ newly fails}\}. \text{ [definition]}$

Under fixed laws create decomposes into enablement (d' supplies a missing resource) and unblocking (d' opens a held-shut relation) [57]; the step-5 ledger is create written out. Future path effect is ordinal: $p_1 > p_2$ when $\text{frees}(p_1) \supseteq \text{frees}(p_2)$ and $\text{blocks}(p_1) \subseteq \text{blocks}(p_2)$; incomparable pairs stay incomparable, and every composition in the method is ordinal or multiplicative, with weighted sums excluded.

5.6 Competitive survival test

For each candidate and each frontier product, normalized to D^* , twelve dimensions are compared ordinally (better, equal, worse):

- outcome · human obligations (transitions with a human agent) · perception burden · decision burden
- handoffs · time · skill · presence
- robustness (failure modes per episode) · agency (who can change the goal mid-episode) · future possibilities (frees) · new dependencies (blocks, consumables, subscriptions, data)

Verdicts:

- **moves** — better on at least one dimension and worse on zero against at least one frontier product, and better on at least three obligation-type dimensions against all three.
- **holds** — matches everywhere. Killed.
- **loses** — otherwise. Killed.

Feature lists stay outside the comparison (Figure 23).

Figure 23 · Frees/blocks competitive frontier (Kitchen family, normalized to D*)

	outcome	human obligations	perception burden	decision burden	handoffs	time	skill	presencerobustness	agency	future paths	new dependencies		
Deadline Cooker (hypothesis)	=	+	+	+	+	+	=	+	=	=	+	-	moves
Airfryer 5000 w/ Steam (same company)	=	=	=	=	=	=	=	=	=	=	=	=	reference
Steam airfryer rival (verify)	=	=	=	=	=	=	=	=	=	=	=	=	holds
Meal delivery (substitute)	-	+	+	=	+	+	+	+	-	-	-	-	loses

Reading the grid

Cells compare each row with the desired state held fixed: + better · = equal · - worse (ordinal, kept unsummed).

moves: better on ≥1 dimension and worse on zero against at least one frontier product, and better on ≥3 obligation-type dimensions against all three.

holds: matches everywhere. loses: otherwise. holds and loses are killed.

The substitute row shows why feature lists mislead: delivery frees six obligations and loses on outcome, robustness, agency and dependencies.

Rival entries marked (verify) await primary-source checks. Derived from Section 5.6 applied to Table 3.

Figure 23. The dry-path service. From "a stronger umbrella" to a machine: the five layers, the desired state, the dependency changed, and the loop perceive → desired state → decide → act → verify → learn with auto / assist / manual. Method-generated hypothesis.

5.7 Output

For each survivor: current state; desired state or causal bundle; current path; current embodiment; binding dependencies; counterfactual results; passible alternative paths; new path created; required capabilities; new product architecture; frees; blocks; future path effect; competitor frontier; critical unknown; falsification test.

6. Worked cases

A procedure earns its keep on real products. This section runs the seven steps on the horse, on six verified product families of one manufacturer, and on five everyday objects, and reports the losses beside the survivors.

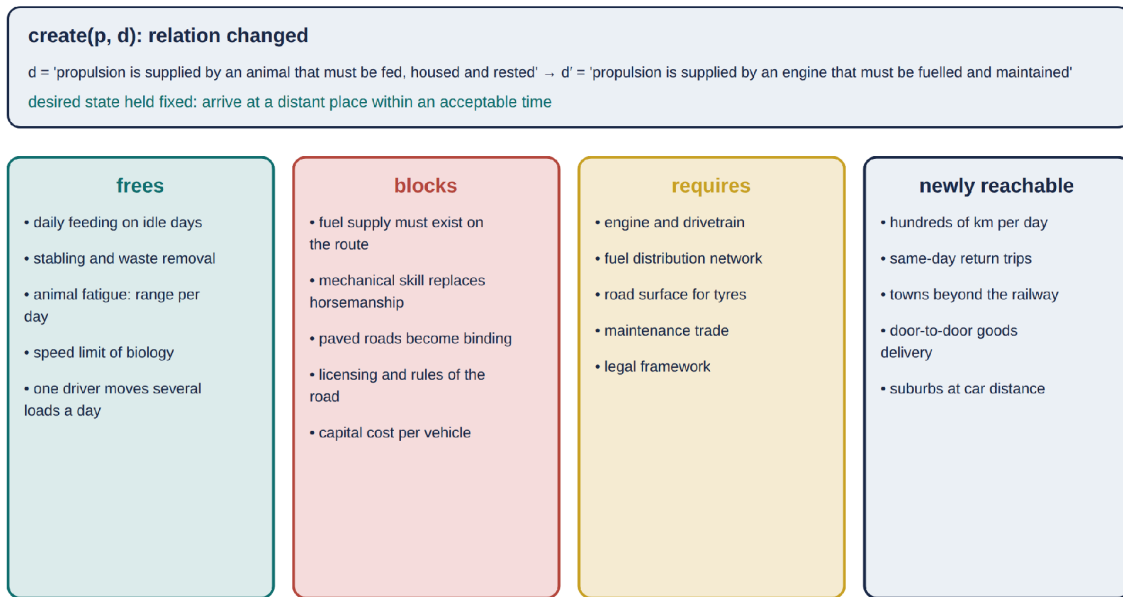
Key points.

- Every concept in this section is a method-generated hypothesis; Versuni has announced none of them.
- Six product families, product facts from company pages: four hypotheses generated, two families returned holds because the candidate already exists among rivals.
- Four resolved states are regulation loops; two (coffee, garment care) stay device-local.
- Five everyday products run through the same steps in Table 5 and are developed into machines in Section 10.

6.1 Historical calibration

The horse → automobile → telepresence → coordination ladder (Figure 1) reads as four successive create operations, each changing a different dependency — propulsion (animal → engine), presence

(body $\rightarrow$ signal), synchrony (simultaneous $\rightarrow$ asynchronous), agency (human $\rightarrow$ delegated). The ladder was assembled after the fact and serves as calibration for the stopping rule rather than as evidence for the method. Figure 24 writes the first of these creations as a ledger: the horse system before, the car system after, the dependencies removed, the dependencies installed, and what became newly reachable.

Figure 24 · The horse-to-car ledger (step 5 written for the first path creation)

Reading: the same form is filled for every candidate in Section 6; the horse case shows that a path creation frees, blocks and requires at once.

Historical; illustrative of the ledger form rather than evidence for the method.

Figure 24. Counterfactual causal pruning. Airfryer ladder $D_0 \dots D_4$; eight-probe survivor matrix; the shallow test S and the broad test B as two thresholds; D_3 marked bundle. Derived from the Phase-2 run of case 1.

6.2 Versuni: a verified design testbed

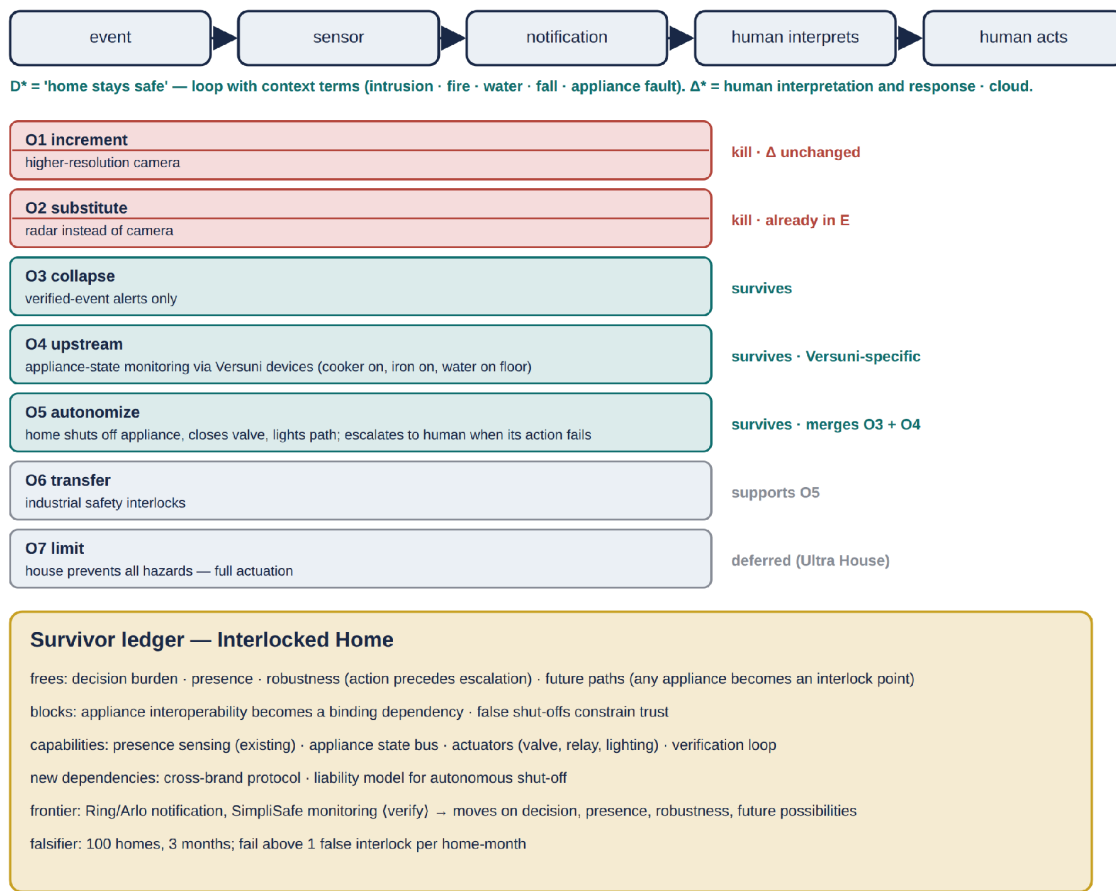
Versuni (formerly Philips Domestic Appliances; brands Philips, Baristina, Saeco, Senseo, Gaggia, L'OR Barista, Preethi, Walita) offers six product families with primary-source mechanism descriptions [60–70]. Product facts were fetched from company pages on 1 September 2026; rival claims marked ⟨verify⟩ await primary-source checks. Each family was run through the seven steps with the shipped product as input rather than target. Every concept below is a method-generated hypothesis; Versuni has announced none of them. Table 3 compresses four families; two further families are reported as method losses.

Table 3. Worked product cases (method-generated hypotheses; product facts from [60–70]).

Field	Kitchen: Airfryer 5000 with Steam (Feb 2026), Air Steam Cooker 7000	Coffee: Baristina (Mar 2024), Baristina Latte (Apr 2026), LatteGo, Gaggia	Air: Air Performer 3-in-1 (Nov 2022)	Home Safety 5000 (Mar 2024)
Current embodiment	Basket + hot air + steam, detachable tank, SteamClean	One-swipe grinder–portafilter–16-bar pump, milk frother	HEPA purifier + bladeless fan + heater, auto mode	360° camera, doorbell, non-line-of-sight Wi-Fi/Thread presence sensing
Current path	plan → buy → prep → load → select → wait → unload → clean	want → walk → beans/water present → swipe → wait → milk → drink → rinse	notice reading → runs → filter warning → buy and replace	event → sensor → notification → human interprets → human acts
Counterfactual break	"crispy food" fails replace; "controlled moisture" fails bypass	"espresso" fails replace; "fresh drink" fails delegate only under ritual	"clean air in this room" fails context-shift	"knowing what happens" fails goal-removal
Actionable causal resolution	D_3 "a wanted meal at the wanted time" — bundle (wanted × timed × low-obligation)	D_2 "a wanted drink at a wanted moment" — context-conditional on ritual	D_2 "indoor air within a healthy band" — loop	D_2 "home stays safe" — loop with context terms (intrusion, fire, water, fall, appliance fault)

Field	Kitchen: Airfryer 5000 with Steam (Feb 2026), Air Steam Cooker 7000	Coffee: Baristina (Mar 2024), Baristina Latte (Apr 2026), LatteGo, Gaggia	Air: Air Performer 3-in-1 (Nov 2022)	Home Safety 5000 (Mar 2024)
Binding dependencies Δ^*	ingredients present; human prep, loading, scheduling	presence at machine; consumable state; rinse	filter consumable; per-room placement; human response	human interpretation and response; cloud
Alternative path (operators surviving)	O3 kits read by the cooker + O5 endpoint sensing and scheduling	O4 routine prediction + O5 self-ordering, self-descaling	O4 source control + O5 whole-home loop (purifier, ventilation, hood under one policy)	O3 verified-event alerts + O4 appliance-state monitoring through Versuni's own devices + O5 interlocks
Method-generated hypothesis	Deadline Cooker: appliance owns scheduling, humidity, endpoint verification, self-cleaning; human owns loading and deadline	Anticipating Barista: presence obligation drops from four steps to one; swipe kept as optional ritual trigger	Envelope Loop: purifier becomes one actuator among ventilation and hood, source control first	Interlocked Home: presence sensing plus appliance interlocks; human is the escalation endpoint
Competitor / substitute	Ninja/Cosori steam airfryers, Thermomix (verify) keep the same obligations	De'Longhi/Jura automatics with scheduling (verify) keep presence	Dyson, IQAir (verify) keep filter logistics	Ring/Arlo notification, SimpliSafe monitoring (verify)
Freedom created	presence, decision and perception freed	presence and decision freed	human obligations and future possibilities freed	decision, presence, robustness, future possibilities freed
New block	ingredient recognition	routine data	cross-device control	appliance interoperability
Falsifier	200 meals, blinded doneness; fail under 85% acceptable unaided	60 households, 4 weeks, prediction vs timer; fail if >15% drinks discarded	30 homes, PM2.5 logged; fail under 30% fewer filter-hours	100 homes, 3 months; fail above 1 false interlock per home-month

Two families returned holds, which is the method losing as designed. Floor care (HomeRun 7000, AquaTrio, OneUp [64–66]) resolved to the loop "floor stays clean" and generated a plumbed dock with entry-point soil capture and fault-only intervention; plumbed stations already exist among rivals (verify), so the candidate holds on mechanism and moves only with the upstream component (installation retention under 70% at 90 days is the failure result). Garment care (OneTurn hybrid iron-steamer, Jan 2026 [69]) resolved to the invariant "garment wearable with zero care step" and generated prevention at storage — a finishing closet — which existing rival categories already hold (verify); the candidate was killed as a category entry and retained as an assist direction. Figure 25 shows the full population and kills for the Home Safety family.

Figure 25 · Worked case: Home Safety 5000 → Interlocked Home

Product facts: Versuni newsroom (Mar 2024). Every concept is a method-generated hypothesis; Versuni has announced none of them.

Figure 25. The faster-horse problem as a system. Left: the 1880 transport system as elements and dependencies (animal, feed, stable, farrier, driver, road, load) with the request "faster horse" acting on one parameter of one element. Right: the same system after the substitution of propulsion; feed and stable become fuel and garage, obligations removed in red, obligations installed in gold, and the widened reachable set. Conceptual; historical dates from public record.

Reading across the six runs: four resolved states are regulation loops whose autonomous options require shared perception and one policy across devices; two (coffee, garment) stay device-local. Two of six generated concepts coincide with existing rival products and were returned as holds rather than inventions — the honest count. Section 7 states what these runs establish, what stays open, and how the method is to be tested against the methods of Section 3.

6.3 Five everyday examples

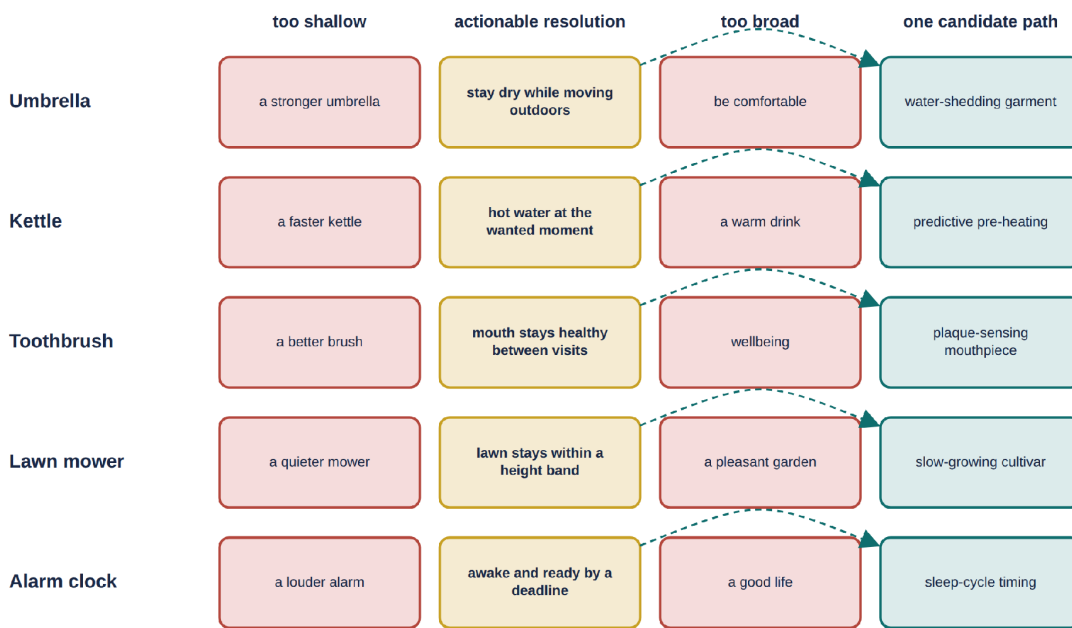
The method is general; Table 5 runs it briefly on five products outside Versuni's range so that the reader can see the same moves on familiar objects (Figure 26). Each row gives the embodiment, the level the stopping rule selects, one dependency changed, and a candidate path that follows. Section 10 develops the same five into non-obvious paths and machines.

Table 5. Five everyday products through the seven steps (method-generated hypotheses; illustrative).

Product	Too shallow	Actionable resolution	Too broad	Dependency changed	Candidate path
Umbrella	"a stronger umbrella"	"stay dry while moving outdoors" (invariant)	"be comfortable"	the person must carry and hold a canopy	water-shedding garment; covered walkways; timed routes that avoid rain
Kettle	"a faster kettle"	"hot water at the moment it is wanted" (bundle: hot × timed)	"a warm drink"	water heated in a batch, on demand, by the user	instant hot-water tap; predictive pre-heating; heat stored in the vessel

Product	Too shallow	Actionable resolution	Too broad	Dependency changed	Candidate path
Toothbrush	"a better brush"	"mouth stays healthy between visits" (loop)	"wellbeing"	cleaning depends on daily human dexterity and memory	plaque-sensing mouthpiece; enzymatic rinse; diet-side prevention
Lawn mower	"a quieter mower"	"lawn stays within a height band" (loop)	"a pleasant garden"	cutting depends on a person pushing a blade weekly	robot mower; slow-growing grass; ground cover that grows to a fixed height
Alarm clock	"a louder alarm"	"be awake and ready by a deadline" (context-conditional on the deadline)	"a good life"	waking depends on a sound at a fixed time	light-based dawn simulation; sleep-cycle timing; removal of the early obligation itself

Reading the table: the shallow column keeps the object inside the answer; the broad column loses all architectural guidance; the middle column is where structurally different paths appear, and the changed dependency is what makes them appear. The lawn mower and the toothbrush resolve to loops, as four of the six Versuni families did — a regulation held within a band — and the same autonomous options follow.

Figure 26 · Five everyday products on one ladder**Reading**

Red boxes keep the object inside the answer (shallow) or lose all architectural guidance (broad).

The gold box is where structurally different paths appear; the dashed arrow is the dependency that makes them appear.

Toothbrush and lawn mower resolve to loops, as four of the six Versuni families did; the same autonomous options follow.

Figure 26. Obvious versus non-obvious paths. Two routes to the same desired state: the obvious route keeps the dependency structure and improves a parameter; the non-obvious route changes one binding dependency. The test box asks which dependency changed. Conceptual.

7. Discussion and empirical validation

The cases of Section 6 show what the method produces; the question of whether it produces more than the alternatives stays open. This section states what is claimed, what is inherited, and how the claim is tested.

Key points.

- What the method claims: a stopping rule, a path-creation ledger, a survival test, and a computational representation of innovation as causal path creation. What it inherits: escape operations (TRIZ, C-K, analogy) and option-preservation content (empowerment, viability, power-seeking).
- The Ultra House is a limit case derived from four loop-type families, with zero empirical standing.
- The validation protocol can make FPIM lose: causal-distance gain under $d = 0.3$, usefulness or feasibility loss over $d = 0.3$, or κ under 0.6 for the stopping rule.
- Limitations are listed in full in Section 7.4, including the simulation-only status of every AFI result to date.

7.1 What the method claims and what it inherits

The generation logic inherits from TRIZ trimming, C-K expansion and laddering; the option-preservation content inherits from empowerment, viability and power-seeking [16–19, 39].

The synthesis adds three procedural elements: a decidable stopping rule with two tests, a path-creation operation with a frees/blocks ledger, and a survival test normalized to the desired state. The strongest independent element is the representation of innovation as a change in the reachable set after a relational intervention, which is publishable apart from AFI vocabulary.

7.2 The limit case: Ultra House

Four of six resolved states were regulation loops (Section 6). Repeated removal of human obligation across such loops converges, on those families, toward one architecture: ultra perception → shared world model → desired-state model → passibility model → policy → distributed actuation → verification → homeostasis, in which specialized appliances become effectors of a common environment (Figure 27). Convergence held for regulation-type domains and stayed open for ritual-type domains (coffee, garment care), so the Ultra House enters as a limit case rather than a result. Its governing principle — automate unwanted obligation, preserve wanted option — keeps auto, assist and manual as modes, consistent with variable-autonomy findings that hazardous settings favour collaboration over full autonomy [71] and with the argument that consent to smart-home data collection is legally sufficient yet morally inadequate for user autonomy [72]. Smart-home agents currently reach 76% on valid tasks [73] and 0.0% on invalid multi-device instructions [54]; a homeostatic home depends on closing that gap.

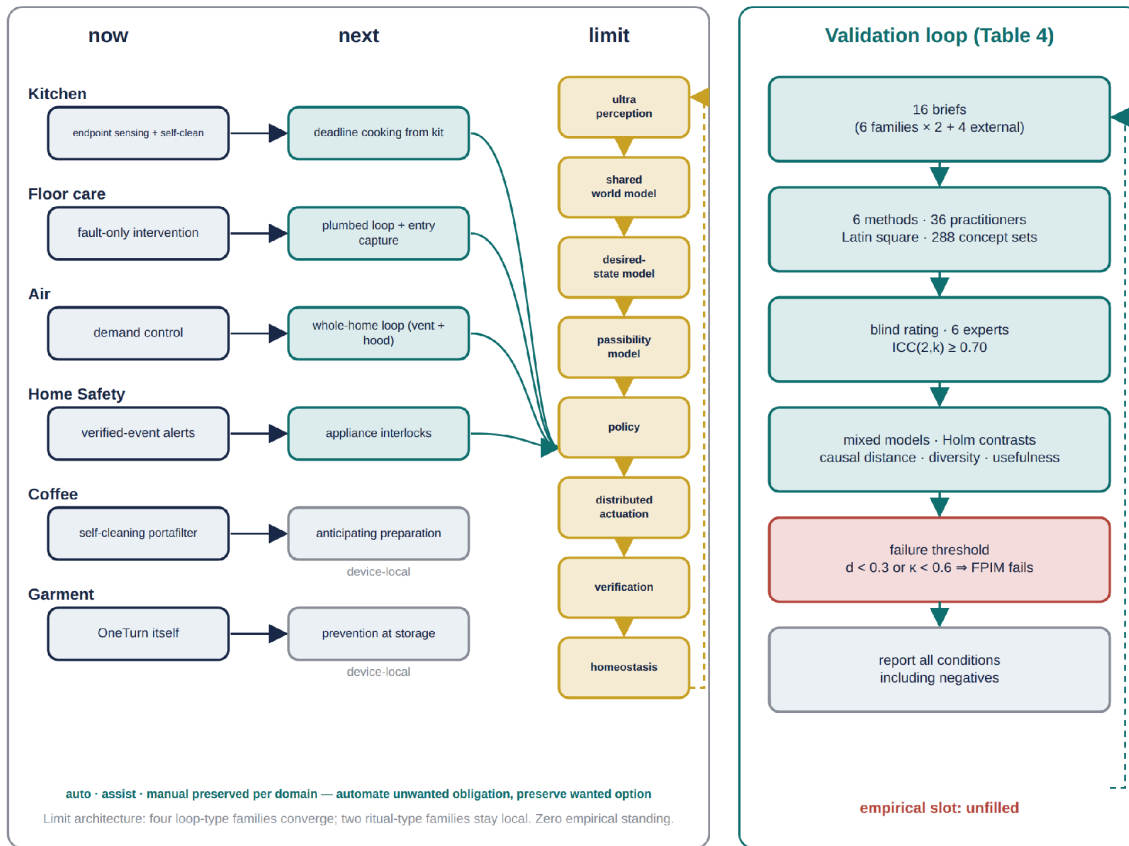
Figure 27 · now → next → limit, and the validation loop

Figure 27. The horse ladder under the eight probes. Levels D_0 – D_5 against remove, replace, bypass, prevent, delegate, context-shift, agent-shift and goal-removal; the shallow test clears at D_2 ; two actionable levels (D_3 for bodies, D_4 for information) before the broad test trips at D_5 . Derived from Section 2.4.

7.3 Validation protocol

Table 4. Pre-registered validation protocol.

Element	Specification
Hypotheses	H1 FPIM concepts show greater causal distance from the current embodiment than concepts from TRIZ, JTBD, C-K/functional analysis, structured LLM ideation and unstructured LLM ideation. H2 FPIM concepts rate at least equal on usefulness and feasibility. H3 FPIM raises solution-space diversity (mechanism classes) and constraint removal. Ho for each: zero difference.
Sample and tasks	16 briefs (six Versuni families × 2, plus four outside home appliances); 36 practitioners (≥3 years), within-subject Latin square across six methods, two briefs per method after 90 minutes of training each; LLM conditions share model, temperature and token budget, the structured prompt encoding each method's steps; 288 concept sets
Controls	3 hours per brief; identical input contract; counterbalanced order; evaluators blind to method and author; write-ups normalized to one template by a third party
Measures	Six blind expert raters, 1–7: novelty, usefulness, surprise, feasibility, competitive differentiation. Computed: causal distance (binding dependencies changed, two coders); diversity (pairwise dissimilarity of mechanism-class codes); constraint removal (human-obligation count vs current product). ICC(2,k) ≥ 0.70 per variable
Analysis	Mixed-effects models (method fixed; participant, brief random); Holm-corrected planned contrasts; standardized mean differences with 95% CI; stopping-rule reproducibility by two independent analysts, $\kappa \geq 0.6$
Failure threshold	FPIM fails if the causal-distance contrast against the best baseline has $d < 0.3$, if usefulness or feasibility falls below any baseline by $d > 0.3$, or if κ for D^* falls under 0.6. All conditions, variables and negative results are reported.

The method may lose on feasibility, usefulness or expert ratings while winning on causal distance, and it may fail reproducibility; the protocol is built to reveal each outcome.

7.4 Limitations

- Causal goals differ from physical causes: the probes intervene on a requirement list, and a requirement's survival depends on the analyst's model of the user; two analysts may build different ladders — the κ test measures exactly this.
- Human purposes are plural and contextual; the bundle and context-conditional outcomes represent this rather than resolve it.
- Path availability resists continuous measurement; the ledger is discrete and ordinal by design.
- P and D require operational validation for design teams: the AFI evidence base concerns navigating algorithms in simulation, the readout was redundant with perception alone where rerouting was open, and its transfer to teams is a hypothesis.
- Novelty of generated concepts guarantees zero value; the survival test is a filter rather than a proof.
- External validation of AFI remains limited.
- The Versuni cases are analytical runs by the author on public product facts, with rival facts pending verification; they are worked examples rather than field experiments, and two of six concepts coincide with existing products.
- The robotics analogy holds for component decomposition and reachability formalisms and fails wherever the fixed-environment assumption is imported.
- The Ultra House is a limit architecture derived from four loop-type cases, with zero empirical standing.

8. A protocol for dissecting any concept

Sections 2 and 5 dissected the horse and stated the method; this section folds both into one checklist that a reader can apply to any request tomorrow.

Key points.

- Fourteen steps take any request from its verbatim wording to a tested path hypothesis.
- Every step has a quality gate; a candidate that changes zero dependencies is struck out.
- Three audit questions close the protocol: is the object still inside the answer? would an engineer know what to build? which dependency changed?

The dissection of Section 2 generalizes to any request. Table 6 states it as a protocol with a quality gate at each step; Figure 28 draws it as a checklist. The protocol is the method of Section 5 read from the side of the request: steps 1–5 of the protocol prepare the input contract, steps 6–10 are the seven steps, and the three closing questions are the audit.

Table 6. The dissection protocol for any concept, with quality gates.

#	Step	What to write	Quality gate (the step is done when...)	Horse example
1	Record the request verbatim	The exact words of the customer, brief or founder	The words are quoted, unedited	"a faster horse"
2	Split the request into five layers	Object named · attribute wanted · function meant · obligations tolerated · outcome desired	All five are filled; the obligations list has ≥ 5 entries drawn from observed use	horse · faster · transport · feeding, stabling, waste, fatigue, range · arrive sooner at a distant place

#	Step	What to write	Quality gate (the step is done when...)	Horse example
3	Draw the system, then the use path	Elements and dependencies around the object; the ordered transitions of one real episode with the agent of each	Every element has a dependency type; every transition has one agent	animal, feed, stable, farrier, driver, road, load; harness → load → drive → rest → feed → stable
4	List the obligations	What a human must perceive, decide, remember, move, operate, verify, clean, maintain and pay, per episode and per period	Each obligation is tied to one dependency	feed daily (animal), muck out (animal), shoe monthly (animal), rest every 40 km (stamina)
5	Name the frontier	Best own analogue, best rival, best cross-category substitute, each with a primary-source mechanism	Three entries, each sourced	best carriage horse; rival breeder; the railway
6	Build the ladder	$D_0 \dots D_n$ by deleting one embodiment term per level	Each level differs from the previous by exactly one term	faster horse → faster horse-drawn transport → faster land transport → arrive sooner → be present → achieve the outcome
7	Probe every level	Eight probes per level; survivor table	Table complete; two analysts agree on ≥80% of cells	Figure 11
8	Fix the level	Shallow test, broad test; type the result (invariant, bundle, loop, context-conditional)	One level or unresolved, with the reason	D_3 for bodies, D_4 for information; context-conditional
9	Type the requirements and list Δ^*	Necessary · architectural · derived · legacy · preference · optional	Every requirement carries one type	propulsion necessary; animal architectural; feeding derived
10	Generate by the six dependency changes	Substitute · collapse · prevent · delegate · transfer · limit; one candidate each, naming the dependency changed	Candidates that change zero dependencies are struck out	Figure 13
11	Write the ledger	Relation changed · frees · blocks · requires · newly reachable	Frees line non-empty	Figure 20
12	Compete	Twelve ordinal dimensions against the three frontier entries at the same desired state	Verdict moves / holds / loses with the dimensions named	car moves on range, presence, obligations; blocks on fuel, roads
13	Falsify	The one assumption that would kill the survivor; cheapest test; failure result	Test card written before any build	fuel supply along routes; a 100-km trial with stated fuel stops
14	Audit with three questions	Is the object still inside the answer? Would an engineer know what to build? Which dependency changed?	Answers: absent · yes · one named dependency	horse absent · a motor carriage · propulsion

The formal reading of step 2 is a five-tuple $R = \langle \text{object, attribute, function, obligations, outcome} \rangle$; the ladder of step 6 deletes the object first, then the attribute's carrier, then the function's mechanism; the desired state D^* lies between the function and the outcome; and the obligations of step 4 are the binding dependencies Δ^* once they are typed in step 9. Quality follows from two disciplines: everything in steps 1–5 comes from observation or primary sources rather than from the analyst's imagination, and every candidate in steps 10–12 must name the dependency it changes or be struck out.

Figure 28 · The dissection protocol for any concept

Steps 1–5 prepare the input contract; 6–13 are the seven steps of Section 5 read from the side of the request; 14 is the audit. Table 6.

Figure 28. The request-to-path funnel, with the horse counts. One request → five layers → six ladder levels → forty-eight probe cells → six binding dependencies → six changes → nine candidates → two kills → seven survivors → one test per survivor. Derived from Section 2.

9. Use it in your place: the five-layer worksheet and a guide for teams

Key points.

- Figure 5 is the working tool of this paper: five layers around any request. Figure 29 is the same figure left blank, for a team to fill in for its own product.
- The guide below fits on one page and is written for an R&D team that wants to stop engineering faster horses this quarter.

Table 7. The five-layer worksheet (fill one per request).

Layer	Question to answer	Your product
Object named	Which thing does the request name?	
Attribute wanted	Which property of that thing is to improve, and in which direction?	
Function meant	What does the customer use the thing for, stated with zero reference to the thing?	

Layer	Question to answer	Your product
Obligations tolerated	What must a person perceive, decide, remember, move, operate, verify, clean, maintain and pay for, per episode and per period, for the function to happen? List at least five.	
Outcome desired	What state of the world does the customer pay for, once the function has done its work?	
Desired state (middle three layers)	Function + obligations removed + outcome, in one sentence free of the object	
Dependencies to attack	For each obligation: which relation makes it necessary?	
Candidates (six changes)	substitute · collapse · prevent · delegate · transfer · limit — one each, naming the dependency changed	
Kills	Any candidate that changes zero dependencies	
Cheapest test	The assumption that would kill the strongest candidate, and the test that resolves it	

Figure 29 · The five-layer worksheet — fill it in for your own product

outcome desired
what state of the world does the customer pay for?
.....

obligations tolerated
what must a person perceive, decide, move, operate, verify, clean, maintain, pay? (>5)
.....

function meant
what is the thing used for, said with zero reference to the thing?
.....

attribute wanted
which property, in which direction?
.....

object named
which thing does the request name?
.....

desired state (function + obligations removed + outcome), with the product's name absent:
.....

six dependency changes — one candidate each, naming the dependency changed; strike out any that changes none

substitute	collapse	prevent	delegate	transfer	limit
.....					
.....					

Figure 29. The earring that hears. Hearing assistance and audio engineered inside jewellery with the identity chain first; the machine, frees, blocks, falsifier. Method-generated hypothesis; precedents named.

9.1 A guide for R&D teams: stop engineering faster horses

The trap. Innovation engineering fixation is the habit of optimizing the attributes of the solution you already ship — ten percent faster, lighter, cheaper — while the causal structure of the product stays untouched. It feels like progress because every metric moves. It traps a roadmap in a local optimum because it leaves the customer's invisible obligations exactly where they were.

Invisible obligations, O(e). Every embodiment imposes burdens that users stop noticing because every alternative shares them. The horse had to be fed on idle days, stabled, shod, rested every 30 km, and its waste shovelled from the street; nobody asked for a horse that removed those chores, because "that is just how horses work". The automobile won by deleting the biological obligations, at the price of mechanical ones (fuel, roads). The push mower costs an hour every Saturday, sweat, and clippings to dispose of; the robot mower and the slow-growing cultivar reach the same desired state — grass held within a height band — and delete the labour.

How to map them.

- Take one real use episode and write every transition, with the person who performs it.
- For each transition, write what the person must perceive, decide, remember, move, operate, verify, clean, maintain and pay for.

- Mark each burden with the dependency that makes it necessary (energy, material, information, agent, sequence, cost, rule).
- The list is $O(e)$. Anything on it that survives your next release is a faster horse.

The exercise (ninety minutes, this week).

1. Pick the top item on the roadmap. Write its request verbatim.
2. Fill the five layers of Figure 29: object, attribute, function, obligations (at least five), outcome.
3. Write the desired state from the middle three layers, with the product's name absent from the sentence.
4. For each obligation, generate one candidate by each of the six changes: substitute, collapse, prevent, delegate, transfer, limit. Name the dependency each candidate changes; strike out any that changes none.
5. Keep the candidate that frees the most obligations and blocks the fewest. Write the one assumption that would kill it and the cheapest test.
6. Put that test — rather than the ten-percent improvement — on the roadmap.

10. Non-obvious innovations in six directions, and how each becomes a machine

Section 8 gave the protocol and Section 9 the worksheet; this section shows what comes out when both are applied to ordinary objects, and takes each result one step further — from a path to a machine that holds the desired state on its own. It also deepens the "why": a product serves a chain of needs, and the winning embodiment is rarely the one that serves the first need best.

Key points.

- A product's name states one need; the product serves a chain of them. The umbrella protects from rain, and also from sun; it is carried, so it is a burden; it breaks, so it is a cost; it is visible, so it is fashion. A dissection that stops at "rain" throws away most of what the object is bought for.
- The optionality principle: the best products maximize secondary-use optionality while minimizing the cost of carrying it. In this paper's terms, optionality is path availability and the carrying cost is the obligation set — the ledger measures both. The Monobloc chair is the cleanest case on record.
- Six directions produce non-obvious innovations: remove an obligation; maximize the bundle; give a function to the purely decorative; give a bundle to the purely functional; attack the cause upstream; attack the way the object fails.
- Ten worked innovations follow, five per family of directions, each with its machine, its ledger and its falsifier; every one is a method-generated hypothesis, and precedents are named wherever a rival already holds part of the path.

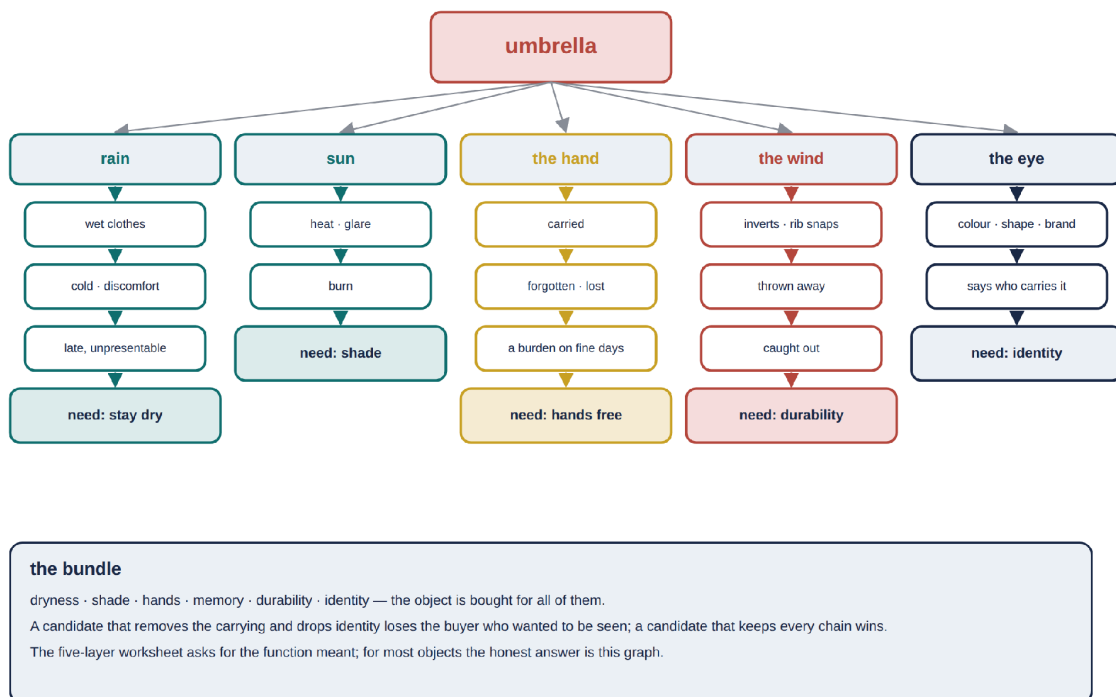
10.1 The need chain: a product serves more than its name says

The five-layer worksheet asks for the function meant. For most objects the honest answer is a graph rather than a word (Figure 30). The umbrella:

- Primary need: stay dry in rain. Chain: rain → wet clothes → cold, discomfort, ruined garments, a late and unpresentable arrival.
- Second need, same object: shade in sun. Chain: sun → heat, glare, burn → the same object, a different weather.
- The carrying chain: the object must be carried → a hand is occupied, it is forgotten at home, lost in cafés, left on trains → the object is a burden exactly when the weather is fine.
- The failure chain: wind inverts it, a rib snaps → it is thrown away → cost, waste, and being caught out in the very moment it was for.

- The visibility chain: it is seen → colour, shape and brand say something about the carrier → the umbrella is fashion, and a fashionable umbrella is bought for that alone.
- The bundle: dryness, shade, hands, memory, durability, identity. A candidate that removes the carrying obligation and loses the identity chain (a rain-avoiding route) fails against a good umbrella for the person who wanted to be seen with one; a candidate that keeps every chain wins.

The rule that follows — among embodiments that meet the primary need, the one that serves the largest bundle of further needs at the lowest obligation tends to win — is an observation the author associates with Nassim Nicholas Taleb; its exact wording was searched for in his published aphorisms and stays unlocated, so it is stated here in our own terms and supported by the case that follows. In the language of Section 4 the rule is a statement about path availability: the embodiment with the largest bundle opens the most paths for the buyer at the fewest obligations, and the ledger of step 5 measures exactly that.

Figure 30 · The need chain of an umbrella: one object, five chains**Figure 30. The hanger that finishes.** Garment care held where the garment rests; the machine, frees, blocks, falsifier; the link to the Versuni garment case. Method-generated hypothesis.

10.2 Optionality: the principle, and what lies beyond it

The rule of Section 10.1 has a sharper form: the best products maximize secondary-use optionality while minimizing the cost of carrying that optionality. Read through this paper's terms, it is a statement about path availability, and it goes further than a rule of thumb (Figure 41):

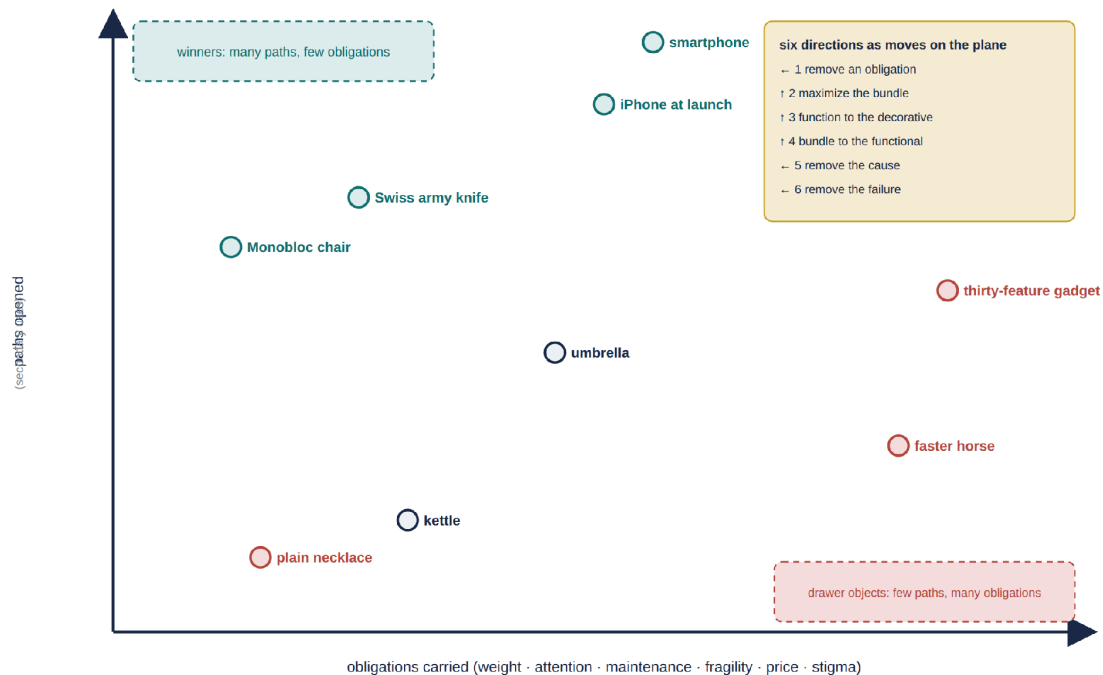
- Optionality is path availability. An object's secondary uses are the paths it opens for its owner beyond the primary one — the umbrella as sunshade, cane, pointer, shield; the smartphone as camera, map, wallet, torch; the Monobloc as step, table, stack. Each is a passible path that the owner holds the moment the object is at hand.
- The carrying cost is the obligation set. Weight, attention, maintenance, fragility, price, stigma, storage: everything the owner must do or bear to keep the option alive. It is the $O(e)$ of Section 1.1, read as the price of the paths.
- The rule is the ledger. Frees is the optionality gained; blocks is the carrying cost installed. A candidate that opens many paths and carries few obligations dominates ordinarily; the ledger of step 5 and the survival test of step 6 compute exactly this, with zero weighted sums.
- Beyond the rule, optionality is convex: an option is exercised rarely and valued always, because the rare need is the costly one — the umbrella earns its place on the one day it rains, and the smartphone's torch on the one night the power fails. A product should be judged on the days its options fire, and priced on the days they are carried.
- Beyond the rule, optionality compounds: a platform opens paths that open paths — the phone's applications opened uses that stayed unlisted at the maker. The measure of a great embodiment is the size of the path structure it leaves open after purchase (future path availability, Section 4.1), and the sign that a product has become a platform is that its owners find paths its designers left unenumerated.
- Beyond the rule, carrying cost compounds too: every option carried adds weight, attention or maintenance, and the sum of small costs kills the object (the gadget with thirty features and a

drawer to live in). The design task is the ratio rather than the count — the most paths per obligation carried — which is why one-piece, one-material, zero-maintenance objects (the Monobloc) beat multi-part ones at the same number of uses.

- Beyond the rule, the method makes optionality searchable rather than lucky. Step 4 of the protocol lists obligations; Section 10.1 lists the need chains; the six directions of Section 10.4 are six ways to move an embodiment toward more paths per obligation: remove an obligation, add uses, give a use to the decorative, give the identity path to the useful, remove the cause, remove the failure.
- The formal reading, for an AFI reader: the owner's Freedom rises with the paths an object opens and falls with the distortion it imposes; the object is a change in the owner's path structure, and the ledger is the record of that change. The relation is used ordinally throughout; the method defines zero composite score.

Design consequences, stated as tests a candidate must pass:

- Count the paths: how many distinct desired states can the object serve in a week of ordinary life? Fewer than three, and it is a single-use object that must win on the primary need alone.
- Count the obligations: what must the owner do to keep every one of those paths alive? More obligations than paths, and the object is a drawer object.
- Check convexity: does at least one path fire in a rare, costly moment? An object with zero rare-day value is bought on price alone.
- Check compounding: does the object leave paths open that the maker left unenumerated? If yes, the identity and platform chains are in play and the survival test must include them.

Figure 31 · Optionality against carrying cost

Qualitative placement; the ledger of step 5 records the same two quantities for any candidate. Ordinal reading only.

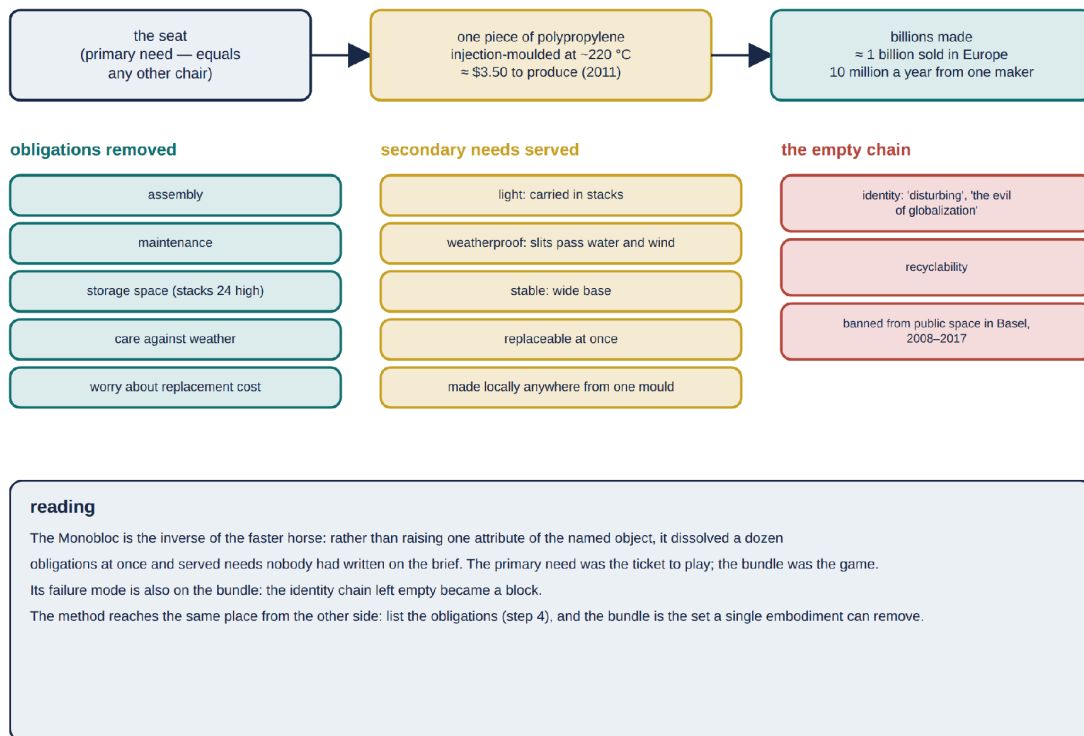
Figure 31. The embodied navigation loop, 2025–2026. Sense → perceive → estimate state → model world → imagine paths → select → safe control → act → observe → replan, with the dependency structure Δ drawn as a fixed frame around the loop. Sources: [20, 42–51].

10.3 The Monobloc lesson

The Monobloc is the world's most common plastic chair: one piece of polypropylene injection-moulded at about 220 °C, roughly 3.50 dollars to produce in 2011, stackable twenty-four high, produced in billions, with about one billion sold in Europe alone; one Italian manufacturer made over ten million a year by 2004 [78]. As a seat it equals other chairs. It won on the bundle (Figure 32):

- Obligations removed: assembly, maintenance, storage space, care against weather, replacement cost worth worrying about.
- Secondary needs served: light enough to carry in stacks, weatherproof (slits let water and wind through), stable (wide base), replaceable, manufacturable locally anywhere from one mould.
- The primary need — a seat — was the ticket to play; the bundle was the game.
- Its failure mode is also on the bundle: aesthetics and recyclability drew criticism and a public-space ban in Basel from 2008 to 2017 [78] — a reminder that the identity chain, left empty, is a block.

The Monobloc is the inverse of the faster horse: rather than raising one attribute of the named object, it dissolved a dozen obligations at once and served needs nobody had written on the brief. The method reaches the same place from the other side: list the obligations (step 4 of the protocol), and the bundle is the set of obligations a single embodiment can remove.

Figure 32 · The Monobloc lesson: the seat was the ticket, the bundle was the game

Facts from Wikipedia, Monobloc (chair), accessed 2 September 2026 [78].

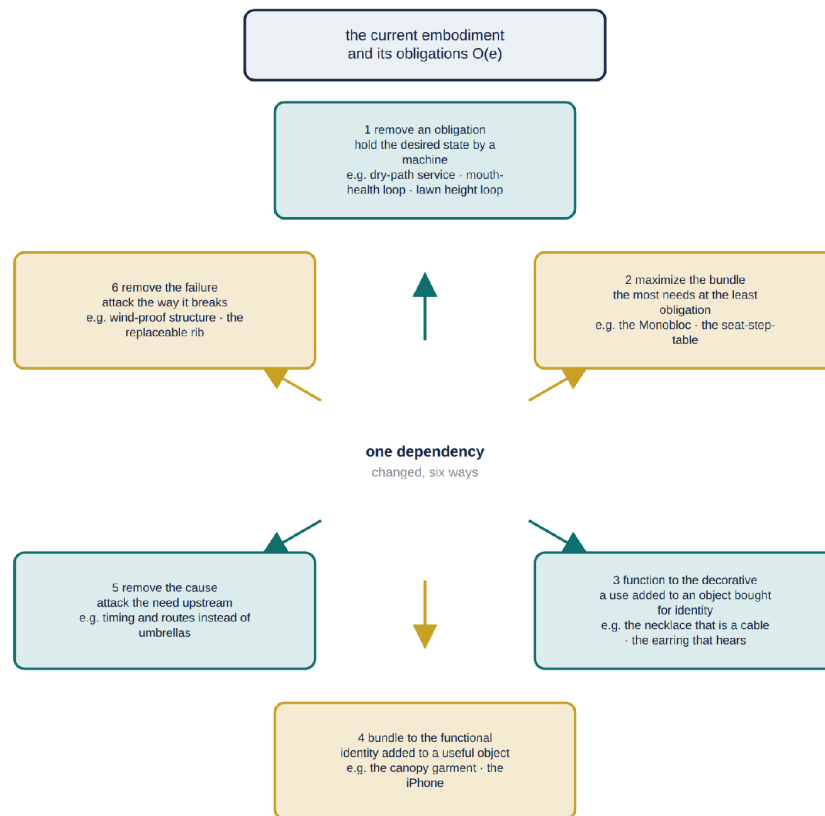
Figure 32. The mouth-health loop. From "a better toothbrush" to a sensing mouthpiece that regulates plaque within a band and escalates to a professional. Method-generated hypothesis.

10.4 Six directions of non-obvious innovation

The obligation map (Section 2.6) and the need chain (Section 10.1) give six directions in which a non-obvious path can be searched (Figure 33). Each is a different way of changing a dependency:

- **Direction 1** — remove an obligation and hold the desired state by a machine. The five innovations of Sections 10.6–10.10.
- **Direction 2** — maximize the bundle: serve the most needs at the least obligation. The Monobloc pattern.
- **Direction 3** — give a primary function to the purely decorative: rings, earrings, bracelets and collars are bought for the identity chain alone; a function added there rides on an object people already carry. Precedents: the wristwatch that became a computer, the ring that measures sleep and pays at the till.
- **Direction 4** — give the bundle to the purely functional: the Monobloc lacked identity; the iPhone had function and identity from day one. A functional object that acquires the identity chain wins the buyers who buy for it.
- **Direction 5** — attack the cause upstream of the need: the dry-path service attacks exposure rather than rain.
- **Direction 6** — attack the failure chain: the way the object breaks is a need in itself; the wind-proof umbrella and the replaceable rib are paths.

Sections 10.6–10.10 develop direction 1; Sections 10.11–10.15 develop the others.

Figure 33 · Six directions of non-obvious innovation

Each direction is a different dependency change; a candidate is named by the direction it takes and the dependency it changes.

Figure 33. The lawn that keeps its own height. From "a quieter mower" to a growth-dependency change (slow cultivar or fixed-height cover) with a small robot as trimming actuator. Method-generated hypothesis.

10.5 Reading guide to the ten problem-solution innovations with the method

Each of the ten below is written in the same order, so that the reader can compare them side by side and run the same form on their own object with the worksheet of Section 9:

- Five layers → desired state → dependencies changed → the non-obvious path → the machine → frees and blocks → falsifier (the dissection).
- Named innovations: three candidates per problem, each with a name, how to create it, and its properties — because a dissection yields a population rather than a single answer.
- What is ruled out: the paths the physics, the body, the law or the economics close, and the obligations that stay whatever the embodiment — stated so that a team stops spending on them.
- The best path, and why: the ordinal verdict of the survival test against the frontier, with the runner-up and the condition under which it wins instead.

Figure 34 shows the complete card for the first problem; Figure 35 collects the ten verdicts on one board.

Figure 34 · The problem-solution innovation card: 'a stronger umbrella'

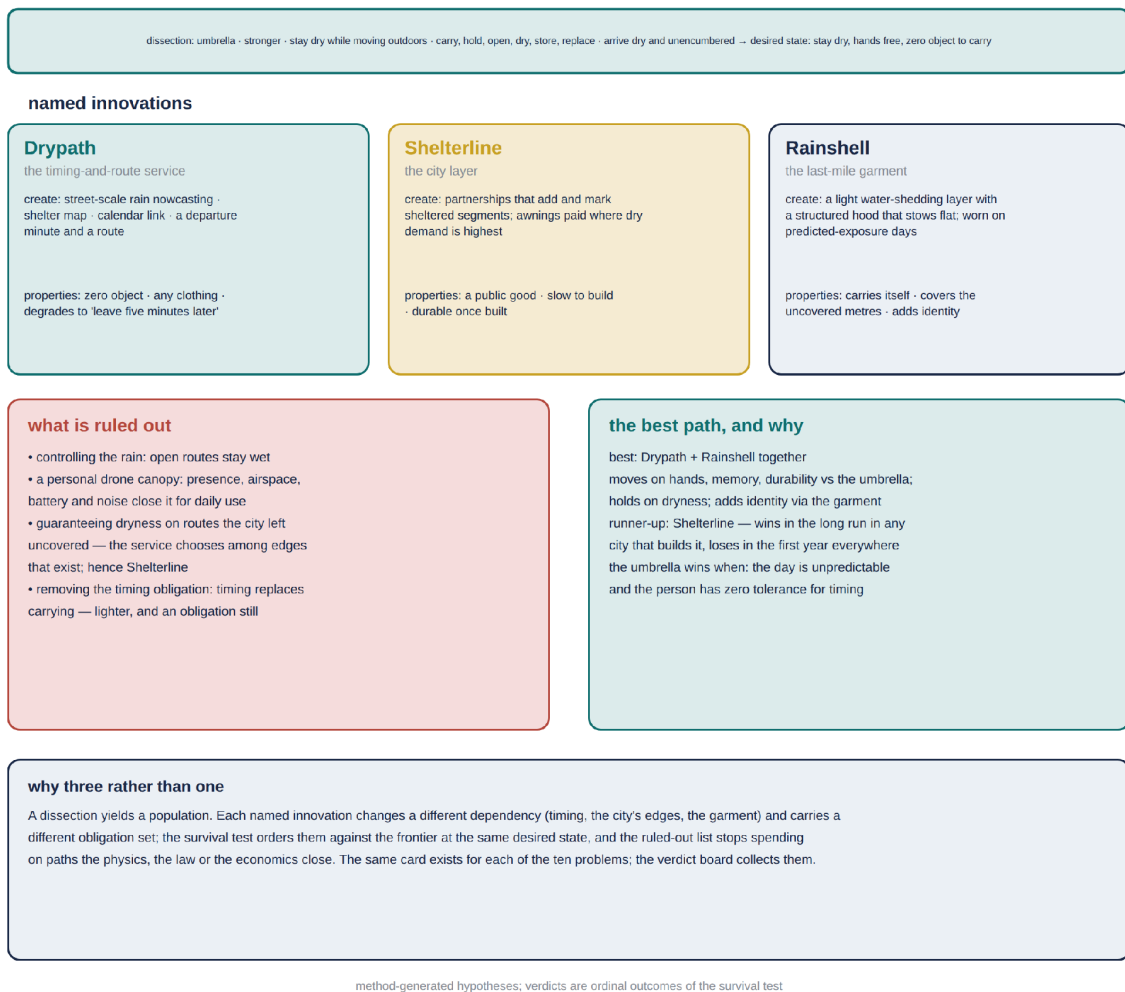


Figure 34. The necklace that is a cable. A decorative object given a primary function: the shape-holding connector, its small safety machine, frees, blocks, falsifier. Method-generated hypothesis; ring precedents named.

Figure 35 · The verdict board for the ten problem-solution innovations

problem → solution	best path	runner-up	ruled out
umbrella → dry path	Drypath + Rainshell	Shelterline	controlling rain · drone canopy
kettle → hot water first	Routineboil, then Cupready	Heatvault (if the core delivers)	instant cup on a domestic socket
toothbrush → mouth loop	Plaquemap + Dentalink	Rinseloop	stopping plaque for good
mower → lawn height	Lowgrow + Edgebot	Groundcover	stopping ordinary grass growing
alarm → wake-up	Dawnwindow + Cyclewake	Sleepcalendar	waking with zero deadline
umbrella → garment	Brimcape / Hoodshell	Fieldcoat	sheltering a second person
stool → seat-step-table	Trimono (if load-rated)	Monobloc-plus first	zero-compromise three-load geometry
necklace → cable	Formcord	Powerpendant	high power on skin
earring → hearing	Earpearl	Hoopbuds	severe-loss aids in jewellery mass
hanger → finishing	Wardrobeoop	Steamhanger · Tensionhanger	deep wool creases by tension alone

Verdicts are ordinal outcomes of the survival test against the frontier at the same desired state; every entry is a method-generated hypothesis with a falsifier in the text.

Figure 35. Optionality against carrying cost. Objects placed by the paths they open (secondary uses) against the obligations they carry; the winners sit high and left; the drawer objects low and right; the six directions of Section 10.4 drawn as moves on the plane. Qualitative placement; conceptual.

10.6 "A stronger umbrella" → the dry-path service

- Five layers: object umbrella · attribute stronger · function stay dry while moving outdoors · obligations carry it, hold it with one hand, open and close it, dry and store it, replace it after wind · outcome arrive dry and unencumbered.
- Desired state: stay dry while moving outdoors, hands free, with zero object to carry (invariant).
- Dependencies changed: the canopy carried by the person → shelter supplied by the route and the garment; exposure decided by the walker → exposure decided by timing.
- Non-obvious path: a dry-path service — rain fields perceived at street resolution, departures timed and routes chosen along covered and sheltered segments, with a water-shedding garment as the residual layer for the last uncovered metres.
- The machine (Figure 36): perceive (rain radar, canopy and arcade map, the person's calendar) → desired state (dry arrival by time t) → decide (depart at t' , route r) → act (a nudge to the person, a booking of a sheltered vehicle for the uncovered gap) → verify (arrival, dryness report) → learn. Auto for timing, assist for route, manual override always.
- Frees: the hand, the carrying, the storage. Blocks: data on the person's movements; dependence on city geometry. Requires: rain nowcasting at street scale, a shelter map.

- Falsifier: 40 commuters, one wet season; failure if dryness ratings fall below the umbrella's or if fewer than 60% follow the timing.

Named innovations:

- **Drypath** — the timing-and-route service. Create: street-scale rain nowcasting (radar plus crowd reports), a shelter map (arcades, awnings, covered stops), a calendar link, and a notification that proposes a departure minute and a route. Properties: zero object to carry; works with any clothing; degrades gracefully to "leave five minutes later".
- **Shelterline** — the city layer. Create: partnerships that add or mark sheltered segments (awnings, colonnades, covered stops) so that the map has more dry edges; the service pays for awnings where dry demand is highest. Properties: a public good that improves every user's routes; slow to build, durable once built.
- **Rainshell** — the last-mile garment. Create: a light water-shedding layer with a structured hood that stows flat, worn on days Drypath predicts exposure. Properties: covers the uncovered metres; carries itself because it is worn; adds an identity chain.

What is ruled out:

- Controlling the rain itself; a route through open country stays wet.
- A personal drone canopy: presence, airspace rules, battery and noise close it for daily use.
- Guaranteeing dryness on routes the city has left uncovered — the service can only choose among edges that exist, which is why Shelterline exists.
- Removing the obligation to leave at a chosen minute: timing replaces carrying; it is a lighter obligation, and an obligation still.

The best path, and why:

- Best: Drypath with Rainshell, together. Against the umbrella they move on hands, memory and durability, hold on dryness, and add identity through the garment; the umbrella keeps only the rare unplanned exposure.
- Runner-up: Shelterline, which wins in the long run in any city that builds it and loses in the first year everywhere.
- Condition for the umbrella to win: a user with an unpredictable day and zero tolerance for timing.

10.7 "A faster kettle" → hot water that precedes the wish

- Five layers: object kettle · attribute faster · function hot water for a drink · obligations fill, switch, wait, pour, descale · outcome a hot drink at the moment it is wanted.
- Desired state: hot water at the wanted moment (bundle: hot × timed × at hand).
- Dependencies changed: heating on demand in a batch → heat kept ready at the point of use; the human as the trigger → routine as the trigger.
- Non-obvious path: a small insulated reservoir at the tap that holds one cup at temperature, refilled and re-heated by routine, with the kettle reduced to a backstop. The instant hot-water tap already exists among rivals; the non-obvious element is the anticipation, which makes the wait disappear rather than shrink.
- The machine (Figure 37): perceive (time of day, presence, past cups) → desired state (one cup at 95 °C when the person reaches the kitchen) → decide (heat now or hold) → act (reservoir heater) → verify (temperature at pour, cups wasted) → learn. Auto by default; manual pour.
- Frees: waiting, filling, switching. Blocks: standing energy, plumbing. Requires: a reservoir, a presence sensor, a routine model.
- Falsifier: 30 households, 60 days; failure if energy per cup exceeds the kettle's by more than 25% or if wasted cups exceed 10%.

Named innovations:

- **Routineboil** — the kettle that learns. Create: a presence sensor and a routine model in an ordinary kettle; it heats before the predicted cup and holds warm for a short window. Properties: the smallest change; keeps the object people own; zero plumbing.
- **Cupready** — the reservoir at the tap. Create: a one-cup insulated reservoir with a heater and a presence trigger, plumbed to the tap. Properties: zero wait; standing energy in the reservoir; plumbing required.
- **Heatvault** — stored heat with zero standby. Create: a phase-change core that stores a cup's worth of heat at temperature with minimal loss and releases it on demand. Properties: zero standby power once charged; the wait disappears; material and cost risk.

What is ruled out:

- Instant heating of a cup from cold on a domestic socket: a cup at 95 °C needs on the order of 80 kJ, which at 3 kW is about half a minute — the wait can move; power alone leaves it in place.
- Prediction with zero sensing: a schedule alone wastes cups on the days the routine breaks.
- Removing descaling in hard-water areas: any heated water path scales; the obligation moves rather than disappears.

The best path, and why:

- Best: Routineboil first (it holds against rivals on mechanism and moves on presence and waiting for its owners), Cupready for new kitchens (it moves on waiting and filling at the price of plumbing).
- Long-run best: Heatvault, if the phase-change core delivers the stated loss figures; it would remove both the wait and the standby cost that kill the instant tap on energy.
- Condition for the kettle to win: an irregular household with cheap electricity and hard water.

10.8 "A better toothbrush" → the mouth-health loop

- Five layers: object toothbrush · attribute better · function remove plaque · obligations remember twice daily, two minutes of technique, rinse, replace the head, visit the dentist · outcome the mouth stays healthy between visits.
- Desired state: the mouth stays within a health band between visits (loop).
- Dependencies changed: cleaning by daily human dexterity and memory → cleaning decided and performed by a device that senses; plaque as an unmeasured state → plaque as a measured state.
- Non-obvious path: a sensing mouthpiece that maps plaque and gum condition, cleans where needed for as long as needed, and escalates to a dentist when the band is left — the brushing event replaced by a regulation.
- The machine (Figure 38): perceive (optical or impedance plaque map, gum colour, breath markers) → desired state (plaque index under a threshold, gums within colour band) → decide (where, how long, which mode) → act (targeted cleaning) → verify (re-map) → learn; escalate to a professional when the band is left twice. Assist by default; manual mode for people who prefer it.
- Frees: technique, memory, guesswork. Blocks: device dependence, health data. Requires: plaque sensing that beats visual inspection.
- Falsifier: plaque index at three months against a powered brush in 80 users; failure if worse.

Named innovations:

- **Plaquemap** — the sensing mouthpiece. Create: a mouthpiece with optical or impedance sensing over the dental arch, targeted cleaning actuators, a two-minute cycle replaced by "as long as needed where needed". Properties: technique removed; plaque measured rather than assumed; charging and hygiene added.

- **Rinseloop** — the chemistry path. Create: a rinse that regulates the oral biofilm (enzymatic or probiotic), dosed by the mouthpiece's readings. Properties: complements rather than replaces mechanical disruption; simplest daily act.
- **Dentalink** — the escalation service. Create: the mouthpiece reports the plaque and gum band to a dental service that schedules a visit only when the band is left. Properties: visits by need rather than by calendar; a subscription; health data.

What is ruled out:

- Stopping plaque formation itself: biofilm forms on any surface in the mouth; the state can be held within a band, and stays there rather than vanishing for good.
- Replacing mechanical disruption entirely by chemistry: the evidence base supports chemistry as a complement.
- Marketing a medical claim outside the regulatory class: hygiene claims are open, treatment claims are gated.

The best path, and why:

- Best: Plaquemap with Dentalink. It moves on technique, memory and decision, and turns the calendar visit into an exception; it blocks with device dependence and health data.
- Runner-up: Rinseloop, which wins for people who refuse a device and loses on efficacy alone.
- Condition for the brush to win: a user with good technique who values the ritual — the same context term the coffee case carried.

10.9 "A quieter mower" → the lawn that keeps its own height

- Five layers: object mower · attribute quieter · function cut grass · obligations notice the height, fetch, push for 40 minutes weekly, empty, store, refuel · outcome a lawn that looks and feels right.
- Desired state: the lawn stays within a height band (loop).
- Dependencies changed: a person pushing a blade weekly → a regulation held by the garden; grass that grows fast → grass that grows slowly.
- Non-obvious path: the growth dependency itself — a slow-growing cultivar or a ground cover that tops out at a fixed height — with a small robot as the trimming actuator for edges and patches. The robot mower alone already exists; the combination attacks the cause of the obligation rather than its execution.
- The machine (Figure 39): perceive (height by camera, soil moisture, season) → desired state (height band, coverage) → decide (trim, water, over-seed) → act (robot for edges, irrigation) → verify (height map) → learn. Auto for the band, manual for the look.
- Frees: the weekly hour, the noise, the clippings. Blocks: re-seeding cost, a different lawn feel. Requires: a verified cultivar, a height sensor.
- Falsifier: mowing events per season over one full year in 20 gardens; failure if the reduction is under 70%.

Named innovations:

- **Lowgrow** — the cultivar path. Create: a slow-growing, low-height grass cultivar (breeding programmes exist) supplied as seed or turf, with the machine reduced to occasional trimming. Properties: attacks the cause; a lawn that stays a lawn; re-seeding cost.
- **Edgebot** — the small trimming actuator. Create: a low-cost robot for edges and patches, guided by a height map from a garden camera; it complements rather than replaces the cultivar. Properties: light; quiet; handles what the cultivar leaves.
- **Groundcover** — the species change. Create: a ground cover (clover, thyme, sedum mixes) that tops out at a fixed height, sold as a designed mix with a planting service. Properties: zero mowing; a different feel underfoot; pollinator value.

What is ruled out:

- Stopping ordinary grass from growing: growth is the plant's business; the obligation follows from the species chosen.
- A full robot mower on complex terrain with steps and beds: existing robots hold that ground and the method leaves them to the frontier row.
- A zero-maintenance lawn that stays a lawn: some trimming, watering and over-seeding remain in any horizon.

The best path, and why:

- Best: Lowgrow with Edgebot. It moves on the weekly hour, noise and clippings against the mower and against the robot mower alone, and it keeps the lawn feel that Groundcover gives up.
- Runner-up: Groundcover, which wins wherever the lawn feel is optional and pollinators are wanted.
- Condition for the mower to win: a rented garden where re-seeding is refused.

10.10 "A louder alarm" → the wake-up that needs zero alarm

- Five layers: object alarm clock · attribute louder · function wake a person · obligations set it, sleep with the deadline in mind, be shocked awake, snooze, get ready in a rush · outcome awake and ready by a deadline.
- Desired state: awake and ready by a deadline, context-conditional on the deadline.
- Dependencies changed: a sound at a fixed time → light and timing chosen by the sleep cycle; the deadline as fixed → the deadline as negotiated with the calendar.
- Non-obvious path: a sleep-wake loop in the bedroom — light rises at the moment the sleep stage allows within a window before the deadline; the deadline itself is moved earlier or later by the calendar when the night was short; the sound remains as the backstop.
- The machine (Figure 40): perceive (sleep stage by radar or wearable, calendar, commute time) → desired state (ready by t with a full cycle completed) → decide (wake window, light curve, deadline shift) → act (light, blinds, a message to the calendar) → verify (readiness report) → learn. Assist by default; manual alarm always available.
- Frees: the shock, the snooze, the fixed time. Blocks: sensing in the bedroom, calendar access. Requires: a reliable sleep-stage estimate.
- Falsifier: self-reported readiness and sleep quality at 30 days against a fixed alarm in 50 people; failure if either is worse.

Named innovations:

- **Dawnwindow** — light within a window. Create: a bedside light whose rise starts inside a window before the deadline and whose curve follows the season; sound remains as backstop. Properties: gentle; cheap; needs zero sensing.
- **Cyclewake** — sleep-stage timing. Create: a bedside radar or a wearable that detects the light sleep stage inside the window and triggers the light then. Properties: the shock removed; sensing in the bedroom.
- **Sleepcalendar** — the negotiated deadline. Create: a calendar link that shifts the first commitment when the night was short, within limits the user sets. Properties: attacks the deadline itself; social acceptance required.

What is ruled out:

- Waking with zero deadline: the deadline is the desired state's context; the method reports it as context-conditional.
- Detecting sleep stages with zero sensing: timing needs a signal; a schedule alone is Dawnwindow.

- Removing the backstop: a wake-up with zero fallback fails its falsifier on the first missed flight.

The best path, and why:

- Best: Dawnwindow with Cyclewake. It moves on the shock and the snooze against the alarm and against light-only rivals, and keeps the sound as a backstop.
- Long-run best: Sleepcalendar, which moves the most because it attacks the cause, and wins only where employers and families accept negotiated starts.
- Condition for the alarm to win: shift work with fixed clock-ins.

10.11 The umbrella across the whole need chain → the canopy garment (directions 4, 5, 6)

- Need chain: rain and sun (protection), the hand (carrying), wind (failure), the eye (identity).
- Desired state: protected from rain and sun while moving, hands free, wind-proof, seen — a bundle across four chains.
- Dependencies changed: shelter as a separate carried object → shelter as part of what is worn; a rib structure that inverts in wind → a structure that yields and returns; the object as a tool → the object as a garment with identity.
- Non-obvious path: a garment with a deployable brim and hood — a structured cape or coat whose canopy folds into the collar, shades in sun, sheds rain, holds shape in wind by yielding, and is designed first as fashion. The umbrella's four chains are served by one object that is already carried because it is worn.
- The machine (Figure 41): perceive (weather ahead, sun angle, wind) → desired state (dry, shaded, hands free) → decide (deploy or stow) → act (brim and hood deploy; the wearer can override) → verify (wet? warm?) → learn. Manual by default — a garment stays a garment; assist for the deployment.
- Frees: carrying, forgetting, losing, breaking; the fine-weather burden. Blocks: the garment must be worn that day; fashion cycles; cleaning. Requires: a fabric and structure that deploy and hold in wind.
- Falsifier: 40 wearers, one season; failure if fewer than 50% choose it over an umbrella on wet days, or if identity ratings fall below a standard raincoat.

Named innovations:

- **Brimcape** — the structured cape. Create: a cape whose collar holds a folded brim and hood in a yielding frame; deployed by hand, held in wind by giving way and returning. Properties: identity first; covers head and shoulders; shades in sun.
- **Hoodshell** — the jacket path. Create: a jacket with a rigid, foldable hood-brim and a water-shedding shell; the least fashion, the most function. Properties: fits existing wardrobes; cheaper; smaller canopy.
- **Fieldcoat** — the air-supported canopy. Create: a coat whose canopy is a lightweight inflatable frame that deploys from the collar with a hand pump or a small blower. Properties: the largest canopy; the most mechanism; the most risk.

What is ruled out:

- Shading or sheltering a second person: a garment covers its wearer; the umbrella keeps that path.
- Full wind immunity: any canopy yields to a gust; the design goal is returning rather than resisting.
- Being worn on the day it was left at home: the garment removes the carrying obligation only on the days it is chosen — the falsifier measures exactly this.

The best path, and why:

- Best: Brimcape for buyers who buy on identity, Hoodshell for buyers who buy on function; both move on hands, memory and durability against the umbrella.
- Runner-up: Fieldcoat, which wins on canopy size and loses on mechanism, cost and the falsifier's wind test.
- Condition for the umbrella to win: two people, one canopy.

10.12 The Monobloc pattern applied → the one-piece seat-step-table (direction 2)

- Need chain of a household stool: sit (primary); reach a shelf (a step); hold a cup (a side table); store (stack); move (light); survive a balcony (weather); cost (replaceable); look right (identity, the Monobloc's weak chain).
- Desired state: one object that serves sitting, stepping, tabling and stacking, outdoors and indoors, at Monobloc cost, with the identity chain served this time.
- Dependencies changed: one need per object → one object per bundle; assembly and care → a single moulded piece; the mould as a cost centre → the mould as a design object shared by local producers.
- Non-obvious path: a one-piece moulded form whose geometry is a stable step at one orientation, a seat at another and a table at a third, stackable in all, in colours and a surface chosen with designers rather than after them.
- The machine: there is none in the object — the machine here is the production system: one mould, local producers, a design that is the same everywhere; the bundle is held by geometry rather than by control.
- Frees: three objects, assembly, storage, the fear of leaving it outside. Blocks: recyclability of the polymer, the aesthetic risk the Monobloc met. Requires: a geometry that is safe as a step and comfortable as a seat.
- Falsifier: 30 households, 90 days; failure if the object is used for fewer than two of its three functions by most households, or if identity ratings fall below the Monobloc's.

Named innovations:

- **Trimono** — the three-orientation piece. Create: one moulded form that is a seat on one face, a step on another and a low table on a third, stackable in all, load-rated as a step. Properties: three objects in one; the Monobloc's bundle plus identity if the surface is designed.
- **Stackstep** — the seat-step. Create: a two-orientation piece, seat and step only, simpler to make safe. Properties: fewer compromises; higher safety margin; less optionality.
- **Monobloc-plus** — the identity path only. Create: the Monobloc geometry with a designed surface, colour range and a recycled-content polymer, sold with the identity chain served. Properties: zero new function; the missing chain filled; cheapest to launch.

What is ruled out:

- A geometry that is comfortable as a chair, safe as a step and stable as a table with zero compromise: the three load cases pull the form in different directions; Trimono accepts a modest seat.
- Polymer creep under long static load and a step's safety rating in one part: the rating decides the thickness, the thickness decides the cost.
- Recyclability of a mixed polymer: a one-material design is the only route to the recycling chain.

The best path, and why:

- Best: Trimono if the step load rating passes at Monobloc cost; it moves on three obligations (three objects, storage, care) and closes the Monobloc's identity gap.
- Runner-up: Monobloc-plus, which moves on identity alone at almost zero risk and is the right first release.

- Condition for the plain Monobloc to win: price alone, as it always has.

10.13 The necklace that is a cable → the shape-holding connector (direction 3)

- Need chain of a necklace or collar: be seen (identity), feel finished, mark belonging — a purely decorative object with zero primary function; and, separately, the daily need to carry a charging cable and to prop a phone.
- Desired state: an object worn for identity that also connects and holds shape.
- Dependencies changed: the decorative object as inert → the decorative object as a working cable; the cable as a thing to carry and lose → a thing already on the body; the phone stand as an extra object → the cable itself, shaped.
- Non-obvious path: a necklace or collar that is a certified charging and data cable with a shape-memory core, so that it drapes as jewellery, closes as a collar, coils as a stand and hooks as a hanger — the form as function. Precedents for direction 3 exist (rings that measure sleep, rings that pay); the necklace as cable is open.
- The machine (Figure 42): small but present — perceive (what is connected, current drawn, temperature at the neck) → desired state (power delivered within limits, the object cool) → decide (negotiate current) → act (deliver, cut) → verify (temperature, connector wear) → learn. Manual for the shape; auto for the safety.
- Frees: carrying a cable, forgetting it, a stand. Blocks: conductor near the skin (safety rules), connector wear from daily bending, fashion cycles. Requires: a bend-rated conductor, a shape-memory core, jewellery finishing.
- Falsifier: 50 wearers, 60 days; failure if the cable fails certification after cyclic bending, or if fewer than 40% wear it on days they have zero need of a cable.

Named innovations:

- **Linkchain** — the chain-link cable. Create: a necklace whose links are a jacketed conductor with bend-rated joints and hidden connectors at the clasp; drapes as a chain, charges as a cable. Properties: the most jewellery; the least cable.
- **Formcord** — the shape-holding cord. Create: a cable with a shape-memory alloy core that holds any bend — coiled as a phone stand, hooked as a hanger, closed as a collar. Properties: the form as function; the phone stand removed; the most novel.
- **Powerpendant** — the pendant battery. Create: a pendant that is a small battery with a short cable stowed in the chain; a charge on the body. Properties: power rather than connection; heavier; the strongest daily use.

What is ruled out:

- High-power charging in contact with skin: heat and safety rules cap the current; jewellery charges slowly.
- High-speed data through jewellery connectors: bend cycles and cosmetic finishes limit the connector class.
- Fashion durability of a connector: clasps wear; the jewellery cycle is shorter than the cable's rating in any case.

The best path, and why:

- Best: Formcord — it changes two dependencies (the cable carried, the stand as an object) and is the only one of the three with a path the rings have left open.
- Runner-up: Powerpendant, which moves on the daily need and loses on weight and identity.
- Condition for the plain necklace to win: the buyer who wants the object inert — identity with zero function is itself a path, and the method keeps it in the population.

10.14 The earring that hears → hearing and audio as jewellery (direction 3, with direction 4)

- Need chain of an earring: identity and adornment; separately, the needs to hear better (millions of people), to take calls, to listen — served today by hearing aids that carry stigma and earbuds that are carried and lost.
- Desired state: worn for identity, and it hears, amplifies and plays.
- Dependencies changed: the hearing device as a medical object to hide → an object to show; the earbud as a thing carried → a thing worn all day; the identity chain empty → the identity chain first.
- Non-obvious path: an earring whose body is the transducer, the microphone array and the battery, designed by jewellers with the audio engineered around the form; hearing assistance, calls and music as modes of one worn object. Precedents: hearing aids in jewellery form and audio earbuds exist as separate categories; the merger with identity first is the added element.
- The machine (Figure 43): perceive (sound field, the wearer's hearing profile, the room) → desired state (speech intelligible, music as wanted, the ear comfortable) → decide (gain per band, noise handling, mode) → act (transducer) → verify (the wearer's adjustments, wear time) → learn. Assist by default; the wearer sets the identity.
- Frees: the stigma, the carrying, the choosing between adornment and hearing. Blocks: charging, hygiene, regulatory class for hearing assistance. Requires: transducer and battery inside jewellery mass; a fitting process.
- Falsifier: 60 users with mild hearing loss, 90 days; failure if daily wear time is under six hours or if speech intelligibility is worse than a standard aid.

Named innovations:

- **Earpearl** — the pearl-form hearing earring. Create: transducer, microphone array and battery inside a pearl-sized body on a post; fitted by an audiologist; modes for speech, quiet and music. Properties: identity first; mild-loss assistance; daily charging.
- **Hoopbuds** — the hoop earring with audio. Create: a hoop whose lower arc carries a bone-conduction or open-ear transducer; calls and music with the ear open. Properties: audio rather than hearing assistance; open ear; identity.
- **Clipaid** — the clip-on for unpierced ears. Create: the Earpearl electronics in a clip form. Properties: the largest addressable group; comfort risk over a day.

What is ruled out:

- Full-power hearing aids for severe loss in jewellery mass: the battery and the gain decide; the method limits the claim to mild and moderate loss.
- Treatment claims outside the regulatory class: hearing assistance is gated; audio is open.
- Earbud-grade isolation with an open ear: the open ear is a design choice with a sound-quality price.

The best path, and why:

- Best: Earpearl for mild loss — it moves on stigma, carrying and the choice between adornment and hearing against the hearing aid and against earbuds; it blocks with charging and a fitting.
- Runner-up: Hoopbuds, which wins for audio-first buyers and loses on assistance.
- Condition for the hearing aid to win: severe loss.

10.15 The hanger that finishes → garment care while the garment rests (directions 1, 4)

- Need chain of a hanger: hold the garment's shape; store it; keep it visible; and, upstream, the garment must arrive wearable — which today costs ironing, steaming and a decision every morning (Section 6, garment care).

- Desired state: the garment is wearable when taken off the hanger (an invariant shared with the Versuni garment case).
- Dependencies changed: finishing as an event with a hot tool in a hand → finishing as a state held where the garment rests; the wardrobe as storage → the wardrobe as the machine.
- Non-obvious path: a hanger that steams, tensions and dries the garment overnight, reports what is ready, and is beautiful enough to be the visible part of the wardrobe — the finishing closet of Section 6 reduced to the one object every garment already touches. Precedents: styling cabinets exist; the hanger as the unit is the added element, and it fits the wardrobe people already have.
- The machine (Figure 44): perceive (fabric, crease map by a small camera, humidity) → desired state (wearable by morning) → decide (steam, tension, dry, time) → act (hanger actuators; the wardrobe supplies water and power) → verify (crease map) → learn. Auto by default; manual for delicate garments.
- Frees: ironing, the morning decision, the styling cabinet's footprint. Blocks: water and power in the wardrobe, a per-hanger cost. Requires: low-power steam at the hanger, a crease sensor.
- Falsifier: 50 households, 60 days; failure if ironing episodes drop by less than 50% or if garments show damage.

Named innovations:

- **Tensionhanger** — mechanical finishing. Create: a hanger with shoulder and hem tension that removes light creases overnight using tension alone (zero water, zero power). Properties: cheapest; fits any wardrobe; light creases only.
- **Steamhanger** — the finishing hanger. Create: a hanger with a low-power steam element and a crease sensor, fed from a small tank in the hanger or a wardrobe line. Properties: removes most creases; power and water per hanger.
- **Wardrobeloop** — the wardrobe as machine. Create: a wardrobe rail that supplies steam, air and power to every hanger, with one crease camera for the rail. Properties: the full loop; new wardrobes or a retrofit rail; the finishing closet dissolved into the wardrobe.

What is ruled out:

- Removing deep creases in heavy wool with tension alone: heat and moisture are required by the fibre.
- Water and power in every existing wardrobe: the retrofit decides the addressable market.
- Damage-free finishing of every fabric on one setting: fabric sensing is required, and delicate garments stay manual.

The best path, and why:

- Best: Wardrobeloop where the wardrobe is new or the rail can be retrofitted — it moves on ironing, the morning decision and the cabinet's footprint against the iron, the steamer and the styling cabinet.
- Runner-up: Steamhanger as the middle path; Tensionhanger as the entry that wins on price and loses on wool.
- Condition for the styling cabinet to win: a household with space and a preference for a closed appliance.

Figure 36 · From 'a stronger umbrella' to the dry-path service

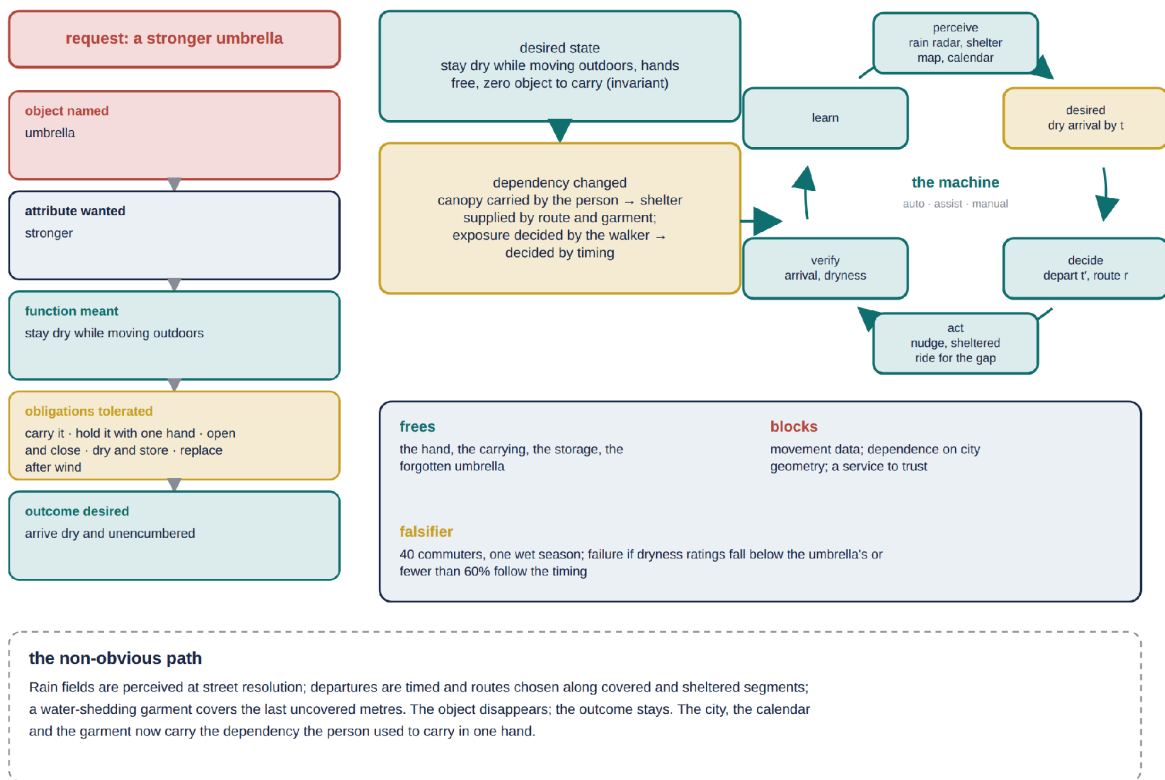


Figure 36. The method in plain words: the coffee example beside each step. Seven steps in the left column; what the coffee-machine team writes at each step in the right column, from the use path to the falsifier. Derived from Section 5.0.

Figure 37 · From 'a faster kettle' to hot water that precedes the wish

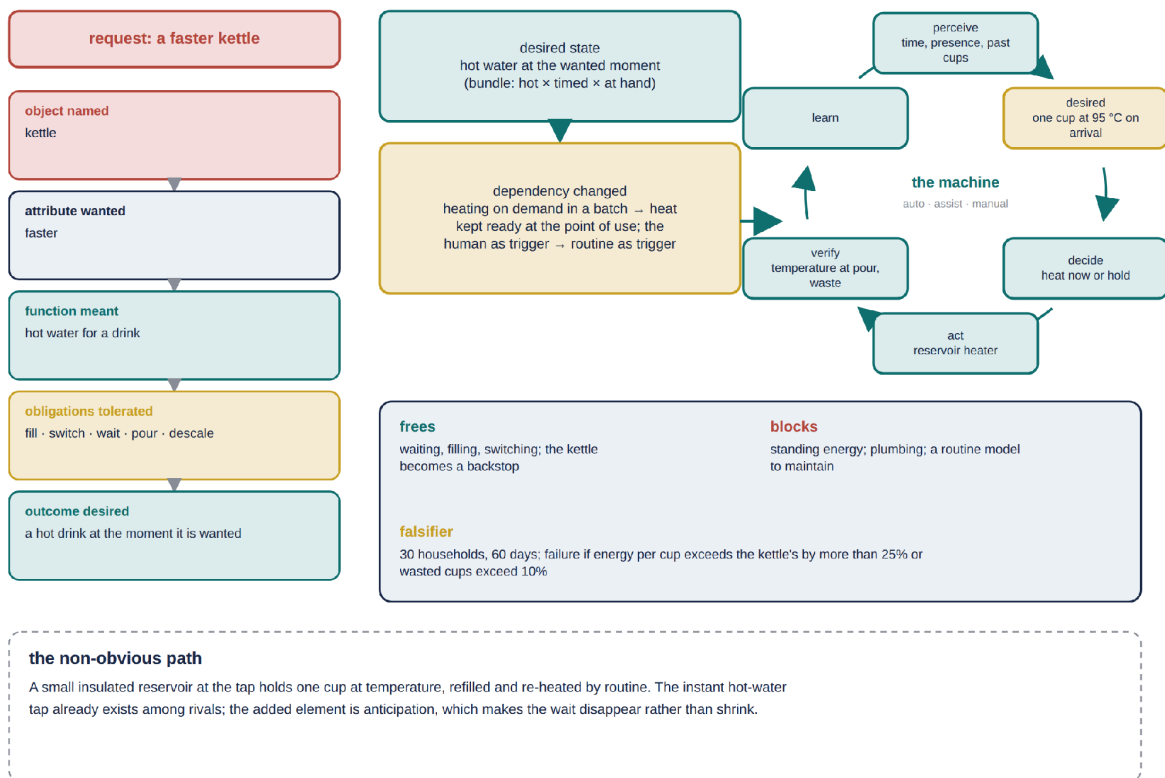


Figure 37. One engine, three scales. The portfolio inventory feeds the obligation inventory and the dependency map; shared dependencies become system innovations; internal processes enter as use paths; the ninety-day plan on the timeline. Derived from Sections 11.2–11.3.

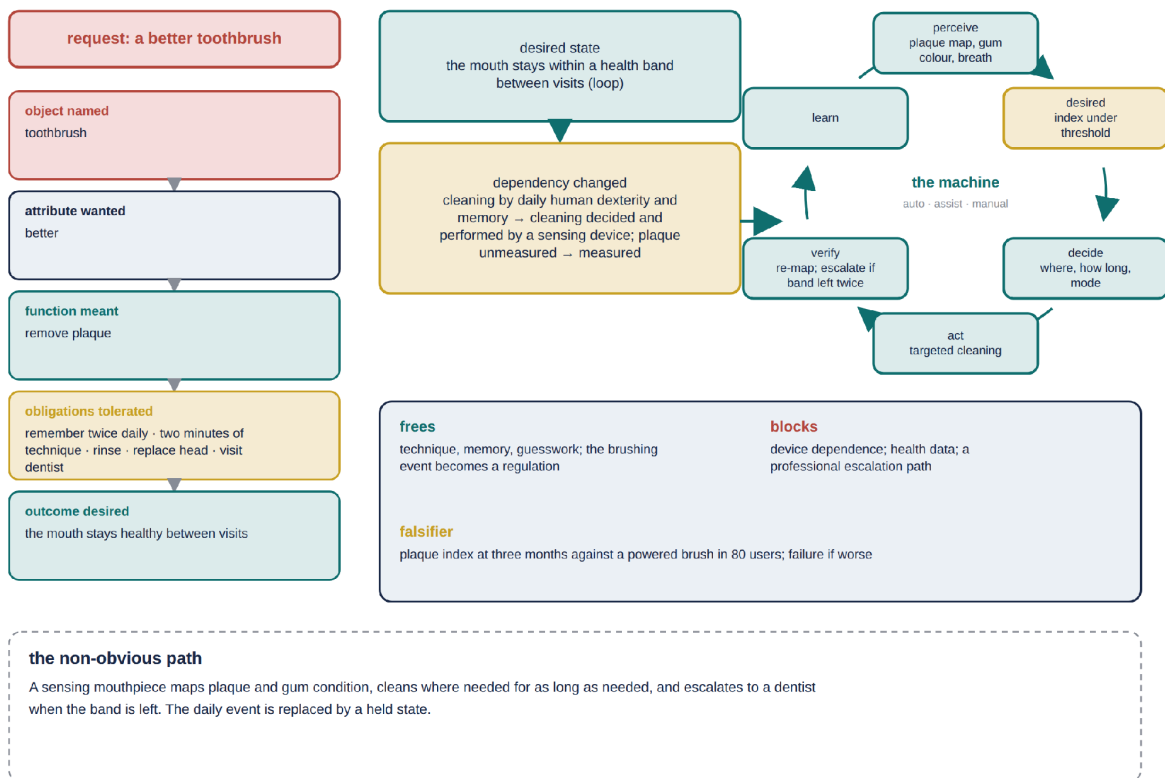
Figure 38 · From 'a better toothbrush' to the mouth-health loop

Figure 38. World paths, perceived paths, distortion. One use space drawn twice: world-admitted transitions (grey) and team-perceived transitions (colour); five distortion types marked at their edges (omission, false path, aliasing, latency, blockage treated as absolute). Prior programme measurement: three bodies over one world, 950/948/972 admitted transitions against 917/264/199 carried [57].

Figure 39 · From 'a quieter mower' to the lawn that keeps its own height

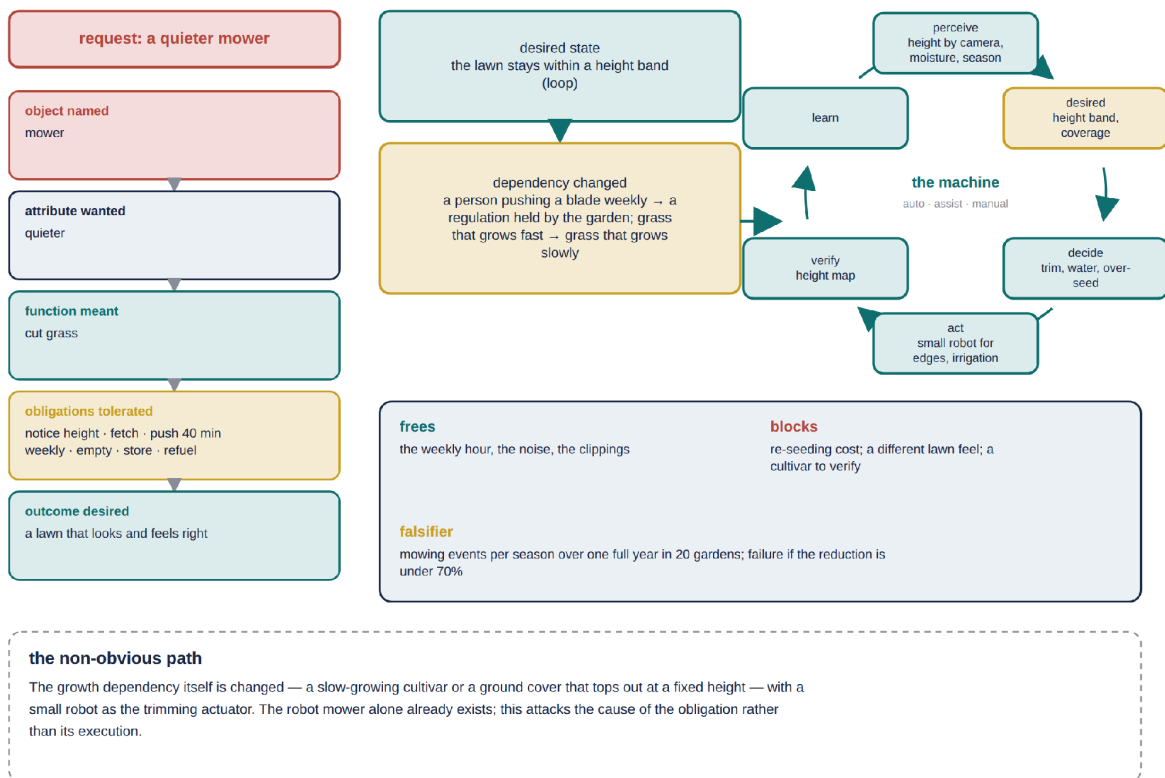


Figure 39. The need chain of an umbrella. One object, five chains: rain, sun, the hand, the wind, the eye; each chain ends in a need, and the bundle is what the object is bought for. Conceptual.

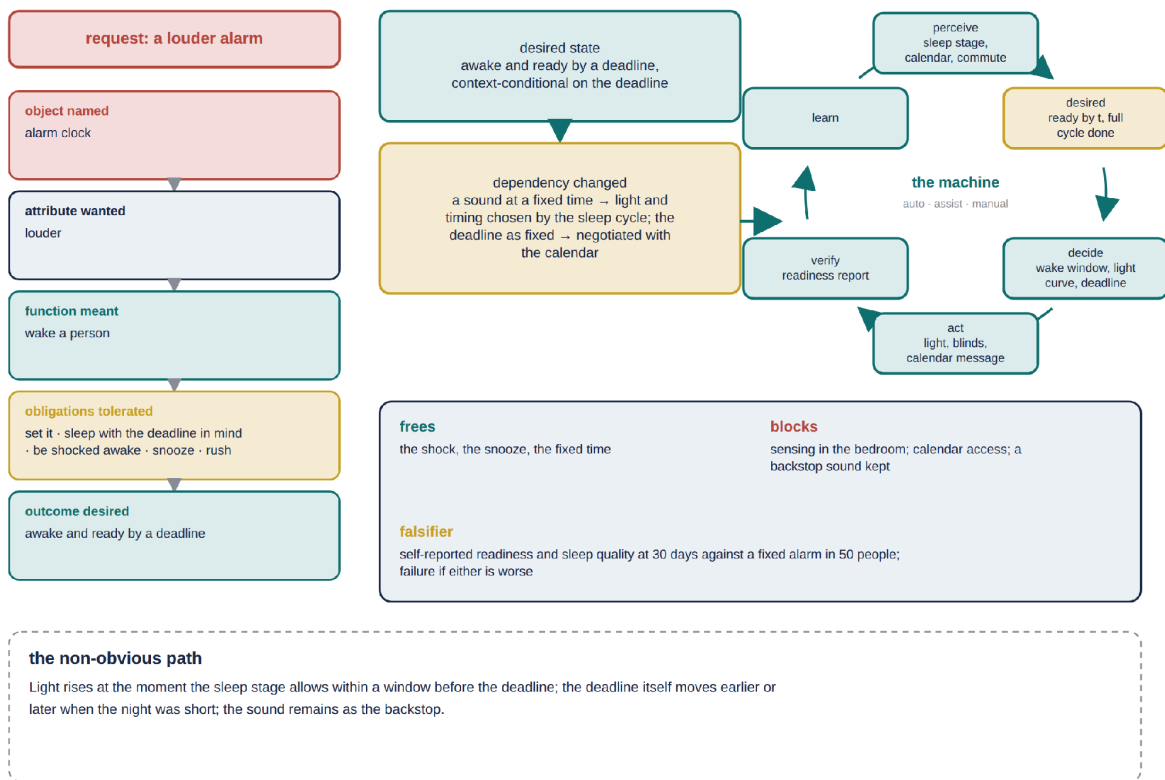
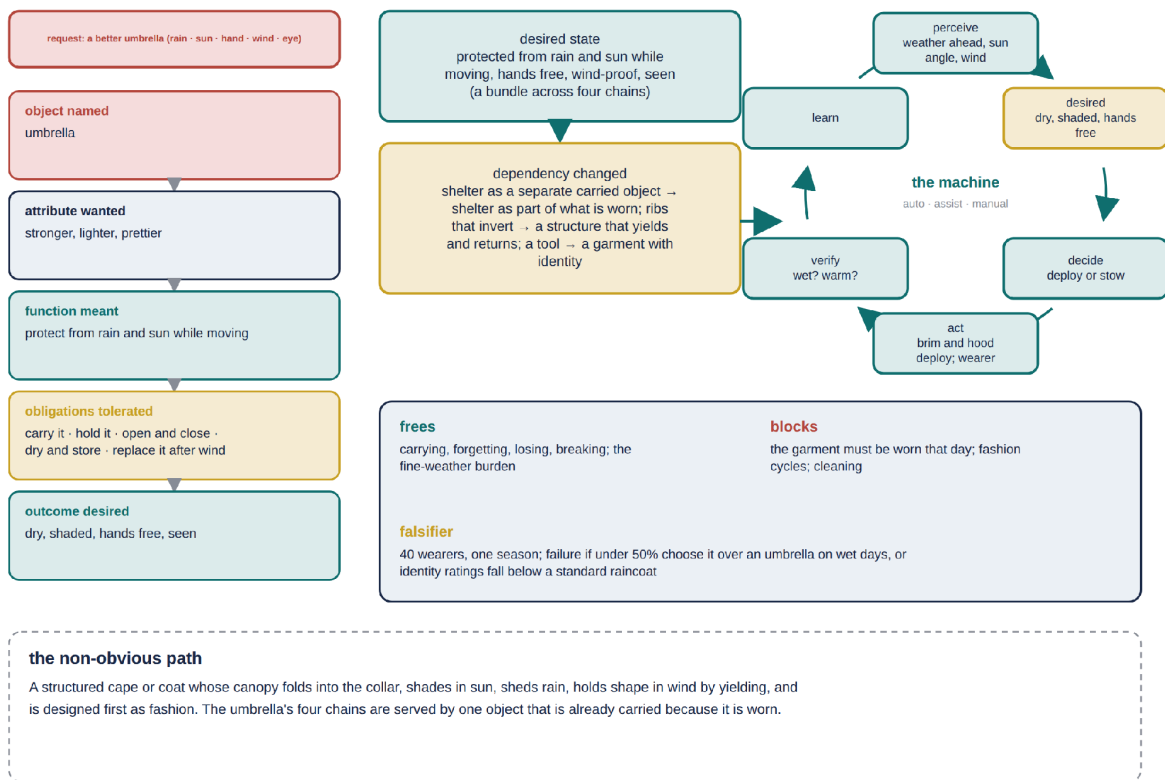
Figure 40 · From 'a louder alarm' to the wake-up that needs zero alarm**Figure 40. The wake-up that needs zero alarm.** From "a louder alarm" to a sleep-wake loop of light, timing and a negotiated deadline, with the sound kept as backstop. Method-generated hypothesis.

Figure 41 · From the umbrella's whole need chain to the canopy garment

method-generated hypothesis · auto / assist / manual preserved · the human keeps veto

Figure 41. The Monobloc lesson. The seat as the ticket to play; the bundle — cost, weight, stacking, weather, stability, local manufacture, replaceability — as the game; the identity chain left empty as the block. Facts from [78].

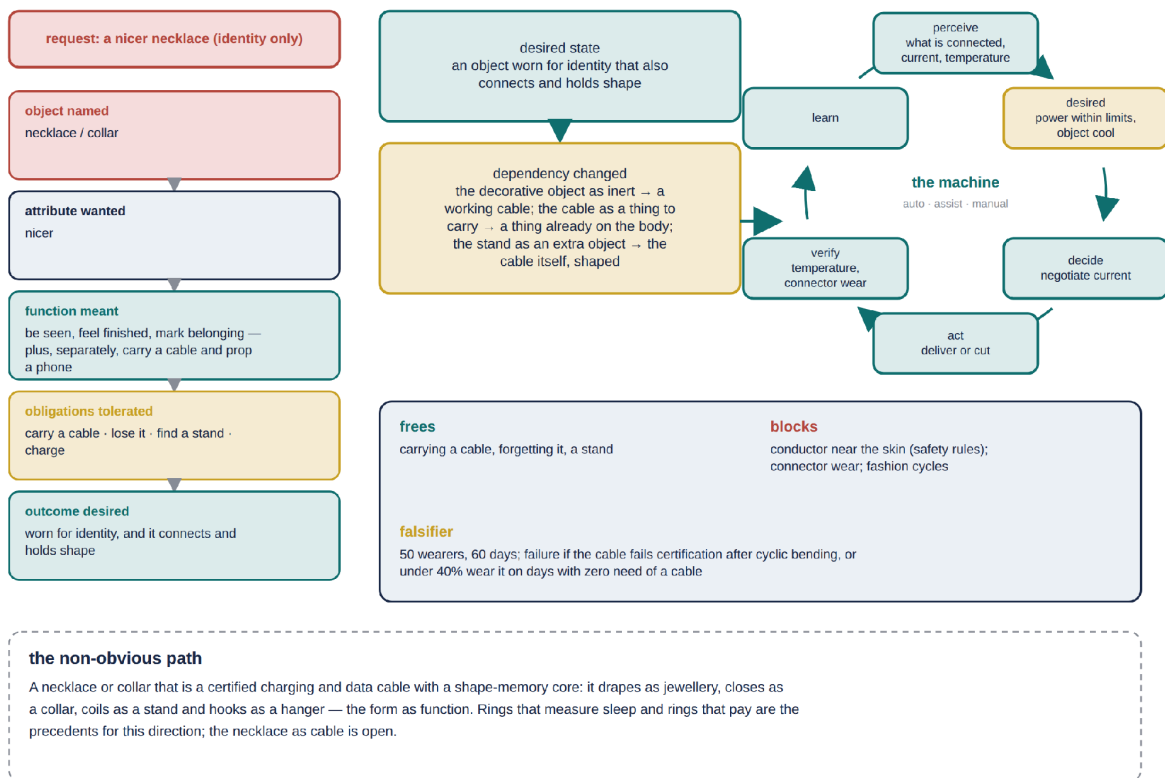
Figure 42 · From a plain necklace to the cable that holds its shape

Figure 42. Six directions of non-obvious innovation. Remove an obligation; maximize the bundle; give a function to the decorative; give a bundle to the functional; attack the cause upstream; attack the failure chain — each a different dependency change, with the worked example that illustrates it. Derived from Section 10.4.

Figure 43 · From an earring to hearing and audio as jewellery

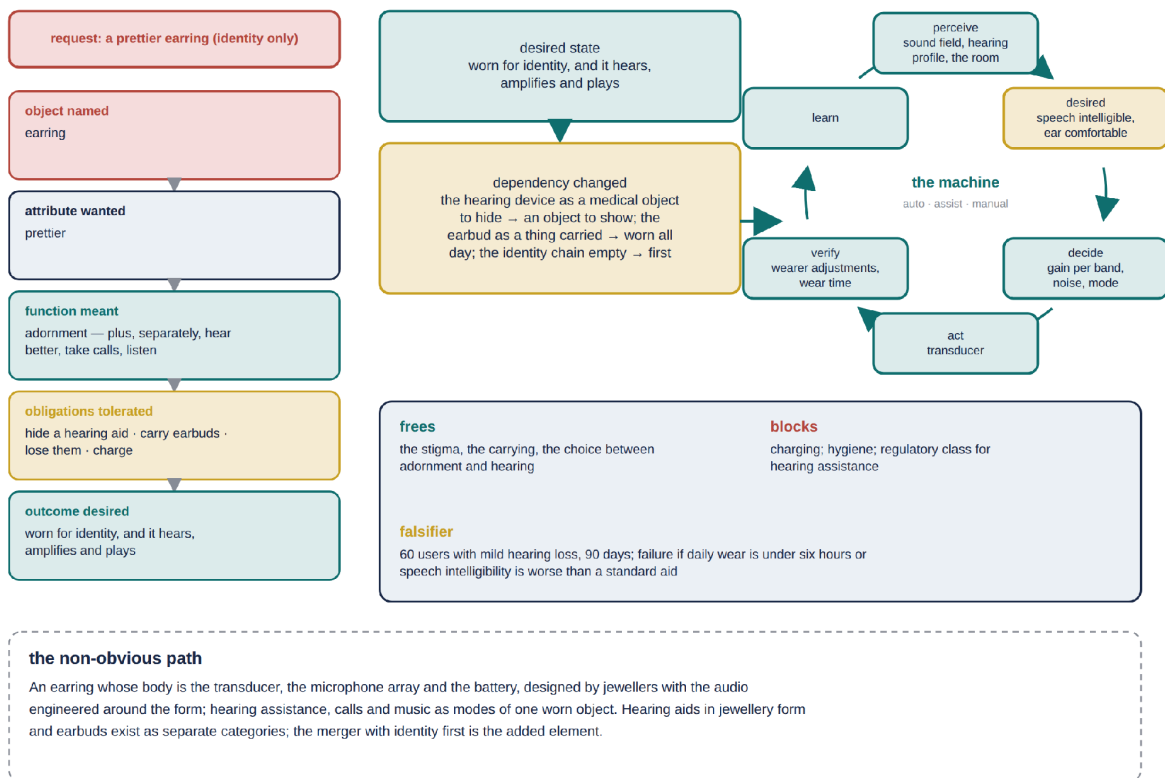


Figure 43. The canopy garment. The umbrella's four chains served by one worn object; the machine that deploys and stows; frees, blocks, falsifier. Method-generated hypothesis.

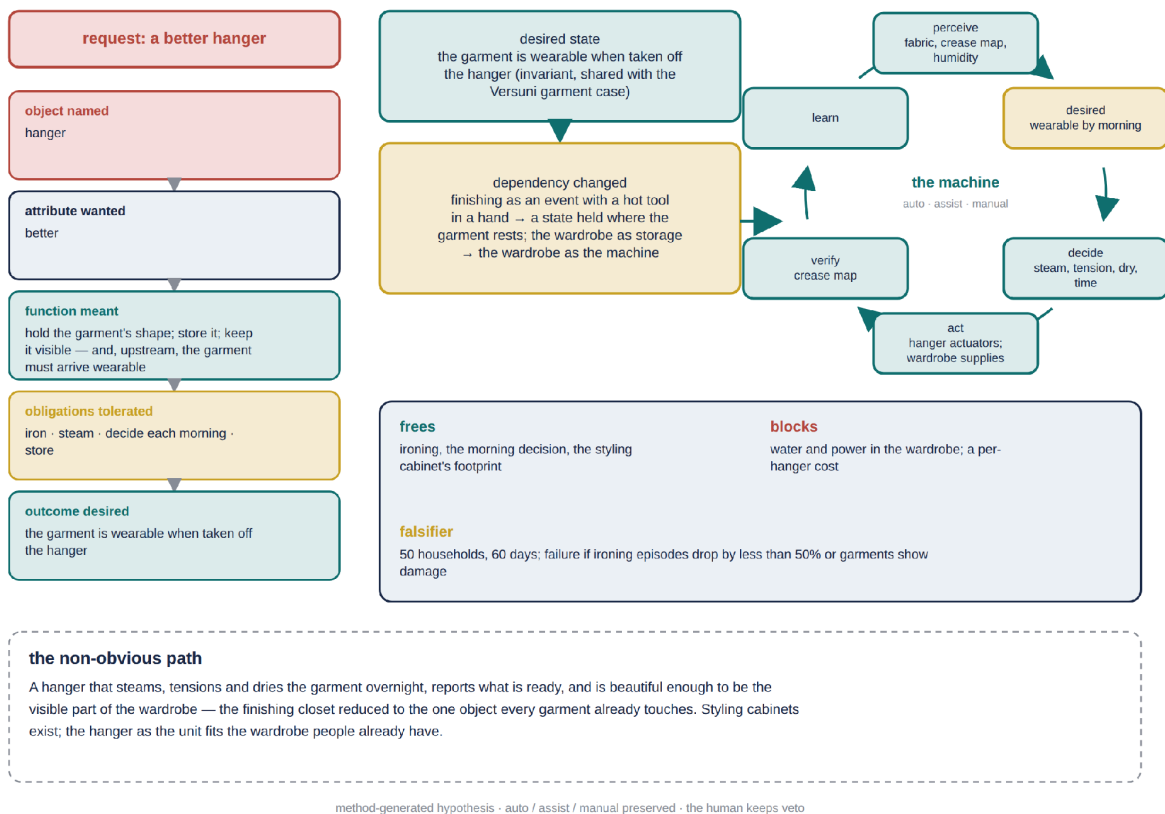
Figure 44 · From a hanger to garment care while the garment rests

Figure 44. How fixation poisons innovation. The causal chain from exposure to displacement: exposure anchors the representation → the search stays inside the embodiment → obligations stay invisible → the market blind spot → displacement by a dependency change; a second chain shows the same mechanism inside generative systems; the four points where the method intervenes are marked. Derived from Section 1.2 with references [1–9, 29, 74–77].

11. From method to machine: what a company does first

The ten innovations of Section 10 were built one at a time. A company needs the machine that builds them: a standing process that runs the dissection across every product and every internal path on a cadence. This section says what to build and in which order.

Key points.

- The ten innovations of Section 10 and the six Versuni families of Section 6 share one architecture: perceive → desired state → decide → act → verify → learn, with auto / assist / manual preserved.
- A company builds the innovation machine before it builds any product machine: a standing process that dissects every product and every process on a cadence, produces test cards rather than feature lists, and kills faster horses by rule.
- The first ninety days are an inventory: every product, every area of work, every use path, every tolerated obligation.

11.1 The innovation machine (Figure 45)

- Radar — inputs: product facts from primary sources, one observed use episode per product, the frontier set (own best, rival, substitute), and the company's own processes written as use paths.
- Dissection engine — the seven steps of Section 5 run per product and per process on a cadence; the five-layer worksheet (Figure 29) as the intake form.

- Population — the candidates produced by the six dependency changes, each naming the dependency it changes; feature additions struck out by rule.
- Competition — the survival test against the frontier at the same desired state; verdicts moves / holds / loses.
- Test cards — one falsifier per survivor, budgeted; the test replaces the feature on the roadmap.
- Learn — results of tests, kills and their reasons, and the obligation inventory feed the next cycle.
- Governance — three metrics: obligations removed per release, causal distance of the roadmap, share of roadmap items that are feature additions (target: falling).

11.2 What to do first: the ninety-day inventory (Table 8)

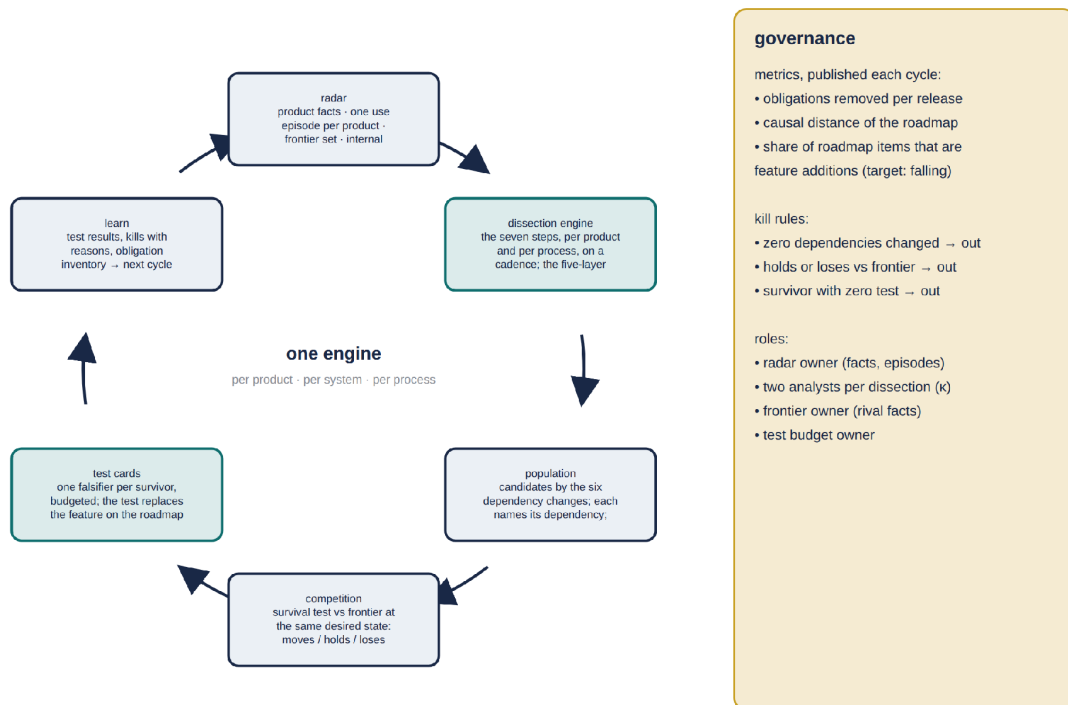
Table 8. The first ninety days.

Weeks	Action	Output	Gate
1–2	List every product and every area of work; for each product name the family, the desired state as currently claimed, and the owner	The portfolio inventory	Every product has an owner and a claimed desired state
3–4	For each product, observe one real use episode and write the use path with the agent of each transition; do the same for three internal processes (a support call, a repair, a release)	Use-path library	One agent per transition; primary-source mechanisms
5–6	Fill the five-layer worksheet for every product and process; list at least five obligations each	Obligation inventory across the portfolio	Each obligation tied to one dependency
7	Cluster obligations by dependency across products (consumables, presence, cleaning, scheduling, interpretation of readings); the clusters are the company's shared path creations	Dependency map	Each cluster names the products it touches
8–9	Run the ladder, probes and stopping rule per product; fix the level; type the requirements	Desired states and Δ^* per product	$\kappa \geq 0.6$ between two analysts on a sample
10–11	Generate by the six changes; write ledgers; run the survival test against the frontier	Candidate population with verdicts	Feature additions struck out; every survivor has a ledger
12	Write one test card per survivor; put the tests on the roadmap; define the cadence for the next cycle	The first test budget; the machine's calendar	The roadmap's feature-addition share is measured and published

11.3 System and process innovations

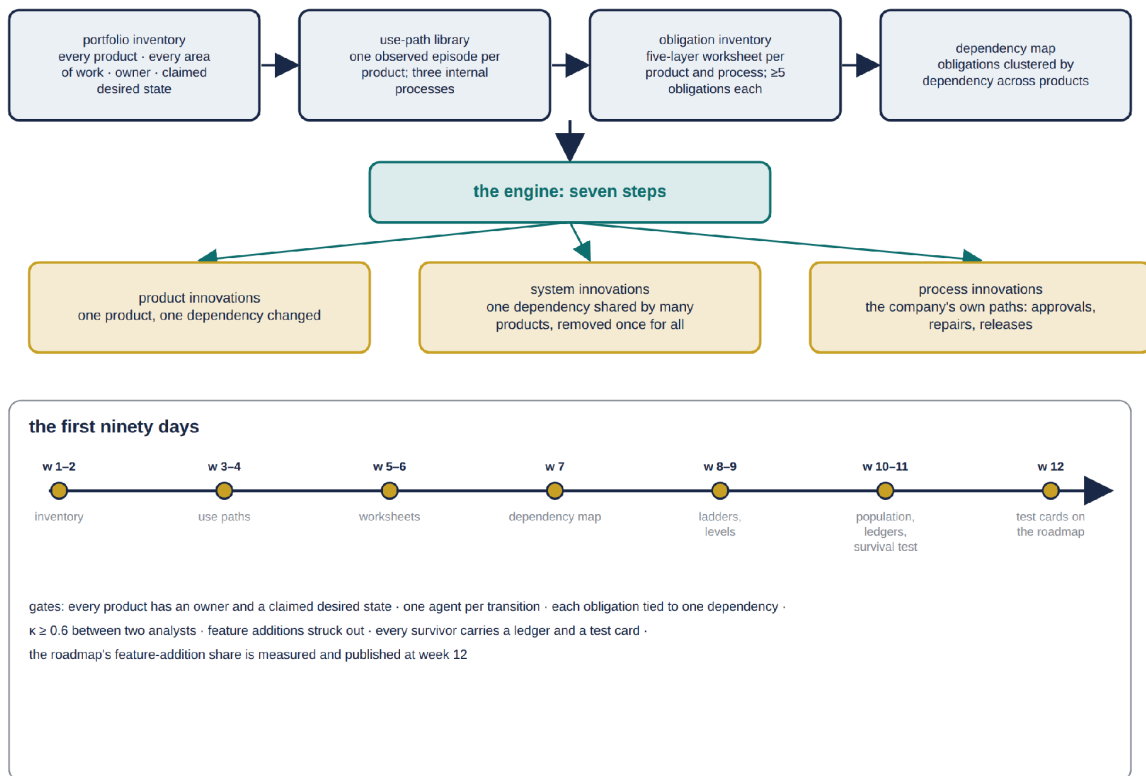
- A process is a use path with agents, and the method applies unchanged: the request "a faster approval" is the faster horse of a workflow; the desired state is "the decision reached by time t with the risk within band"; the obligations are hand-offs, waiting, re-entry of data, sign-offs; the dependency changes are the same six.
- System innovations appear when the dependency map of week 7 shows one dependency shared by many products — presence, consumable state, cleaning, scheduling — and a single machine removes it for all of them at once; this is the route by which product loops converge into the environment loop of Section 7.2.
- The machine of Section 11.1 therefore runs at three scales: product, system (shared dependencies), and process (the company's own paths), with one engine and one inventory (Figure 46).

Figure 45 · The innovation machine



The machine produces test cards rather than feature lists; faster horses die by rule at the population stage.

Figure 45. The problem-solution innovation card: the umbrella. One card in the reading-guide format: the dissection line, three named innovations with how to create them and their properties, what is ruled out, and the verdict with the runner-up and the condition under which the umbrella wins. Method-generated hypotheses.

Figure 46 · One engine, three scales, and the first ninety days**Figure 46. The verdict board for the ten.** Ten problems, the best path named for each, the runner-up, and what is ruled out, on one board; verdicts are ordinal outcomes of the survival test. Method-generated hypotheses.

12. The non-obvious forecast: innovation jump, with Apple, Versuni and OpenAI

Section 11 built the machine for one company's portfolio. This section turns the same machine outward and asks what it predicts: for a given company, from its present position, which innovation is most likely to work next — and which directions to leave alone. The forecast is built on lightness rather than on trends, and it is calibrated on innovations that disappeared.

Key points.

- A trend forecast extrapolates what is being bought; the non-obvious forecast asks which obligation will be dissolved next, by whom, from which position. The two diverge exactly where innovation happens.
- The forecasting principle is the lightest path: buyers move to the embodiment that gives them the most paths at the least obligation, according to their perception. People want the most with the minimum; the forecast names the embodiment that supplies it.
- The basis point is the company's present position — the desired states it already holds, the obligations it leaves in place, and the dependency changes it alone can make. A jump is measured from there.
- Calibration: products that disappeared (the whale-oil candle, the stovetop coffee maker, the slow oven, the stand-alone camera) all lost to an embodiment that served the primary need with fewer obligations and either took or dissolved the identity chain.
- Three forecasts follow — Versuni, Apple, OpenAI — each with directions to go, directions to leave alone, a disappearance risk, and the one jump most likely to work. All are method-generated hypotheses with falsifiers.

12.1 What the forecast is, and how it differs from a trend

- A trend is a slope on a sales curve; it says what is being bought more this year. Extrapolated, it says "more of that". A trend forecast is a faster-horse forecast: it improves the attribute the market is already rewarding.
- The non-obvious forecast is a statement about dependencies: which obligation, tolerated today by the buyers of a category, will be dissolved next, by which embodiment, and from which company's position. It is derived by running the dissection of Section 8 on a company rather than on a product.
- The forecasting principle (Figure 47): among the embodiments a buyer can perceive, the buyer moves to the one that offers the most paths at the least obligation — the lightest path in the sense of Section 4. Trends enter only as inputs to what becomes possible (a sensor becomes cheap, a battery becomes light); the direction is set by lightness, and the direction is what a company can plan on.
- The forecast therefore makes three kinds of claim: a jump (the dependency change most likely to work from this company's position), directions to leave alone (paths that are feature additions, paths rivals already hold with a better bundle, paths the physics or the law close, paths outside the company's capabilities), and a disappearance risk (which of the company's own products a lighter embodiment could delete).

12.2 Calibration: great innovations that disappeared

The forecast is only worth trusting if it also predicts loss. Four products that vanished, read through the model (Figure 48):

- The spermaceti (whale-oil) candle — the brightest, cleanest flame of its century — disappeared under kerosene lamps and then the electric bulb. The primary need (light after dark) held still; the obligations removed were trimming, smoke, refilling, fire risk, and a supply chain that hunted whales; the bulb also made the identity chain of a fine candle irrelevant by making light unremarkable.
- The stovetop coffee maker of the grandmother's kitchen — the percolator and the moka pot — retreated before capsule and bean-to-cup machines. The primary need (a hot coffee) held; the obligations removed were grinding, timing, watching the stove and cleaning; the moka pot survives in the ritual chain, as a context-conditional case — exactly the term the coffee dissection carried in Section 6.
- The slow oven, with its long pre-heat and its hour, gave up whole categories of meals to the microwave and then to the air fryer. The primary need held; the obligations removed were pre-heating, waiting and oil; the oven survives for what the small cavities serve poorly, which the ruled-out list of Section 10 would have predicted.
- The stand-alone camera for ordinary photography disappeared into the smartphone. The primary need held; the obligations removed were carrying a second object, transferring the image and choosing to bring the camera; the phone's carrying cost was already paid for other reasons, so its optionality came free — the purest case of Section 10.2.

The disappearance test that follows, stated as three conditions a product must fear together:

- Another embodiment serves the primary need with fewer obligations.
- That embodiment takes the identity chain or makes it irrelevant.
- That embodiment is already carried or owned for another reason, so its carrying cost is near zero.

A product that meets all three stays only where a context term (ritual, craft, identity) holds a niche; the forecast treats that niche as real and small.

12.3 The positioning dissection: the basis point

A company is dissected with the same five layers as a product, read at the portfolio level (Table 9; Figure 49). The result is the basis point from which a jump is measured.

Table 9. The positioning dissection.

Layer	Question for a company	What it yields
Position (the ticket)	Which desired states does the company already hold for its buyers, and through which embodiments?	The list of tickets to play
Obligations left in place	Across the portfolio, what must buyers still perceive, decide, remember, move, operate, verify, clean, maintain and pay?	The obligation inventory (Section 11.2) — where the jumps hide
Capabilities (passible changes)	Which dependency changes are open to this company alone, given its sensing, actuation, distribution, trust and identity chain?	The company's own six directions
Identity chain	Which of the company's objects are bought for identity, and which have that chain empty?	Direction 4 candidates; disappearance risks
Frontier	Who serves the same desired states with a lighter path today?	The survival test's opponents
The jump	The dependency change that removes the most obligations for the most buyers with capabilities the company alone holds	The forecast
Leave alone	Feature additions; paths rivals already hold with a better bundle; paths the physics or the law close; paths outside the capabilities	The directions to leave alone
Disappearance risk	Which own products meet the three-condition test against a lighter embodiment?	What to retire or re-embody first

12.4 Forecast one: Versuni

- Position: the home's regulated states — cooked food, coffee, clean floors, wearable garments, breathable air, a safe home — through appliances people already own, sold under brands they trust (Section 6).
- Obligations left in place: consumables (filters, beans, water, pads), cleaning of the appliance itself, presence at the appliance, scheduling, and the interpretation of readings — the same five clusters across the whole portfolio (the dependency map of Section 11.2 in practice).
- Capabilities: sensing inside the appliance (humidity, temperature, air quality, presence), actuation at the point of use, a connected layer, brands with an identity chain in coffee and kitchen, and a distribution that reaches every home.
- The jump (Figure 50): the obligation-free loop, appliance by appliance and then across them — cooking by deadline with endpoint sensing (the deadline cooker), coffee that precedes the wish, the finishing hanger for garments, the air envelope loop, and interlocks for safety — with one shared consumable-and-cleaning service under the brand. The dependency changed is presence and scheduling; the capability is sensing already inside the products; the bundle grows with every appliance that joins the loop.
- Directions to go: cross-appliance loops (shared consumables, cleaning and scheduling as one service); the hanger as the unit of garment care; the kitchen that cooks by deadline; the air loop with ventilation and hood partners; identity in the kitchen and coffee lines, where the chain is already strong.
- Directions to leave alone: presets and modes (feature additions); a voice assistant as a feature; robot vacuums against Roborock and Ecovacs on mechanism alone (the frontier holds it); a decorative refresh with zero obligation removed; entering categories where the company has zero sensing advantage.

- Disappearance risk: the stand-alone iron (prevention at storage takes its need), the single-function kettle and toaster (absorbed into loops and multi-cavity cookers), and any product whose consumable is its business model once a rival removes the consumable.
- The next wow, forecast: the deadline cooker with the finishing hanger — two objects that remove the two daily obligations every household still pays. Falsifier: the tests of Section 6 and Section 10.15; failure thresholds as stated there.

12.5 Forecast two: Apple

- Position: the pocket computer and the objects on the body — phone, watch, earwear — with the strongest identity chain in consumer technology, a trust position on privacy, its own silicon, and a distribution that reaches the wrist and the ear (public record; product names only).
- Obligations left in place: the phone must be carried, charged daily, taken out and looked at; the screen is the interface for almost everything; attention is taxed by notifications; the user moves between applications by hand.
- Capabilities: sensing on the body (motion, heart, hearing, sight through the earwear and the watch), silicon that runs models on the device, the identity chain, trust, and a store that turns every device into a platform (compounding optionality, Section 10.2).
- The jump (Figure 51): the phone that stays in the pocket — the ear-and-eye loop, in which earwear and glasses perceive, a model on the device decides, and the person is spoken to or shown only what the desired state needs. The dependency changed is the screen as interface and the hand as the actuator; the obligations removed are taking out, looking, tapping and juggling; the identity chain is already there, which is what the earring case of Section 10.14 lacked and had to build.
- Directions to go: hearing and vision as regulated states on the body (the earwear's recent hearing-assistance features are the first step of exactly this loop); health as a loop rather than a dashboard; the device that speaks less and does more; models on the device where privacy is the bundle.
- Directions to leave alone: the headset as a trend — by the optionality test of Section 10.2 it carries a high obligation set (weight, isolation, charging, social cost) for paths that the phone already opens, and it serves a bundle few people carry daily; foldable screens (a feature addition to the screen-as-interface dependency); a chatbot as a feature rather than a loop; a car (a category where the company's capabilities remove few obligations that incumbents leave in place).
- Disappearance risk: the tablet between the phone and the laptop, and, on a longer horizon, the phone itself to the ear-and-eye loop — the company that forecasts this retires its own product on its own terms.
- The next wow, forecast: the ear-and-eye loop that lets the phone stay in the pocket for a whole day. Falsifier: 200 users, 30 days; failure if phone pickups fall by less than 40% or if task completion is worse.

12.6 Forecast three: OpenAI

- Position: the answer engine — a model that writes the answer or the draft, reached through a chat and an interface for developers, with a distribution measured in hundreds of millions of people (public record).
- Obligations left in place: the person must ask; must supply the context each time; must verify what came back; must move the output into the world by hand; and must attend to a chat window. The answer arrives; the outcome stays the person's job.
- Capabilities: the models, the developer surface, the distribution, and a growing ability to act inside tools on the person's behalf.
- The jump (Figure 52): from answering to holding a state — the silent agent that takes a desired state ("the report is filed by Friday", "the fridge stays stocked"), perceives the person's world with

consent, acts inside tools, verifies its own result, and speaks only when the state is reached or a veto is needed. The dependency changed is the ask-and-verify loop that today sits with the person; the obligations removed are asking, re-supplying context, verifying and moving; the block installed is trust, which is why verification must be inside the machine rather than left to the user.

- Directions to go: verification as the product (the machine that checks itself before it speaks); the perception side (context from the person's tools and devices, with consent, so the ask disappears); agents that hold one state well rather than a chat that does everything; the developer surface as the place where other companies' loops are built.
- Directions to leave alone: modes and features in the chat box (feature additions on the interface dependency); hardware for its own sake (a device with zero obligation removed is a carrying cost); benchmark chasing as a trend (a faster horse of scores); advertising inside answers (it installs a block on the trust dependency that the jump depends on).
- Disappearance risk: the chat interface itself — the ask is the obligation, and the company that removes it retires its own front door; and answer-only products in general, once agents hold states.
- The next wow, forecast: the agent that stays silent until the thing is done and checked. Falsifier: 100 users, 60 days on three household or work states; failure if states are held less than 80% of the time or if unverified errors exceed one per week.

12.7 Reading the three together

- The three jumps share a form: presence, asking and looking are the obligations being dissolved; the machine perceives and verifies; the person keeps veto. This is the environment loop of Section 7.2 seen from three positions.
- Each "leave alone" list contains a trend the market currently rewards. That is the point: trends reward the attribute of the current embodiment; the forecast rewards the removal of an obligation.
- Each disappearance risk is one of the company's own products. A company that runs the forecast retires its own faster horses before a rival does.

12.8 How to apply the forecast to your company, and how to build the machine

- Basis point: run the positioning dissection (Table 9) once. Its output — tickets, obligation inventory, capabilities, identity chains, frontier — is the origin from which every jump is measured; a jump is stated as obligations removed and paths opened relative to it, rather than as a share of a trend.
- Inputs to the machine (Figure 53): the product inventory and use paths of Section 11.2; the frontier facts; the capability list; and the disappearance test run on every own product each cycle.
- Cadence: the positioning dissection quarterly; the product-level machine of Section 11 continuously; the forecast published once a year with its falsifiers and revisited against the tests.
- Outputs: three lists — go, leave alone, retire — and one jump with a test card; the jump goes on the roadmap in place of the trend.
- Governance: the same three metrics as Section 11.1, plus one: the share of the roadmap that is a jump rather than a slope.
- The wow, as a rule: people want the most with the minimum — the next wow in any category is the embodiment that opens the most paths for the least obligation from the company's own position. The forecast names it; the test decides it.

Figure 47 · Trend forecast versus non-obvious forecast

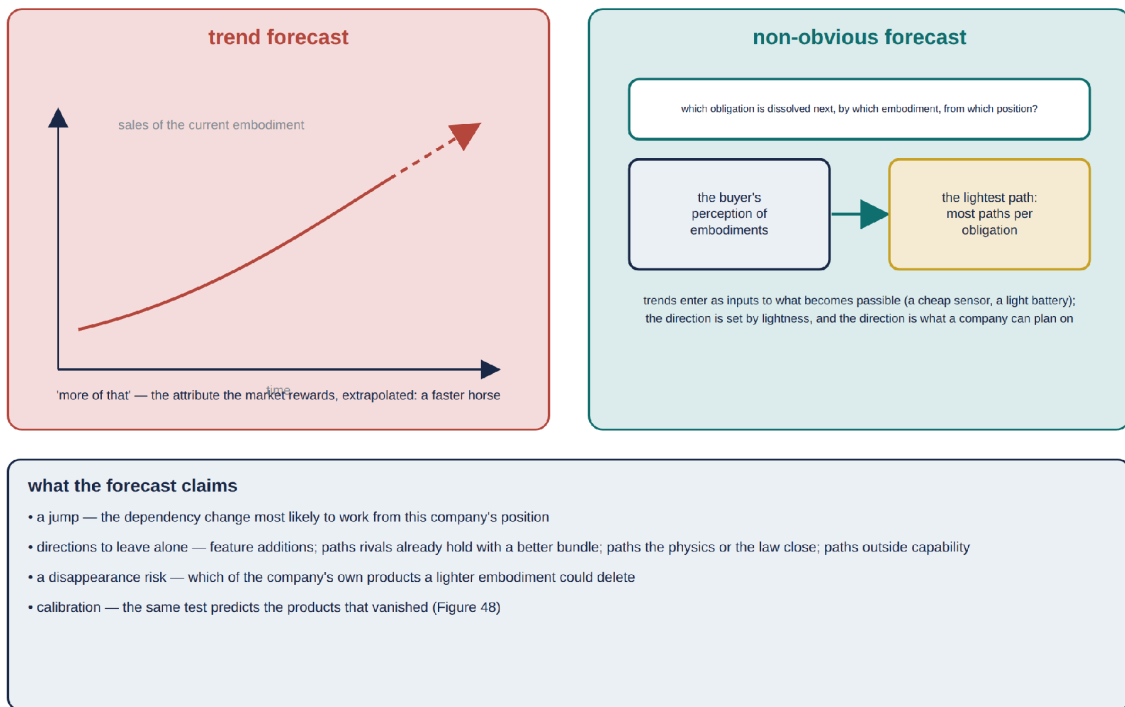


Figure 47. Trend forecast versus non-obvious forecast. A trend extrapolates the attribute the market rewards; the non-obvious forecast names the obligation dissolved next from a company's position, with trends entering only as what becomes possible. The lightest-path principle drawn as the buyer's move to the most paths at the least obligation. Conceptual.

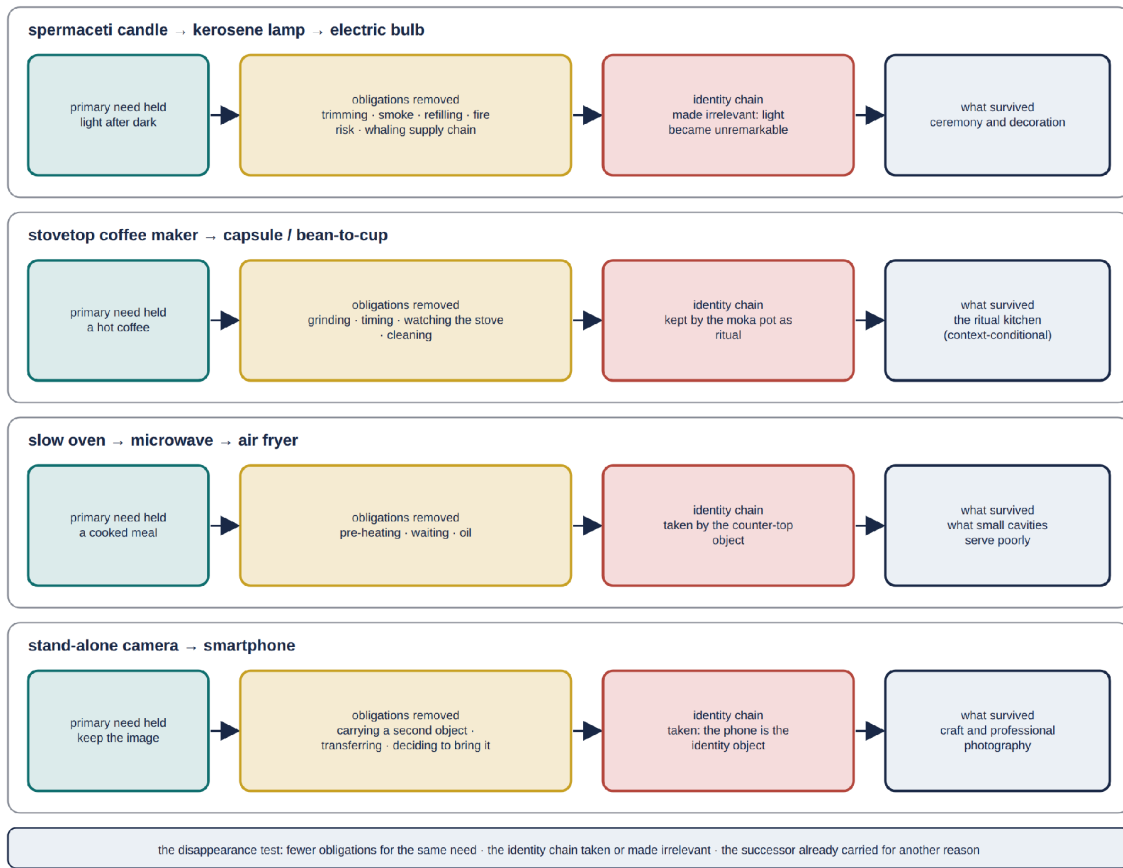
Figure 48 · Great innovations that disappeared, read through the model

Figure 48. Great innovations that disappeared. The spermaceti candle, the stovetop coffee maker, the slow oven and the stand-alone camera, each with the primary need held, the obligations removed by the successor, the fate of the identity chain, and the niche that survived. Historical, illustrative.

Figure 49 · The positioning dissection: the company card and the basis point

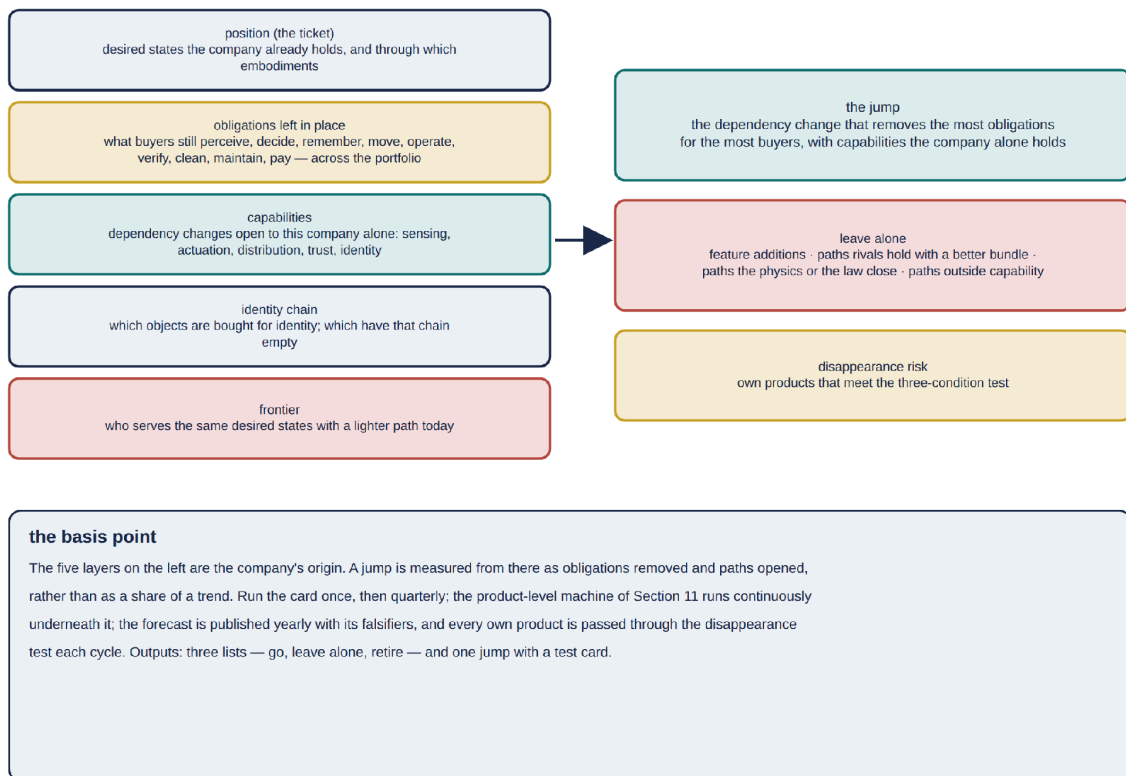


Figure 49. The positioning dissection. The company card: tickets, obligations left in place, capabilities, identity chains, frontier, the jump, the leave-alone list and the disappearance risk — the basis point from which a jump is measured. Derived from Table 9.

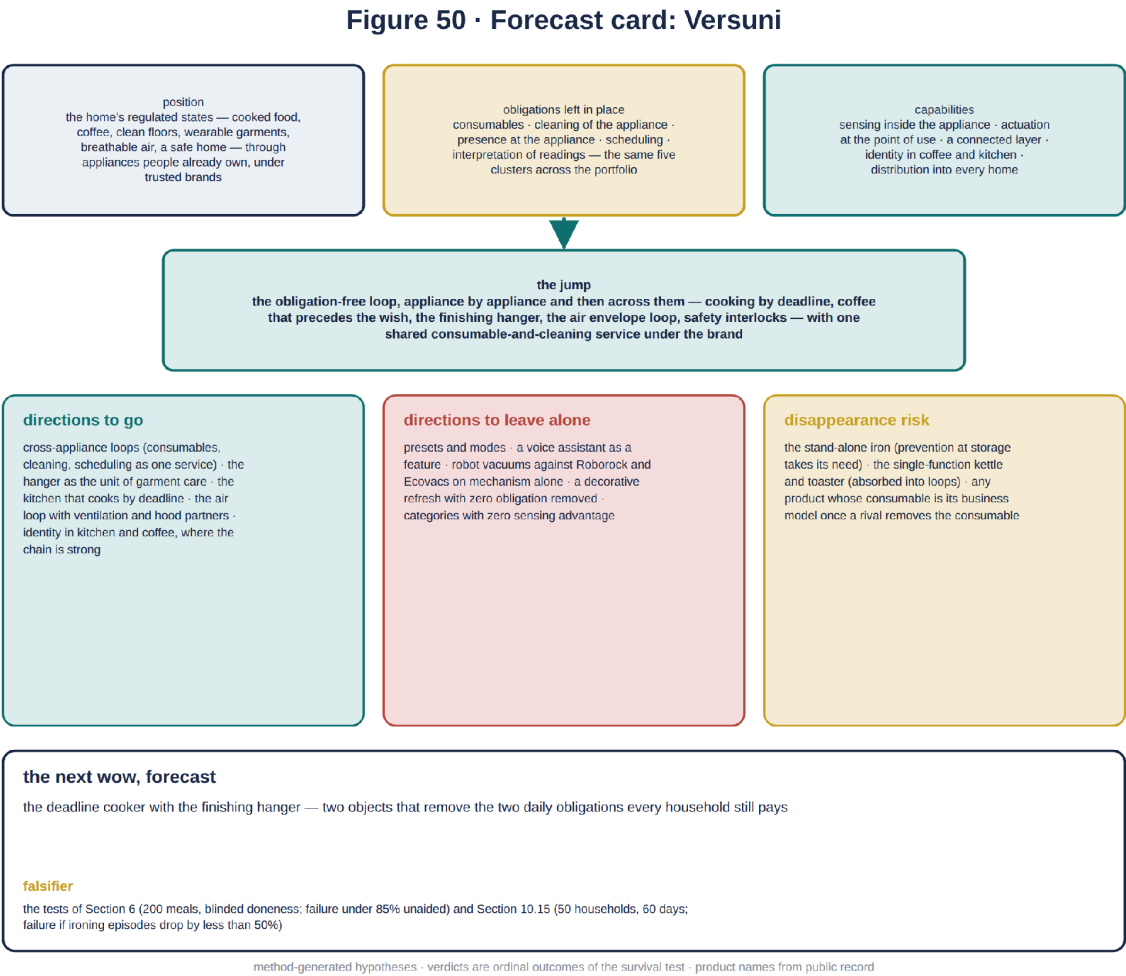


Figure 50. Forecast card: Versuni. Position, obligations left, capabilities, the jump (obligation-free loops with one consumable-and-cleaning service), directions to go, directions to leave alone, disappearance risk, the next wow with its falsifier. Method-generated hypotheses.

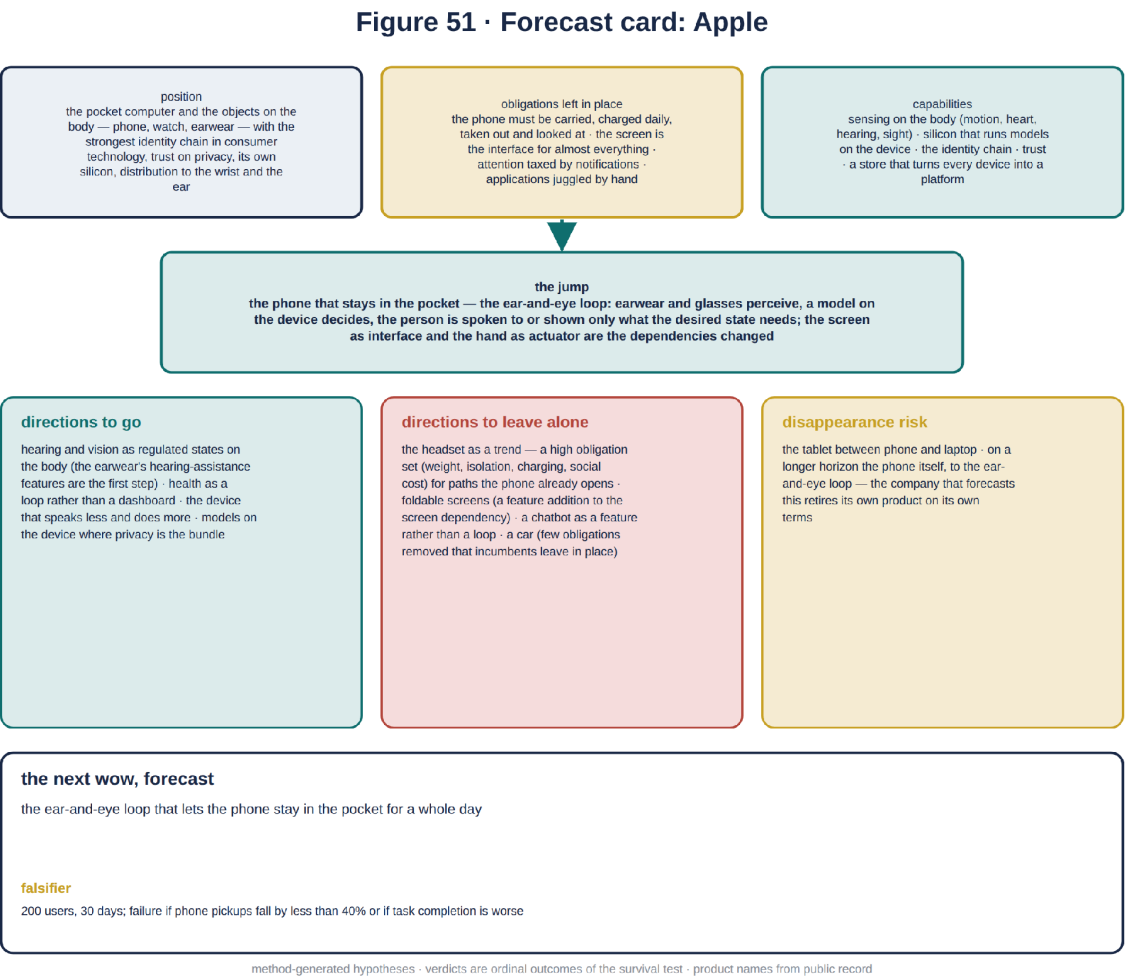
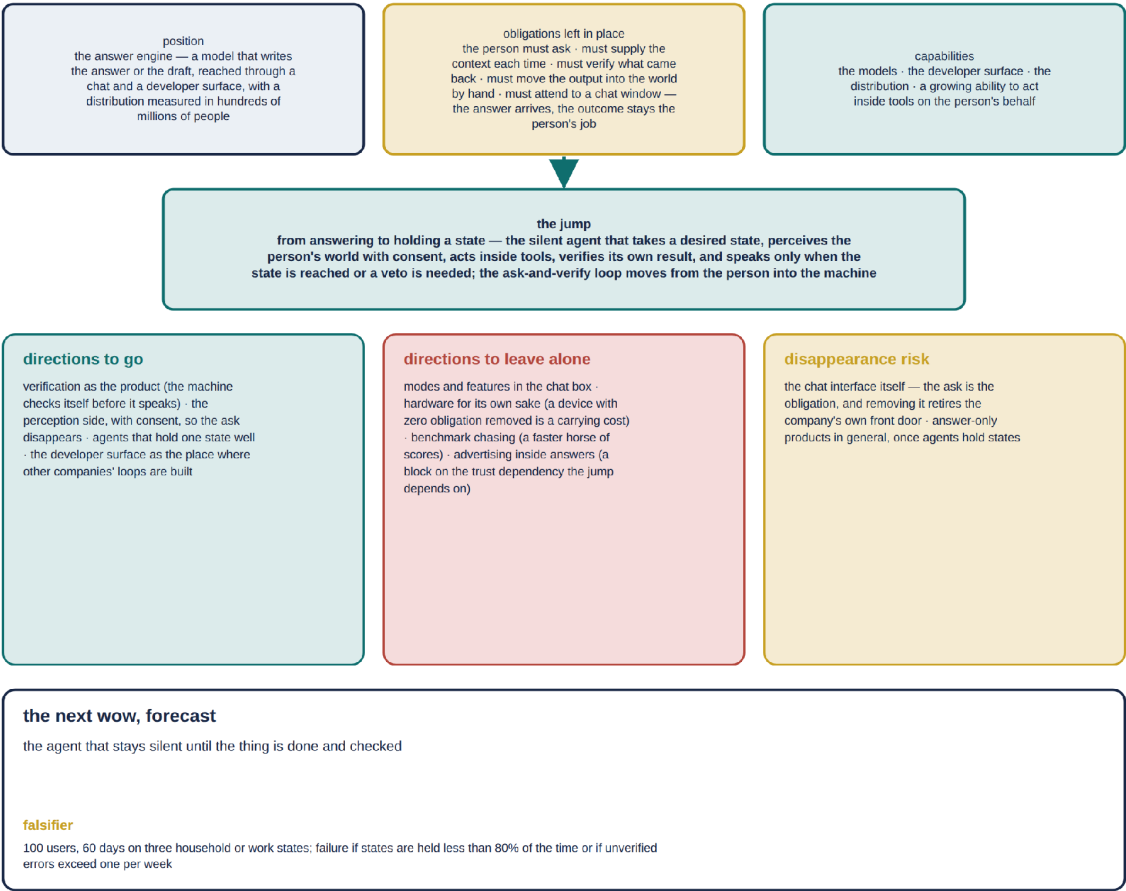


Figure 51. Forecast card: Apple. The ear-and-eye loop that lets the phone stay in the pocket; go, leave alone (the headset by the optionality test), disappearance risk, falsifier. Method-generated hypotheses; product names from public record.

Figure 52 · Forecast card: OpenAI



method-generated hypotheses · verdicts are ordinal outcomes of the survival test · product names from public record

Figure 52. Forecast card: OpenAI. From answering to holding a state: the silent agent with verification inside the machine; go, leave alone, disappearance risk (the chat itself), falsifier. Method-generated hypotheses.

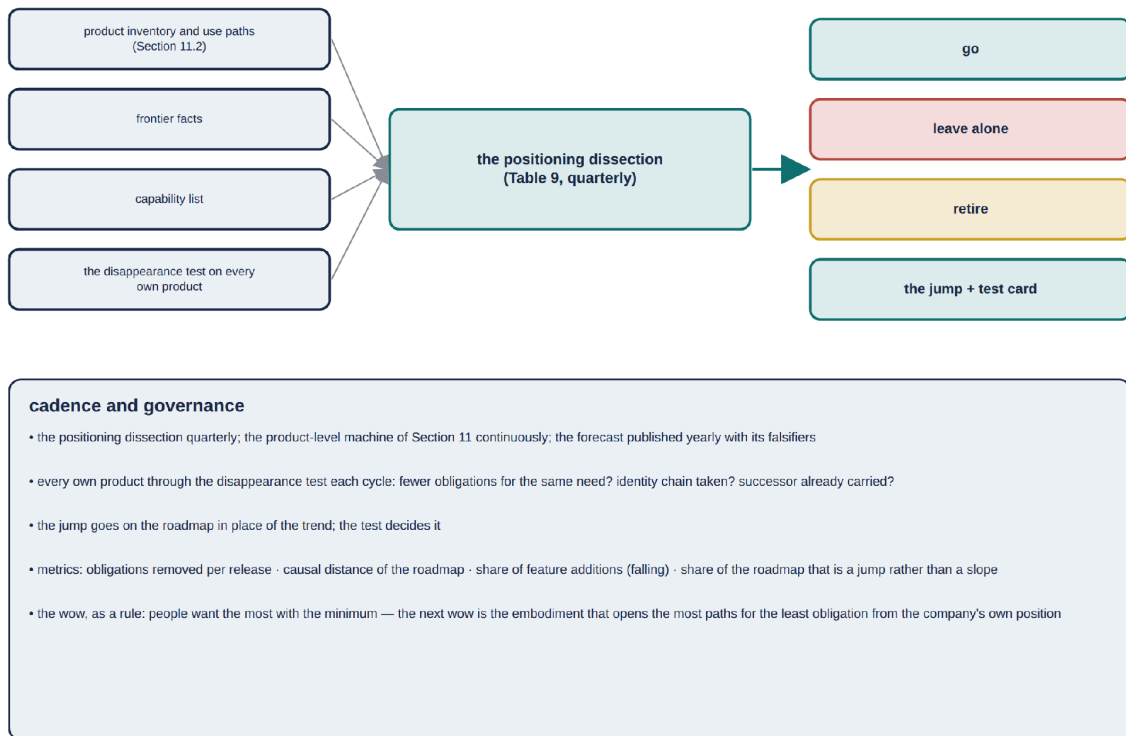
Figure 53 · The forecast machine: inputs, cadence, outputs

Figure 53. The forecast machine. Inputs (inventory, use paths, frontier, capabilities, the disappearance test), the positioning dissection, the three lists and the jump, the cadence and the governance metric. Derived from Section 12.8.

13. Conclusion

Fixation now operates inside the engines that generate ideas, which makes the procedure around generation the decisive variable. This paper proposed FPIM as a methodological synthesis that fixes the causal resolution of an innovation problem by a counterfactual survivor rule with two decidable tests, generates alternatives by intervening on the dependencies that bind a desired state to its embodiment, and evaluates candidates by what they free and block against a frontier normalized to the same desired state. The method inherits its escape operations from TRIZ, C-K and analogy, its option-preservation content from empowerment and viability, and its design object from AFI's definition of Freedom as path availability; it adds a stopping rule, a path-creation operation and a survival test, and it represents innovation as a change in the reachable set after a relational intervention. Run on six verified product families of one manufacturer, it generated four method-generated hypotheses and returned two honest losses. Whether the procedure raises causal distance and diversity at equal usefulness against strong baselines is the question the pre-registered protocol answers; until it runs, the method's standing is that of a testable proposal with stated failure conditions.

References

- [1] Jansson, D. G., & Smith, S. M. (1991). Design fixation. *Design Studies*, 12(1), 3–11. [https://doi.org/10.1016/0142-694X\(91\)90003-F](https://doi.org/10.1016/0142-694X(91)90003-F)
- [2] Crilly, N., & Cardoso, C. (2017). Where next for research on fixation, inspiration and creativity in design? *Design Studies*, 50, 1–38. <https://doi.org/10.1016/j.destud.2017.02.001>
- [3] Song, Y., Zheng, C., Jing, Q., Hansen, P., Sun, L., & Chen, L. (2025). Understanding design fixation in generative artificial intelligence. *Proceedings of the ASME IDETC/CIE 2025*, DETC2025-168630. <https://doi.org/10.1115/DETC2025-168630>

- [4] Doshi, A. R., & Hauser, O. P. (2024). Generative AI enhances individual creativity but reduces the collective diversity of novel content. *Science Advances*, 10(28), eadn5290. <https://doi.org/10.1126/sciadv.adn5290>
- [5] Meincke, L., Nave, G., & Terwiesch, C. (2025). ChatGPT decreases idea diversity in brainstorming. *Nature Human Behaviour*, 9, 1107–1109. <https://doi.org/10.1038/s41562-025-02173-x>
- [6] Wenger, E., & Kenett, Y. N. (2026). Large language models are homogeneously creative. *PNAS Nexus*, 5(3), pgag042. <https://doi.org/10.1093/pnasnexus/pgag042>
- [7] Ma, K., Grandi, D., McComb, C., & Goucher-Lambert, K. (2025). Do large language models produce diverse design concepts? A comparative study with human-crowdsourced solutions. *Journal of Computing and Information Science in Engineering*, 25(2), 024501. <https://doi.org/10.1115/1.4067332>
- [8] Feng, W. B., Hélie, S., & Panchal, J. H. (2025). Enhancing design concept diversity: multi-persona prompting strategies for large language models. *Design Science*, 11, e52. <https://doi.org/10.1017/dsj.2025.10037>
- [9] Liu, S., Shen, Y., Zhang, Y., Hou, Z., Wang, X., Luo, J., & Zhang, Z. (2026). iDesignGPT enhances conceptual design via large language model agentic workflows. *Nature Communications*, 17, 1997. <https://doi.org/10.1038/s41467-026-68672-1>
- [10] Altshuller, G. (1999). *The Innovation Algorithm: TRIZ, Systematic Innovation and Technical Creativity*. Technical Innovation Center. ISBN 978-0-9640740-4-0
- [11] Hatchuel, A., & Weil, B. (2009). C-K design theory: an advanced formulation. *Research in Engineering Design*, 19(4), 181–192. <https://doi.org/10.1007/s00163-008-0043-4>
- [12] Linsey, J. S., Markman, A. B., & Wood, K. L. (2012). Design by analogy: a study of the WordTree method for problem re-representation. *Journal of Mechanical Design*, 134(4), 041009. <https://doi.org/10.1115/1.4006145>
- [13] Ulwick, A. W. (2002). Turn customer input into innovation. *Harvard Business Review*, 80(1), 91–97.
- [14] Pearl, J. (2009). *Causality: Models, Reasoning, and Inference* (2nd ed.). Cambridge University Press. <https://doi.org/10.1017/CBO9780511803161>
- [15] Woodward, J. (2003). *Making Things Happen: A Theory of Causal Explanation*. Oxford University Press. <https://doi.org/10.1093/0195155270.001.0001>
- [16] Klyubin, A. S., Polani, D., & Nehaniv, C. L. (2005). Empowerment: a universal agent-centric measure of control. 2005 IEEE Congress on Evolutionary Computation, 1, 128–135. <https://doi.org/10.1109/CEC.2005.1554676>
- [17] Salge, C., Glackin, C., & Polani, D. (2014). Empowerment — an introduction. In M. Prokopenko (Ed.), *Guided Self-Organization: Inception* (pp. 67–114). Springer. https://doi.org/10.1007/978-3-642-53734-9_4
- [18] Massiani, P.-F., von Rohr, A., Haverbeck, L., & Trimpe, S. (2025). Viability of future actions: robust safety in reinforcement learning via entropy regularization. *ECML PKDD 2025, LNCS*, 129–145. https://doi.org/10.1007/978-3-032-06106-5_8
- [19] Aubin, J.-P. (1991). *Viability Theory*. Birkhäuser. <https://doi.org/10.1007/978-0-8176-4910-4>
- [20] Bar, A., Zhou, G., Tran, D., Darrell, T., & LeCun, Y. (2025). Navigation world models. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR 2025)*. arXiv:2412.03572
- [21] Araujo de Lima, G., Rizzioli, R., Berzieri, D., & Favi, C. (2026). Using functional analysis for product decomposition in engineering design: insights from a literature review. *Research in Engineering Design*, 37(3), 26. <https://doi.org/10.1007/s00163-026-00492-2>
- [22] Gero, J. S., & Kannengiesser, U. (2004). The situated function–behaviour–structure framework. *Design Studies*, 25(4), 373–391. <https://doi.org/10.1016/j.destud.2003.10.010>
- [23] Jiang, S., Li, W., Qian, Y., Zhang, Y., & Luo, J. (2025). AutoTRIZ: automating engineering innovation with TRIZ and large language models. *Advanced Engineering Informatics*, 65, 103312. <https://doi.org/10.1016/j.aei.2025.103312>
- [24] Mohammadi, A., & Zeng, Y. (2025). Enhancing TRIZ through environment-based design methodology supported by a large language model. *AI EDAM*, 39, e12. <https://doi.org/10.1017/S0890060425000083>
- [25] Ma, K., Brubaker, E. R., & Goucher-Lambert, K. (2025). Simulating design theory using LLM agents: a case study of C-K theory. *Proceedings of the ASME IDETC/CIE 2025, DETC2025-169634*. <https://doi.org/10.1115/DETC2025-169634>
- [26] Suh, N. P. (2001). *Axiomatic Design: Advances and Applications*. Oxford University Press. ISBN 978-0-19-513466-7
- [27] Gibson, J. J. (1979). *The Ecological Approach to Visual Perception*. Houghton Mifflin.
- [28] Alhasan, A., Kallen, R. W., & Richardson, M. J. (2025). Intention prediction is affordance perception. *Ecological Psychology*, 37(1), 21–35. <https://doi.org/10.1080/10407413.2024.2427403>
- [29] Christensen, C. M. (1997). *The Innovator's Dilemma*. Harvard Business School Press. ISBN 978-0-87584-585-2
- [30] Wang, B., Zhao, X., Zuo, H., Song, Y., Han, J., Childs, P., & Chen, L. (2025). From analogy to innovation: a creative conceptual design approach leveraging large language models. *Advanced Engineering Informatics*, 66, 103427. <https://doi.org/10.1016/j.aei.2025.103427>
- [31] Clay, J., & Sha, Z. (2025). Paradigmatic design thinking: how generative AI changes the role of human designers. *Proceedings of the Design Society*, 5, 2571–2579. <https://doi.org/10.1017/pds.2025.10271>
- [32] Byrne, D., Hargaden, V., & Papakostas, N. (2025). Application of generative AI technologies to engineering design. *Procedia CIRP*, 132, 147–152. <https://doi.org/10.1016/j.procir.2025.01.025>
- [33] Georgieva, I., & Georgiev, G. V. (2025). Exploring the use of generative text AI in design creativity inquiries. *Computers in Human Behavior: Artificial Humans*, 6, 100219. <https://doi.org/10.1016/j.chbah.2025.100219>
- [34] Naser, M. Z. (2025). Causality, explanations, machine learning, and engineering. *Foundations of Science*, 30(4), 945–970. <https://doi.org/10.1007/s10699-025-10006-3>
- [35] Beckers, S., & Halpern, J. Y. (2019). Abstracting causal models. *Proceedings of the AAAI Conference on Artificial Intelligence*, 33(1), 2678–2685. <https://doi.org/10.1609/aaai.v33i01.33012678>

- [36] Geiger, A., Ibeling, D., Zur, A., Chaudhary, M., Chauhan, S., Huang, J., Arora, A., Wu, Z., Goodman, N., Potts, C., & Icard, T. (2025). Causal abstraction: a theoretical foundation for mechanistic interpretability. *Journal of Machine Learning Research*, 26, 1–64.
- [37] Tiomkin, S., Salge, C., & Polani, D. (2025). Process empowerment for robust intrinsic motivation. *Journal of Physics: Complexity*, 6. <https://doi.org/10.1088/2632-072X/adf2ec>
- [38] Kiefer, A. B. (2025). Intrinsic motivation as constrained entropy maximization. *Entropy*, 27(4), 372. <https://doi.org/10.3390/e27040372>
- [39] Turner, A. M., Smith, L., Shah, R., Critch, A., & Tadepalli, P. (2021). Optimal policies tend to seek power. *Advances in Neural Information Processing Systems*, 34. arXiv:1912.01683
- [40] Sen, A. (1999). *Development as Freedom*. Oxford University Press.
- [41] Zhu, F., Wu, H., Guo, S., Liu, Y., Cheang, C., & Kong, T. (2025). IRASim: a fine-grained world model for robot manipulation. *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV 2025)*, 9834–9844. arXiv:2406.14540
- [42] Hafner, D., Pasukonis, J., Ba, J., & Lillicrap, T. (2025). Mastering diverse control tasks through world models. *Nature*, 640, 647–653. <https://doi.org/10.1038/s41586-025-08744-2>
- [43] Kim, M. J., Pertsch, K., Karamcheti, S., et al. (2025). OpenVLA: an open-source vision-language-action model. *Proceedings of the Conference on Robot Learning (CoRL 2024)*, PMLR 270, 2679–2713.
- [44] Black, K., Brown, N., et al. (2025). $\pi 0.5$: a vision-language-action model with open-world generalization. *Proceedings of the Conference on Robot Learning (CoRL 2025)*, PMLR 305, 17–40.
- [45] Xiong, N., Xu, M., Chen, W., Liu, J., Zhang, C., Wang, Y., & Yang, J. (2026). Vision-language-action models for embodied artificial intelligence: a comprehensive survey. *Journal of Field Robotics*. <https://doi.org/10.1002/rob.70340>
- [46] Xiong, H., Xu, X., Wu, J., Hou, Y., Bohg, J., & Song, S. (2025). Vision in action: learning active perception from human demonstrations. *Proceedings of the Conference on Robot Learning (CoRL 2025)*, PMLR 305, 5450–5463.
- [47] Qin, L., Wang, M., Li, P., Zhou, W., & Li, H. (2025). Active perception meets rule-guided RL: a two-phase approach for precise object navigation in complex environments. *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV 2025)*.
- [48] Li, G., Tsagkas, N., Song, J., Mon-Williams, R., Vijayakumar, S., Shao, K., & Sevilla-Lara, L. (2025). Learning precise affordances from egocentric videos for robotic manipulation. *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV 2025)*. arXiv:2408.10123
- [49] Long, K., Yi, Y., Dai, Z., Herbert, S., Cortés, J., & Atanasov, N. (2026). Sensor-based distributionally robust control for safe robot navigation in dynamic environments. *International Journal of Robotics Research*, 45(2). <https://doi.org/10.1177/02783649251352000>
- [50] Gao, Z., Yang, G., Bayrooti, J., & Prorok, A. (2025). Provably safe online multi-agent navigation in unknown environments. *Proceedings of the Conference on Robot Learning (CoRL 2024)*, PMLR 270, 5466–5486.
- [51] Chandra, R., Zinage, V., Bakolas, E., Stone, P., & Biswas, J. (2025). Deadlock-free, safe, and decentralized multi-robot navigation in social mini-games via discrete-time control barrier functions. *Autonomous Robots*, 49(2), 12. <https://doi.org/10.1007/s10514-025-10194-8>
- [52] Yang, R., Chen, H., Zhang, J., et al. (2025). EmbodiedBench: comprehensive benchmarking multi-modal large language models for vision-driven embodied agents. *Proceedings of the 42nd International Conference on Machine Learning (ICML 2025)*, PMLR 267, 70576–70631.
- [53] Jiang, Y., Zhang, R., Wong, J., Wang, C., Ze, Y., Yin, H., Gokmen, C., Song, S., Wu, J., & Fei-Fei, L. (2025). BEHAVIOR Robot Suite: streamlining real-world whole-body manipulation for everyday household activities. *Proceedings of the Conference on Robot Learning (CoRL 2025)*, PMLR 305, 1246–1281.
- [54] Li, S., Guo, Y., Yao, J., Liu, Z., & Wang, H. (2025). HomeBench: evaluating LLMs in smart homes with valid and invalid instructions across single and multiple devices. *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL 2025)*. <https://doi.org/10.18653/v1/2025.acl-long.597>
- [55] Melo de Magalhães, G. (2026). *The Architecture of Freedom Intelligence: a design theory of path availability*. Zenodo. <https://doi.org/10.5281/zenodo.18636095>
- [56] Melo de Magalhães, G. (2026). *The Freedom World Model: representing what an agent can reach, what it can distinguish, and what it currently fails to see*. Programme document, deposit pending.
- [57] Melo de Magalhães, G. (2026). *What if every path was already there? On whether possibilities are ever created, and on the perception that makes them look new*. Programme document, deposit pending.
- [58] Melo de Magalhães, G. (2026). Observer-dependent navigability in swarm intelligence: a path-theoretic decomposition of performance into perception and distortion. *Preprints.org*. <https://doi.org/10.20944/preprints202603.0945.v1>
- [59] Melo de Magalhães, G. (2026). *The general law of navigability: perception as clustering of recursive possibility in the AFI*. SSRN 6721198.
- [60] Versuni (2026, February 2). *Introducing Philips Airfryer 5000 Series with Steam*. <https://www.versuni.com/newsroom/philips-airfryer-5000-series-steam/>
- [61] Versuni. *Philips Air Steam Cooker*. <https://www.versuni.com/products/philips-air-steam-cooker/>
- [62] Versuni (2024, March 19). *Get ready for real espresso in just one swipe with the new Philips Baristina*. <https://www.versuni.com/newsroom/get-ready-for-real-espresso-in-just-one-swipe-with-the-new-philips-baristina/>
- [63] Versuni (2026, April 6). *Real cappuccinos made simple with the new Baristina Latte*. <https://www.versuni.com/newsroom/philips-baristina-latte/>

- [64] T3 (2023, October). Versuni launches latest vacuum cleaner in the UK, the Philips HomeRun 7000 Series. <https://www.t3.com/news/versuni-launches-latest-vacuum-cleaner-in-the-uk-the-philips-homerun-7000-series>
- [65] Versuni (2022, June 1). Philips Domestic Appliances introduces new 3-in-1 cordless wet & dry vacuum. <https://www.versuni.com/newsroom/philips-domestic-appliances-introduces-new-3-in-1-cordless-wet-and-dry-vacuum/>
- [66] HomePage News (2026, August). Philips launches OneUp electric mop with dual-tank cleaning system. <https://www.homepagenews.com/home-housewares/philips-launches-oneup-electric-mop-with-dual-tank-cleaning-system/>
- [67] Versuni (2022, November 14). Philips Domestic Appliances launches first all-in-one air purifier with AI technology. <https://www.versuni.com/newsroom/philips-domestic-appliances-launches-first-all-in-one-air-purifier-with-ai-technology/>
- [68] Versuni. Brands. <https://www.versuni.com/about-us/brands/>
- [69] Versuni (2026, January 5). Introducing Philips OneTurn for perfect ironing and quick steaming. <https://www.versuni.com/newsroom/philips-oneturn/>
- [70] Versuni (2024). Versuni launches new Philips Home Safety 5000 series featuring world-first sensing technology. <https://www.versuni.com/newsroom/versuni-launches-new-philips-home-safety-5000-series-featuring-world-first-sensing-technology-1/>
- [71] Contreras, C. A., Rastegarpanah, A., Chiou, M., & Stolkin, R. (2025). A mini-review on mobile manipulators with variable autonomy. *Frontiers in Robotics and AI*, 12. <https://doi.org/10.3389/frobt.2025.1540476>
- [72] Orłowski, A., & Loh, W. (2025). Data autonomy and privacy in the smart home: the case for a privacy smart home meta-assistant. *AI & Society*, 40(6), 4171–4184. <https://doi.org/10.1007/s00146-025-02182-4>
- [73] Rivkin, D., Hogan, F. R., Feriani, A., Konar, A., Sigal, A., Liu, X., & Dudek, G. (2025). AIoT smart home via autonomous LLM agents. *IEEE Internet of Things Journal*, 12(3), 2458–2472. <https://doi.org/10.1109/JIOT.2024.3471904>
- [74] Duncker, K. (1945). On problem-solving (L. S. Lees, Trans.). *Psychological Monographs*, 58(5), i–113. <https://doi.org/10.1037/h0093599>
- [75] Luchins, A. S. (1942). Mechanization in problem solving: the effect of Einstellung. *Psychological Monographs*, 54(6), i–95. <https://doi.org/10.1037/h0093502>
- [76] Purcell, A. T., & Gero, J. S. (1996). Design and other types of fixation. *Design Studies*, 17(4), 363–383. [https://doi.org/10.1016/S0142-694X\(96\)00023-3](https://doi.org/10.1016/S0142-694X(96)00023-3)
- [77] Ward, T. B. (1994). Structured imagination: the role of category structure in exemplar generation. *Cognitive Psychology*, 27(1), 1–40. <https://doi.org/10.1006/cogp.1994.1010>
- [78] Wikipedia contributors (2026). Monobloc (chair). Wikipedia, The Free Encyclopedia. [https://en.wikipedia.org/wiki/Monobloc_\(chair\)](https://en.wikipedia.org/wiki/Monobloc_(chair)) (accessed 2 September 2026).

Glossary

- Actionable causal resolution** — the level of description at which a product's desired state is free of the current embodiment yet still constrains which architecture to build; chosen by the shallow and broad tests (Section 5.3).
- Agent** — whoever perceives, decides, acts or pays at a step of the use path: a person, a device, a service.
- Binding dependency** — a relation the current path relies on (energy, material, information, agent, sequence, cost, rule) that ties the desired state to the present embodiment.
- Broad test** — a level is too broad when two materially unrelated mechanisms satisfy it with identical required capabilities, so it gives zero architectural guidance.
- Bundle** — a desired state made of two or more terms that survive the probes jointly while each alone fails one (for example wanted × timed × low-obligation).
- Candidate law** — a relation proposed by a theory and open to test; distinct from a definition and from an empirical model.
- Causal distance** — the number of binding dependencies a concept changes relative to the current product; a validation measure.
- Create(p, d)** — the operation that replaces a relation d by d' so that a previously impassible path p becomes passible; written out as the ledger.
- Dependency structure (Δ)** — the full list of binding dependencies of a use path.
- Design fixation** — the reproduction of features of an exposed example, flaws included, in later concepts.
- Desired state (D*)** — what the user pays for, stated at the actionable resolution; may be an invariant, a bundle, a loop, or context-conditional.
- Distortion** — the typed divergence between a perceived path structure and the world's: omission, false relation, latency, aliasing, blockage treated as absolute.
- Embodiment (E)** — the present mechanism through which a desired state is reached; the thing a request tends to carry inside itself.
- Freedom** — path availability: the set of paths available to an agent from its present state, admitted by the world and executable by the agent's body.

Frees / blocks — the two lines of the ledger: paths that become passible after a relational change, and paths that close.

Frontier set — the company's best analogue, the best direct rival, and the best cross-category substitute, all compared at the same desired state.

Holds / moves / loses — verdicts of the survival test: matches the frontier everywhere; improves it on at least one dimension and worsens none; otherwise.

Ladder (D₀...D_n) — the sequence of desired-state descriptions obtained by deleting one embodiment term at a time.

Loop — a desired state that is a regulation ("keep the floor within a soil band") rather than an end state.

Passibility — what is reachable from the present state by this agent along an executable path; a subset of possibility.

Path — a sequence of transitions, each admitted by the world's laws and each executable by the agent.

Path creation — altering a relation or resource so that a previously impassible path becomes passible; decomposes into enablement and unblocking.

Path choice / discovery — selecting among perceived passible paths; widening the perceived set toward the world set with the world unchanged.

Perceived paths — the paths a representation carries; for a design team, the paths it can currently see.

Perception — an agent's accessible representation of relevant state and available paths; operationally, the distinctions it holds.

Possibility — what the world's laws admit from some configuration.

Probes — the eight counterfactual questions of step 2: remove, replace, bypass, prevent, delegate, context-shift, agent-shift, goal-removal.

Shallow test — a level is too shallow while its description still contains the embodiment.

Survival test — the twelve-dimension ordinal comparison of step 6.

Use path — the ordered transitions from trigger to end state when a product is used, with the agent of each.

World paths — the paths the world admits, independent of any observer.